



Real-time Car-parking Reservation system using Transfer Learning

SPOTIT

Graduation Project By

Ahmed Elhoseny Ahmed Elhoseny (CS)

Amir Hany Benyamen metry (CS)

Ahmed Gamal Mohamed Hamed (CS)

Ahmed Esmail Mohamed (CS)

Mohamed Gamal Mohamed (CS)

Ahmed Mohamed Mahmoud Abbas (IT)

Ahmed Yousef Elbarbary Eldeghdy (IT)

Ahmed Nasser Mohamed (IT)

Mostafa Mamdouh Ahmed (IT)

Moataz Mohsen Fahmy (IT)

Supervised by

DR\ Sara EL-metwally

DR\ Nehal Sakr

Graduation Project 2022 / 2023

Abstract

Finding a parking space is the most common issue a driver encounters, especially during peak hours. During peak hours, the difficulty arises as many people look around to find vacant parking spaces at once, which causes many negative impacts on cities and drivers themselves, such as pollution, traffic congestion, traffic accidents, waste of time and fuel, emotions, and so on. Nowadays, parking is much costlier and time-consuming process in almost every big city, all over the world. The issue is that, the user couldn't find available parking space at time, and it found that cars may seek very limited parking space which causes severe traffic congestion. As a solution, a smart parking system exists to equip parking lots with many different sensors to automatically detect free parking spaces that would guide drivers to find the nearest parking space as efficiently as possible. but this solution uses embedded sensors that are expensive for installation and inefficient. our project consists of the design of an integrated E-parking management system for easier and quicker parking of vehicles, and consequently: a better saving of resources such as fuel, time and space. The project is creating a mobile app to first inform drivers of the status of parking slot and show them the nearest way to it. Another functionality is to give users the capability to book idle parking slots on the road to the parking. The state of a parking is detected in real-time using a machine learning model from frames captured (from video) by a limited resources camera and then sent to a real-time database which can be consulted and used to reserve idle parking slots.

Acknowledgement

In completing this graduate project, we have been fortunate to have help, support and encouragement from many people. We would like to acknowledge them for their Cooperation. First, of all, thanks are due to ALLAH for getting this work done. Second, we would like to express our appreciation to our supervisors

DR\ Sara ELmetwally

DR\ Nehal Sakr

We are incredibly grateful to our two doctors for their unwavering support and invaluable assistance during our graduation project. Their guidance and expertise have been instrumental in our success, and we cannot thank them enough for their dedication and encouragement. Their mentorship has not only enriched our academic journey but also inspired us to reach new heights. We are truly fortunate to have had the privilege of working with such remarkable individuals who have made a lasting impact on our lives.

Eng. Ahmed yosri

If we gather all words of thanks they are few for what you did with us, you acted like a one of us not as a teacher teaching for our graduation only, you were a friend of ours writing things in project with your hand not just saying notes, you were always beside us in pressure and problems, really the words can't describe what you did for us we hope that you always be as great as you are, thanks for encouraging and being with us all time, let the lord be with you on your way to success.

Table of Contents

Chapter 1: Introduction.

1.1 Introduction.....	
1.2 Problem Definition.....	
1.3 Project Objectives.....	
1.4 Project Scope.....	
1.5 Project Timeline.....	
1.6 Document Organization.....	

Chapter 2: Literature Review.

2.1 Literature Review	
2.2 Summary.....	

Chapter 3: System Analysis.

3.1 System Requirements..... .	
3.1.1 Functional Requirements	
3.1.2 Non-Functional Requirements	
3.2 System Architecture.....	
3.3 Development Methodology.....	
3.3.1 Use Case Diagrams	
3.3.2 Business Model Canvas	
3.3.3 Sequence Diagram	
3.3.4 database vision	
3.3.5 Class Diagram	
3.3.6 context Diagram	
3.3.7 flowchart Diagram	
3.4 Tools and Languages.....	
3.5 Summary.....	

Chapter 4: System Design.

- 4.1 Software Application Design.....
- 4.2 UI in Mobile Application.....
- 4.3 UX in Mobile Application.....
- 4.4 User Interface Screens (Mobile APP)

Chapter 5: System Implementation.

- 5.1 Steps of implementation.....**
 - 5.1.1 Mapping design to sample code.....
 - 5.1.2 System testing.....
- 5.2 implementation**
 - 5.2.1 Convolutional Neural Networks
 - 5.2.2 Adam Optimizer and Learning Rate Decay
 - 5.2.3 Dataset
 - 5.2.4 Data Preparation
 - 5.2.5 Our proposed architecture of the CNN model
 - 5.2.6 Deployment of the proposed solution in a real scenario within the same parking space
 - 5.2.7 Real Implementation on a camera
- 5.3 Mapping Design To implementation**
 - 5.3.1 Implementation to Flutter
 - 5.3.2 Parkings Api Routes
 - 5.3.3 Search algorithm
 - 5.3.4 Gate Scanner
- 5.4 Testing**

Chapter 6: Conclusion and Future work.

- 6.1 Conclusion.....**
- 6.2 Future Work.....**

References.....

List of Figures

Figure 1.1: Project Timeline.....	13
Figure 2.1: Confusion Matrices for Binary Classifier Based On Parking Slot Camera Feeds.....	24
Figure 3.1: System Architecture.....	30
Figure 3.2: Use Case Diagram.....	31
Figure 3.3: Business Model Canvas.....	32
Figure 3.4: Sign Up Sequence Diagram.....	33
Figure 3.5: Map Sequence Diagram.....	34
Figure 3.6: Choose Parking Sequence Diagram.....	35
Figure 3.7: Choose Slot Sequence Diagram.....	36
Figure 3.8: Payment Sequence Diagram.....	37
Figure 3.9: Database Vision.....	38
Figure 3.10: Class Diagram.....	39
Figure 3.11: Context Diagram.....	40
Figure 3.12: Flowchart Diagram.....	41
Figure 4.1: Splash Screen.....	45
Figure 4.2: Introduction Page.....	46
Figure 4.3: Login Page.....	47
Figure 4.4: Sign Up Page.....	47
Figure 4.5: Home Page.....	48
Figure 4.6: Parking Reservation Page.....	48
Figure 4.7: Payment Page.....	49
Figure 4.8: QR Code Page.....	49
Figure 4.9: Sidebar page.....	50
Figure 4.10: Profile Page.....	50
Figure 4.11: user parking history page.....	51

Figure 4.13: change password page.....	52
Figure 5.1: The different image variation that are considered as a challenge in front of manual feature extraction from images	55
Figure 5.2: Example for a hierarchy of features that are detected by a CNN	56
Figure 5.3: The sliding patch over the image and outputting another 2D weighted feature	56
Figure 5.4: An example of convolution layers and maxpooling layers acting on an input image	57
Figure 5.5: A complete CNN architecture with both the feature learning first and the classifier next	57
Figure5.6. Images captured under weather conditions different.....	59
Figure 5.7: Training set patches segmented from the camera view. Images show four parking spaces in both status: busy (first row) and free (second row). They also present some occlusion and shadow situations that we faced.....	60
Figure 5.8: Parking space patches are segmented and numbered as shown in the images. Top images belong to the small dataset CNRPark, while bottom images belong to the full CNRPark-EXT dataset.....	60
Figure 5.9: function which we used for drawing rectangles around each slot.....	62
Figure 5.10: function which we used to enable zooming mode.....	62
Figure 5.11: Original folder structure of the segmented images folder in the PKLot Dataset.....	63
Figure 5.12: function which we used for splitting the whole data into train and test.....	63
Figure 5.13: Original folder structure of the segmented images folder in the CNRPark-EXT Dataset.....	64
Figure 5.14: function which we used for extracting images from the main folders.....	64
Figure 5.15: Data Generation Function.....	64
Figure 5.16: Summary of our CNN Model Architecture.....	67

Figure 5.17: Training accuracy vs validation accuracy when training after the first 13 epochs.....	67
Figure 5.18: Training loss vs validation loss when training after the first 13 epochs.....	68
Figure 5.19: Training accuracy when training after the second 30 epochs.....	68
Figure 5.20: Training loss when training after the second 30 epochs.....	69
Figure 5.21: Confusion matrix of the testing experiment of the model trained on CNR dataset	69
Figure 5.22: Confusion matrix of the testing experiment of the model trained on CNR-EXT dataset.....	69
Figure 5.23: Confusion matrix of the testing experiment of the model trained on PKLot dataset.....	70
Figure 5.24: The ROC of the testing experiment of the model trained on multiple parking.....	70
Figure 5.25: Images captured under different weather conditions from Parking Lot Occupancy Database.....	72
Figure 5.26: Each Raspberry Pi is mounted inside a outdoor camera box (Figure A on the left) and it is mounted on top of the roof of the building, attached to a steel pole (Figure B on the right).....	73
Figure 5.27: Example of classification of a portion of the parking lot.....	74

List of Tables

Table 1.1: functional requirements for System.....	28
Table 1.2: Non-functional requirements for System.....	29



SPOTIT

Chapter 1

Introduction

1.1 INTRODUCTION

Parking can be a daily struggle for many people, as in attempting to find a nearby yet affordable space to park for work in an office building (social needed) It's also a problem during occasions, as hundreds gather around a few blocks or streets for a mall or a festival. Inefficiencies like drivers hovering over to find a free space for their cars can be frustrating to drivers and lead to even more traffic congestion. By bouncing between lots and garages that are full due to lack of immediate awareness of where spaces are open, drivers become restless and may choose to park illegally or leave altogether, creating a loss for the city in more ways than one.

Not only causes loss for city from traffic congestion, wasted time and fuels, and increases the carbon emissions released to the environment, causing pollution and damage to the ecosystem.

The straightforward solution for a such a problem is to offer the driver a reliable, real-time data that allows him/her to check whether on-street parking, surface slots or garages area available or not. These solutions (E-parking) bring to light the problem with existing infrastructure. The need is to establish a highly functional and efficient parking management solution that ensures resident satisfaction and utilizes the existing parking slots and on-street parking throughout the city.

A smart camera is a vision system that has an image capture circuitry and enough computing power to process and extract application-specific information from the captured images. Smart cameras are also able to generate event descriptions or make decisions that are used in intelligent and automated systems. Recently there has been a growing interest in developing smart camera solutions able to detect parking lot occupancy. The approach that we propose performs this task in real-time directly on smart cameras, without using a central server. It is a decentralized, effective, efficient, and scalable approach, based on deep learning techniques. It relies on a deep Convolutional Neural Network (CNN) specifically designed to be executed on smart cameras. The clear advantages of the decentralization are the reduction of the communication overhead and the elimination of computing bottleneck.

As a consequence, the system scales better when the number of monitored parking spaces increases.

In this chapter, we will introduce our proposed project. First, Section 1.2 discusses the motivations and goals of our project respectively. Next one, In Section 1.4, we present the intended project scope. Then Section 1.5 presents the timeline from our project. Finally, Section 1.6 provides an organization of the document that includes the summary: a description of the context of each chapter.

1.2 Problem Definition

With the high percentage of vehicle ownership in Egypt, parking has become a conflicting and confusing situation for several people. Whether at an airport, bus stations and shopping centers, problems with parking are an everyday occurrence. Lack of accessible parking can hurt local businesses and decrease the quality of life for residents.

- **A shortage of legit parking lots and a high percentage of off-street parking.**
- **Searching for a parking spot, instead of moving towards one.**
- **Traffic congestion & accidents**
- **Insecurity problem**
- **Low parking cost**

The cost paid for parking in the wrong place is low and there is not adequate punishment for illegal parking. Several illegal cars pay almost nothing for parking in the wrong place. Inadequate punishment is one of the main reasons for illegal parking.

- **Inefficient use of existing parking capacity**
- **Parking spaces that are an inconvenience**
to nearby residents and businesses. Businesses may have trouble retaining customers and residents may have a problem finding parking close to their homes.
- **Difficulties with parking regulation and pricing**

This problem can cause problems, including traffic congestion as motorists stop in a traffic lane to wait for a space.

- **Waste of time, fuel, emotions**
- **Inconvenient parking options** Parking within a reasonable walking distance (10 M) is hard to find during specific times of the day.
- **Environmental degradation**

Parking lots accumulate a lot of pollutants that do not get absorbed and are thus flushed into water bodies during the rains. They also contribute to the production of sulfur dioxide and particulate matter, which have adverse health effects.

1.3 Project Objectives

The main goal of this project is to develop a cost-effective and AI-assisted parking slot detection system that provides high accuracy while requiring lower setup and maintenance costs compared to the existing solutions. The application will detect vacant parking spaces in a parking lot and guide the driver (using a mobile Application) to available parking spots.

As for the potential impacts of this project, the initiative will provide significant advantages to drivers, parking slot operators, and government/road operators. Potential benefits of our developed smart parking systems are:

- From the end-user standpoint, it provides significant conveniences - saving time, enhancing productivity, and providing economic gain.
- An efficient, better controlled and monitored parking system increases the possibility for additional revenue from the parking operator's standpoint. System's available parking places are promptly disseminated in real-time to automobiles seeking parking. In comparison to other sensor-based systems, this platform is more cost-effective than other solutions. Additionally, the suggested technology has minimal ongoing maintenance expenses and is highly scalable.

- From the standpoint of the government/municipality/road operator, the approach is environmentally friendly since cars spend less time on the road waiting for parking, decreasing road congestion.
- Users of the application can also easily access and view the nearby available parking slots and be navigated to the reserved spot using the phone.

1.4 Project Scope

The scope of my project is to design and implement the functionality of The Development of Parking System on a software system parking garage.

our application the user can view various parking slots and check for the availability of slots. Whenever a user books a particular slot, it will be marked red, and all the available slots will be green. Booking can be done through credit card/net banking. This application also provides an additional feature of canceling the booked slot within 20 minutes from the time of booking. If the user fails to reach the destination on time, then the reservation will be cancelled. On successful payment a QR Code is sent to the user. Hence this application reduces the user's effort and time of searching for the parking slot and avoids conjunction of traffic

1.5 Project Timeline

ALL	TASKS	DURATIONS	START	FINISH
1	Complete project execution	7 months	01/10/2022	21/04/2023
2	Initiating	15 days	05/10/2022	20/10/2022
3	brainstorming	8 days	22/10/2022	30/10/2022
4	Discussing members ideas for main project	10 days	01/11/2022	10/11/2022
5	main idea of project	3 days	11/11/2022	14/11/2022
6	Analysis	25 days	15/11/2022	14/12/2022
7	Problem overview	6 days	14/11/2022	20/11/2022
8	MID-Term break	4 days	21/11/2022	25/11/2022
9	Related and previous work	5 days	25/11/2022	30/11/2022
10	Determining objectives	3 days	01/12/2022	03/12/2022
11	System architecture	4 days	05/12/2022	09/12/2022
12	UML diagrams and business model	5 days	10/12/2022	15/12/2022
13	Design	15 days	15/12/2022	29/12/2022
14	Semster final exams	20 days	30/12/2022	20/01/2023
15	Implementations	4 months	10/12/2022	25/04/2023
16	Back End	1 month	13/12/2022	13/01/2023
17	Computer vision model	2 months	25/01/2023	25/03/2023
18	Flutter	1 month	16/03/2023	16/04/2023
19	Testing	20 days	17/04/2023	08/05/2023

Figure 1.1: project timeline

1.6 Document Organization

This document consists of six chapters in addition to one appendix. A brief description about the contents of each chapter is given in the following paragraphs:

Chapter 1: introduction and the problem on which the project is based and objectives to be achieved through the project.

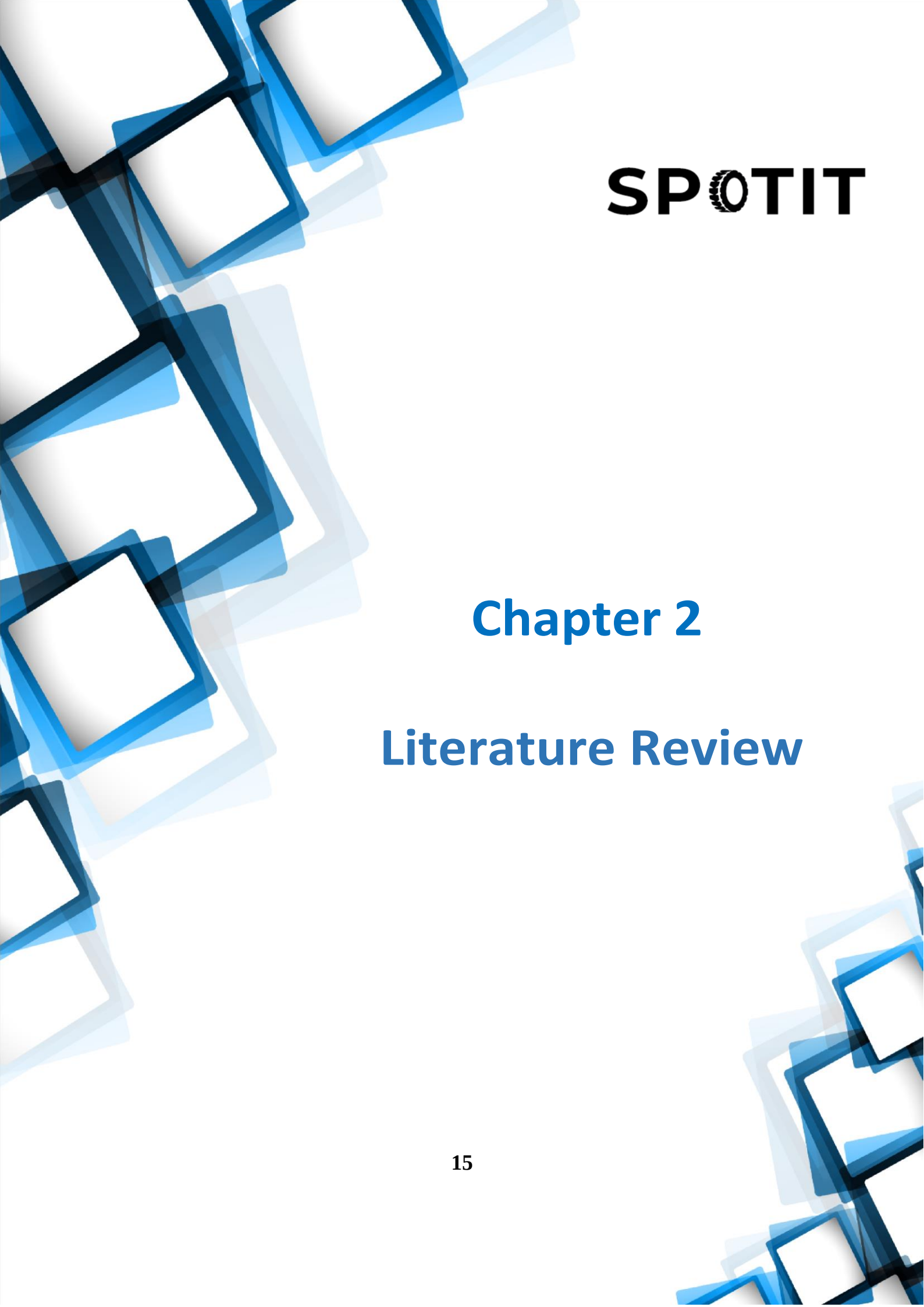
Chapter 2: Literature Review provides the reader with an overview of the previous related work, common technologies used, and the relation between our work and the relevant work.

Chapter 3: System Analysis, includes the analysis of existing systems, system requirements, use requirements, system architecture development, database design and interface design.

Chapter 4: - System Design, provides the system design including user interface and User Experience.

Chapter 5: System implementation, shows the process of mapping design into implementation, sample application code, system testing, results of the investigation, and goals achieved.

Chapter 6: Conclusion and Future Work

The background of the slide features abstract, overlapping blue geometric shapes, primarily squares and rectangles, arranged in a dynamic, non-linear pattern. These shapes are rendered in various shades of blue, from light to dark, creating a sense of depth and movement. They are positioned primarily along the left and bottom edges of the slide, framing the central text area.

SPOTIT

Chapter 2

Literature Review

In this chapter, we provide a background about the tools and techniques needed to build our system. We also review the previous related work to our system, and the common technologies that are used. In the end of the chapter, we provide a comparison between the system we are going to build and the related systems.

2.1 Literature Review

Many solutions were envisioned to tackle the congestions and emissions caused by the lack of parking lots in urban areas. One of the most effective solutions is implementing PGI systems (Parking Guidance and Information) in parking lots using the latest parking sensors and technology to identify a vacant parking space and direct drivers to the empty slot. Vacancy detection systems are an important part of PGI systems. The latter are classified based on the detection method or data input into three different types [1].

- The first type is counter-based systems. They rely on sensors at the entrance and exit of parking lots. They only provide information on the number of spots available in a parking lot. This means that the parking users do not get any further guidance on where the spaces are available. Furthermore, these systems are not compatible with on-street parking spaces, or residential parking spaces.
- The second type is wireless or wired sensor-based systems. They provide a high degree of reliability. They are applied in in-door parking lots such as shopping malls. This type is based on installing sensors in each parking space. The major drawback of this type is too expensive to implement. Most of the parks, offices, and universities have a very big area to cover for parking. The overall cost of implementing such systems will go beyond the expectations. Another major drawback is the malfunctioning of the hardware such as sensors or boards. It is very time consuming when it comes to changing the malfunctioning hardware. Its troubleshooting becomes a very big issue.
- The third type Image and video-based systems (vision-based). Such a system may be expensive because a large amount of data must be transmitted over a wireless network. However, this system can use the current CCTV cameras, which have a

lower installation cost, and is a good option for outdoor parking lots. It is possible to further classify image-based into two categories: the first is car-driven, in which an algorithm is used to detect vehicles as the object of interest. The second is space-driven, which focuses on the detection of empty spaces in the parking area. The most widely used space-driven algorithm is the background subtraction since the classifier can be made under different light conditions. However, this is not suitable for outdoor parking areas under specific climate conditions, such as foggy or rainy weather.

- Finding a parking spot has become the most rigorous task in our daily lives for quite a while. The sensor system, therefore, marked the beginning of intelligent parking. Using the internet of things, the author [2] constructs an automatic parking system that allows the user to locate the nearest accessible parking place. The architecture of the parking system consists of many components, including a centralized server, a Raspberry Pi, an image capture system, a navigation system, a display device, and a user device. Basavaraju et al. [3] demonstrate that the cloud-based IoT design incorporates a cloud service for storing information about parking spot availability and a centralized server for storing smart parking system data. Using infrared sensors, RFID (Radio Frequency Identification), and IoT. Another author [4] creates a prototype for an IoT-based car parking management system for smart cities. This prototype includes parking spot availability, IoT maintenance of a parked car database via a shared server, slot booking, and theft management. Additionally, it offers several benefits, including reduced human interaction, increased flexibility, and enhanced security. This system includes online reservations, parking entry, exit, and parking management modules. The disadvantage of this system not valid for illegal parking slot detection.
- Patil et al. [5] describe a concept that might monitor and manage cars in a parking garage by informing drivers of available parking spots and directing them to the appropriate area. The technique involves modifying the original WSN(Wireless Sensor Network) and using **RFID**(Radio Frequency Identification) is utilized for vehicle check-in and check-out that is quick, safe, and easy. and **ZigBee** (standards

based wireless technology) allows for secure data exchange. The system checks to see if the car arriving for parking is registered, and the data is saved in a database. The parking area's LCD displays give information about available parking spaces and direct vehicles toward parking. Each sensor node collects data that is handled in either a distributed or centralized manner. This technology allows for one-on-one parking but is time intensive and limits the number of concurrent check-ins and check-outs. The driver is unable to reserve a parking place using a mobile device. -The usage of RFID raises the cost.

- There are various alternative approaches where systems provide end-to-end parking through sensors, similar to valet parking. Bluetooth is a wireless technology standard for short-range data transmission that is utilized as a means. Autonomous system technology uses a mechanical device to transfer vehicles between parking lots without driver intervention. As the driver approaches the parking area, he places the vehicle on a movable platform and activates his Bluetooth to initiate the parking procedure. The Bluetooth reader maintains the user's ID in a database, and the ARM microprocessor verifies the ID against the saved digits. The parking process begins when a new vehicle is obtained; when an existing vehicle must be retrieved, the retrieval process begins [6]. These systems are expensive. Therefore, a system is introduced that provides a wireless method for remotely recognizing parking spots using a smartphone. This strategy makes parking easier to locate and pay for. If a vehicle is detected in a parking space, no action is taken; however, if the parking spot is vacant, the wireless sensor node broadcasts the location to the servers for each parking area. When a user is within 2 kilometers of a parking spot, the server locates and alerts the user of the location of the closest accessible parking space [7]. The disadvantage of this system is smartphone location tracking valid up to 2 kilometers.
- Sensor-based methods are usually very efficient, but the drawbacks of these systems are being very high maintenance, overpriced and do not work well with all weather conditions.

- Our project falls under the third type of PGI systems: camera-based vacancy detection. The latter is less costly compared to using sensors since CCTV cameras cover multiple parking spaces with one single angle in open-air parking lots. Also, the same cameras are used for both vacancy detection, and general security and surveillance [8].
- Early related work within vacancy detection designs based on camera inputs focused on image processing, handcrafted feature extractions, and training of machine learning classifiers on these extracted features. Funck et al. started by comparing reference images with input images using principal component analysis as a basis for the detection [9].
- Next works in line focused on training classifiers such as Bayesian, SVM, and Random Forest classifiers along with a multitude of image processing techniques to extract image features such as corners, edges, texture, colors, and background subtraction. The feature extraction techniques used were SIFT, ORB, BRISK, and others [8][10]. Since these methods were all based on manually engineered feature extraction, the accuracies were not higher than 90% and were highly perturbed by the change in weather conditions.
- Bin et al. [11] introduced a smart parking system based on real-time image processing using a video camera as the sensor for parking space detection. The system's structure consists of three modules: image acquisition, image pre-processing, and image detection modules. The image acquisition collects the real-time images with a resolution no less than 640×480. Subsequently, the pre-processing module computes those images. It processes them using the detection algorithm, such as converting the images into grayscale. The detection module is where the final decision is made in real-time to determine the parking space occupancy status. This system is quite old but has an accuracy of 81%.
- In [12], a parking space detection was proposed using an image processing technique with a brown-rounded image drawn at each parking lot. With the brown dots, each parking space can be identified easily. As a sensor, a video camera is used. The system is divided into five modules: initialization module,

image acquisition module, image segmentation module, image enhancement module, and image detection module. The initialization module automatically detects the location of every parking lot from the image. The acquisition module captures and stores the images from a video camera; a high-definition camera is recommended and should not be obstructed by obstacles, such as shadows, trees, and buildings. The segmentation module separates the objects from the background using image subtraction, such as converting RGB images into grayscale and differentiate pixel value by thresholding the binary image. The next phase is enhancement, where it removes the noise by using a binary morphological function and removes the pixels from unrelated objects. Finally, the detection module determines the rounded brown image on each parking space and counts the total detected brown dots before it is displayed. The disadvantages of systems [11] [12]:

- The error rate is high.
 - Not robust to different weather conditions as video camera gives an output of low-resolution images.
 - Not robust to partial occlusions.
 - The experiment is based on toys, so the accuracy is not detected.
 - As the system wasn't tested in real-time, the weather factor is not determined.
- Enríquez et al. [13] listed some of the problems of detecting parking spaces using a vision-based system:
- Image quality: for object recognition tasks, images must be in good quality to be processed. The quality is affected by insufficient lightning at night and adverse weather conditions, such as rainy or foggy weather.
 - Occlusions: the captured images are sometimes obstructed by the surrounding objects, such as trees, buildings, and object shadows.
 - Classification problem: image-based classification systems have proliferated over the years and are maturing.

The same research [13] further highlighted some of the advantages of using a vision-based system over a sensor-based one:

- Coverage: single captured image can cover dozens of parking spaces.
 - Cost: lower installation cost and maintenance cost compared to sensor-based.
 - Versatility: images obtained can be used for other purposes, such as surveillance systems.
- **Vision-based Algorithms**
- Detection of parking spaces using vision-based systems is also assisted by a series of vision algorithms. These algorithms are divided into various approaches to machine-vision as follows [13]:
- Appearance-based approach: the parking space's current appearance is compared with the vacant space's original appearance. An example technique commonly used is tailor and fine-tuning.
 - Recognition-based approach: which focuses on object recognition through machine learning algorithms to detect and classify the vehicles occupying the parking space. This approach is complicated since a wide range of objects required to be recognized.
 - Three-dimensional image processing.
 - Combined technique: image processing is used to improve image quality, avoid light variations, and classify image content using a machine learning algorithm.
 - Monitoring technique; which depends on the parking lots that need to be processed. Two available types of monitoring are a) estimating the occupancy of the entire parking slot and b) checking the vehicle's presence in each parking slot.
- The systems based on convolutional neural networks improved these results (vision-based).Amato et al. [14] defined deep learning as an artificial intelligence branch that seeks to develop techniques that enable computers to learn the complex task of perception at the human level accuracy, such as seeing and hearing. It has the advantage of providing close to human-level accuracy in vehicle and pedestrian detection, image classification, speech recognition, object detection, natural language processing, and many more. They also maintained that one of many deep learning methods, convolutional neural network is an effective method for

vision tasks. CNN is a convolutional layer capable of modeling and distinguishes the spatial correlation between the neighboring pixels better and more effective than normal, fully connected layers. The final output of CNN is the classes where the network has been trained. However, the limitation of using CNN is that its training process is costly in computation and takes a lot of time. On the other hand, the prediction phase is fast and accurate if the network has been trained.

- A smart parking system based on the CNN classifier has been implemented in [14]. A smart camera and two different datasets, PKLot and CNRPark, are used. PKLot dataset is used to train and test the parking lot occupancy detection, while CNRPark is used in two different locations with different points of view. The proposed system provides a distributed, effective, efficient, and scalable solution for real-time parking occupancy detection. As the system uses deep CNN, it is robust to disturbances, such as partial occlusions, and has a good generalization property. A low computation resource is needed for the classification phase, as it only uses Raspberry Pi modules equipped with a smart camera. The system uses a website to live view all the cameras deployed in the different parking places. After being tested in other locations and under different light conditions, the experiment result shows a very high accuracy even under partial occlusions. The test result also indicates that CNN has good generalization capabilities in predicting parking status tested on different datasets.
- Mora et al.[15] and Dhuri et al. [16] used the original VGG16 network in their approaches. Mora et al.[15] evaluated their model using the PKLot dataset, which considered camera angles, ambient conditions, and parking lot variances. Furthermore, the authors suggest a method based on a Bag of Features.
- Valipour et al. used the VGGNet-F pre-trained model on ILSVRC-2012. The authors then retrained the last layers on the Almeida et al. dataset PKLot as well to fine-tune it. Furthermore, this work goes into the details of the choice of the pre-trained model and the methods used in finetuning it. The authors used an SGD optimizer with a 0.01 learning rate. The network was trained for three thousand iterations with 128 samples per mini-batches. The work also provides a system architecture for the use of such a model in production by developing a user interface

to set the bounding boxes of parking spaces and monitor real-time these parking lots [10].

- Acharya et. al built upon Valipour et al.'s work, among others. The authors used the same PKLot dataset along with a pre-trained CNN to extract features. The innovation in this work lies in using a trained SVM as a classifier and getting the final binary output. This paper also does transfer learning on the street where the researchers live, and it is promising. The CNN-SVM combination was interesting in the sense that it complemented the weaknesses of both when used separately [8].
- Nyambal et al. also used fine-tuned pre-trained CNN. The contribution of their work is providing the process of collecting a new dataset of parking spaces images. Thus, it helps understand better how the dataset PKLot can be leveraged. The paper also compares the use of three different solvers when training the model [17].
- Nurullayev and Lee [18] proposed a Convolutional Neural Network (CNN) approach based on dilated convolutions that yielded promising results by removing pixels in the convolution kernel, enhancing the classifier's learning capacity in the pictures' global context.
- Cazamias et al. worked on parking space vacancy detection within the scope of their class project. The authors also used the PKLot dataset. They used a CNN with a simple architecture compared to the pre-trained CNNs mentioned above. Their CNN architecture had three convolutional layers with 10, 20, and 30 filters, respectively. The kernel size they used was five pixels by five. They also used batch normalization layers after each convolutional layer. They used max-pooling layers to reduce the dimensionality at the end of the three convolutional layers. Finally, they used three fully connected layers to classify the input. The paper achieved 0.9997 accuracy on the test set.

They also trained and tested the model on the three different parking lot camera feeds of the dataset to evaluate the generalization of their model [19]. The results are shown in Figure 2.1.

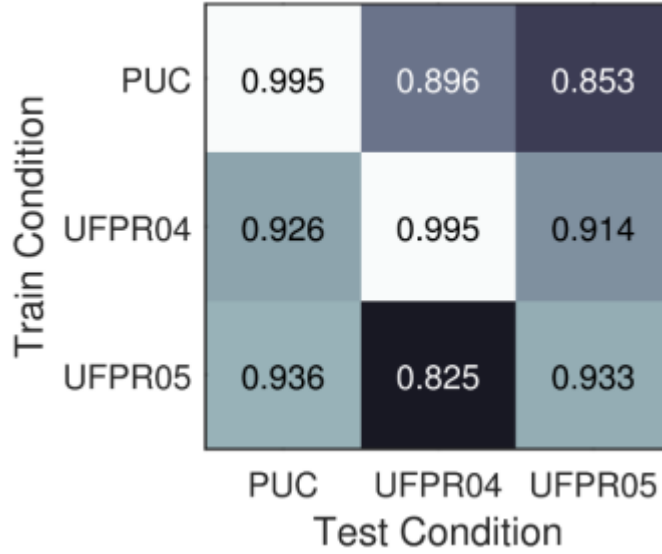


Figure 2.1: Confusion matrices for binary classifier based on parking slot camera feeds. For each result, the CNN was trained on the condition of row i then tested on the condition of column j [19].

- The last paper worth mentioning is the paper related and provided with the dataset we are using in this project. the PKLot dataset and provided with it a description of data collection methods, pre-processing, the guidance on how to constitute the training and testing sets, suggestions on evaluations metrics, and potential research directions [20].
- To summarize, the existing state-of-art works use CNN architectures to solve the problem of parking vacancy detection. Some make use of fine-tuned pre-trained models, while others either create their own architecture or use different classifiers. The PKLot dataset is the backbone dataset for vacancy detection research in the literature. In addition, many researchers collect datasets themselves to examine the transfer learning capabilities of models trained on the PKLot dataset. In this project, we will also use Almeida et al.'s dataset to design a robust and generalizable CNN model that can be used on other parking lot camera feeds without having to retrain it.

2.2 Summary:

These chapters are organized to reflect the scientific steps toward our main objective. A brief description about the contents of each chapter is given in the following paragraphs: This chapter talk about the role of information system in addition to review of literature.



SPOTIT

Chapter 3

System Analysis

Introduction

In this chapter we provide detailed knowledge about our system including functional requirements, non-functional requirements, user requirements, system architecture, use case, and sequence diagrams. We can divide the process of analysis into three parts, which are determining requirements, determining system architecture, and determining development methodology. Section 3.2, Section 3.3, and Section 3.4 will discuss those three parts. In the end of this chapter, we provide information about the tools and languages we needed to use to build our system.

3.1 System Requirements

System requirements are the needed configurations for the system to operate efficiently. The next three subsections will discuss the functional requirements, Nonfunctional requirements, and user requirements.

3.1.1 Functional Requirements

In systems engineering, functional requirements are directly concerned with the system services, where a function is described as a specification of behavior. In this subsection, we list the functions required in our system. We, also, provide a description for each function. Table 1.1 shows the functional requirements for our mental illness patients tracking system based on deep learning.

3.1.2 Non-Functional Requirements

The system needs to operate efficiently and meet the requirements. Any failure of the components of the systems may lead to one or more of the functions to stop or be misused. A non-functional requirement is a requirement that specifies criteria that can be used to judge the operation of a system. mental illness patients tracking system based on deep learning. Table 1.2 shows the non-functional requirements.

Table 1.1: functional requirements for System.

#	Function requirement	Description
1	Register	As a client I want to register to have an account on a system.
2	Login	As a client I want to be authorized to use system to reserve a slot.
3	Manage profile	As a client I can change my personal data.
4	Reservation	As a client I can reserve a slot.
5	Payment	As a client I can choose how to pay.
6	View result	As a client I get my QR code to enter the parking.

#	Non-function	Description
1	Accessibility	Easy design that is suited for different communities to use.
2	Adaptability	Suitable for different inputs like different type of cars.
3	Availability	Available for low versions of android devices as well as high versions.
4	Backup	Easy to recover outputs and deleted results as soft deleting rule is applied.
5	Capacity	Low storage space consumption.
6	Minimize response time (high speed)	System response within a few seconds.
7	Performance	High (recognition and detection) tools and algorithms.
8	Reliability and Safety	System should be safe so patient can rely on.
9	Usability	Easy to use and understand for different users and supported by a consultation system.

Table 1.2: Non-functional requirements for System.

3.2 System Architecture

After determining the requirements for a system, we will describe its main components, their relationships (structures) and how they interact with each other. Figure 3.1 shows the structure of the queue with the main components and the connections between them. When the client wants to reserve a slot, he will login to application, choose his/her location after that choose the nearest parking. He will choose a free suitable slot for his/her car based on the result coming from the camera (computer vision) and pending slots.

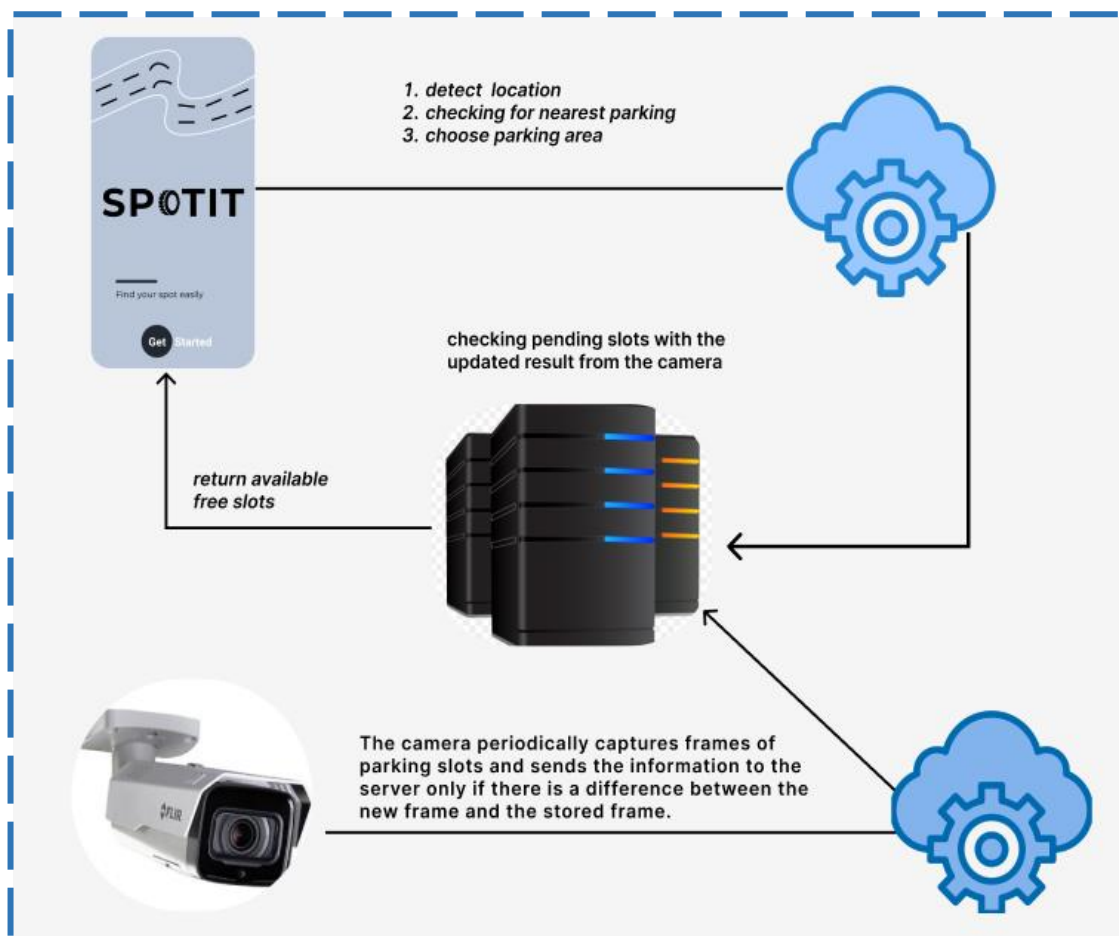


Figure 3.1: System Architecture

3.3 Development Methodology

After we knew the basic structure of the system. We are going to view all of its functions, the relation between them, and the sequence of their executions in the following subsections.

3.3.1 Use Case Diagrams

First, with a use case diagram, we will define the expected behavior (what) of the system, not the exact way to achieve it (how). This helps us design the system from the end user's perspective. Figure 3.2

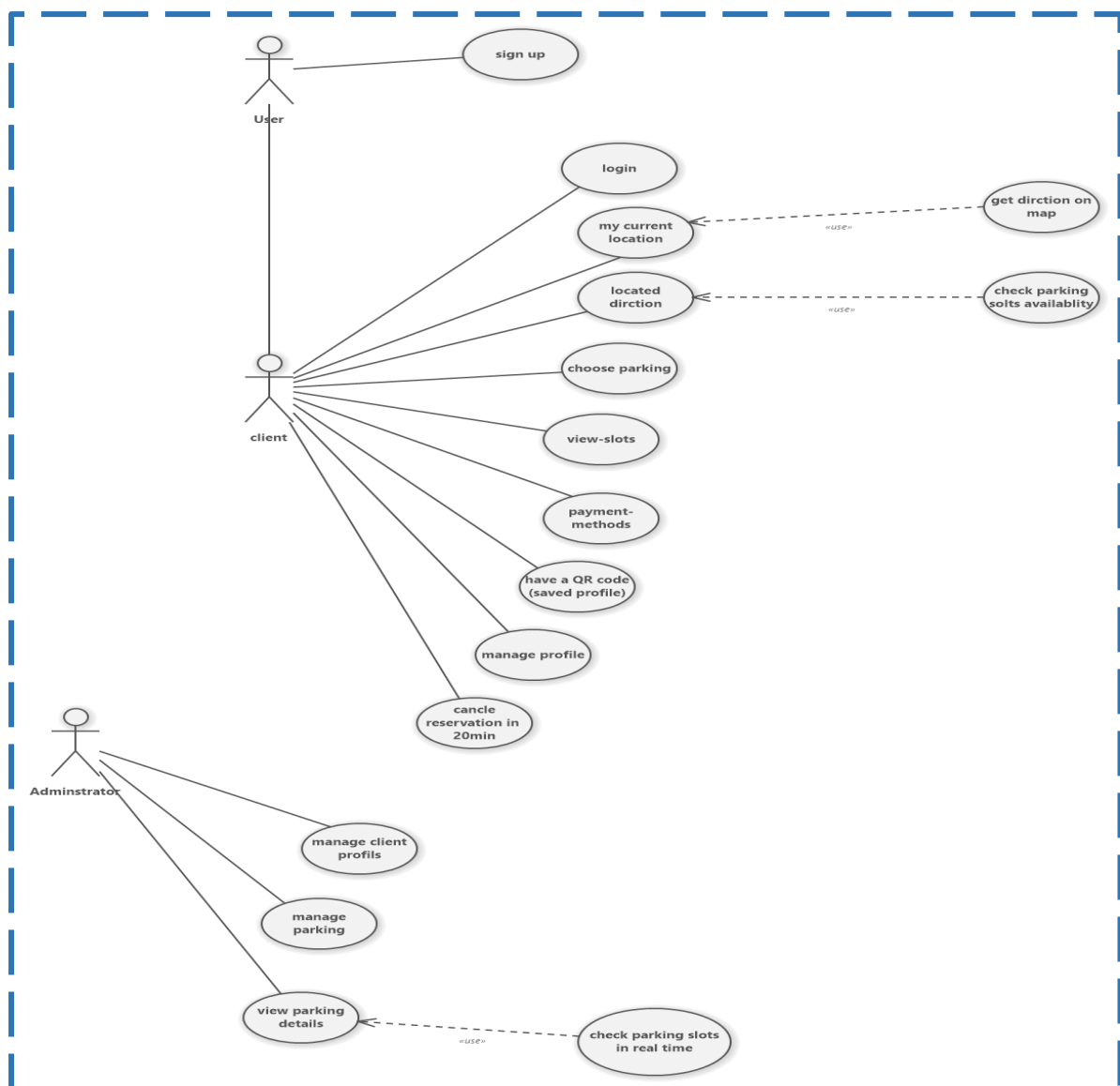


Figure3.2: use case Diagram

3.3.2 Business Model Canvas

With BMC we will study the case of business with our product according to all concepts of this model, so these 9 blocks describe cost and revenue of the project, who is the customer segment and how we communicate with them. Also, how to make them treat with us longest period possible, where we will gain our investments, what is our key resources and gain value propositions.

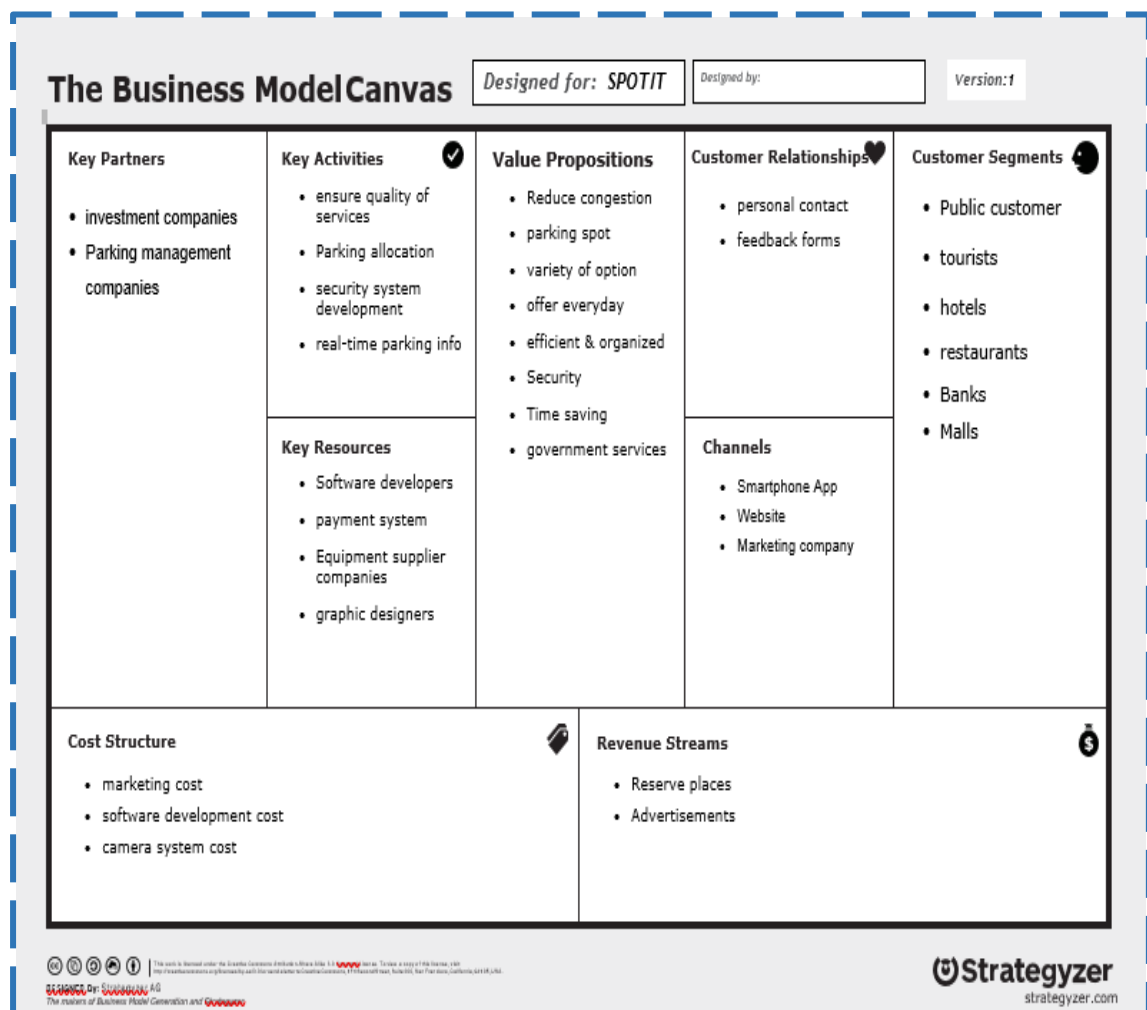


Figure 3.3: Business Model Canvas

3.3.3 Sequence Diagram

Sign UP

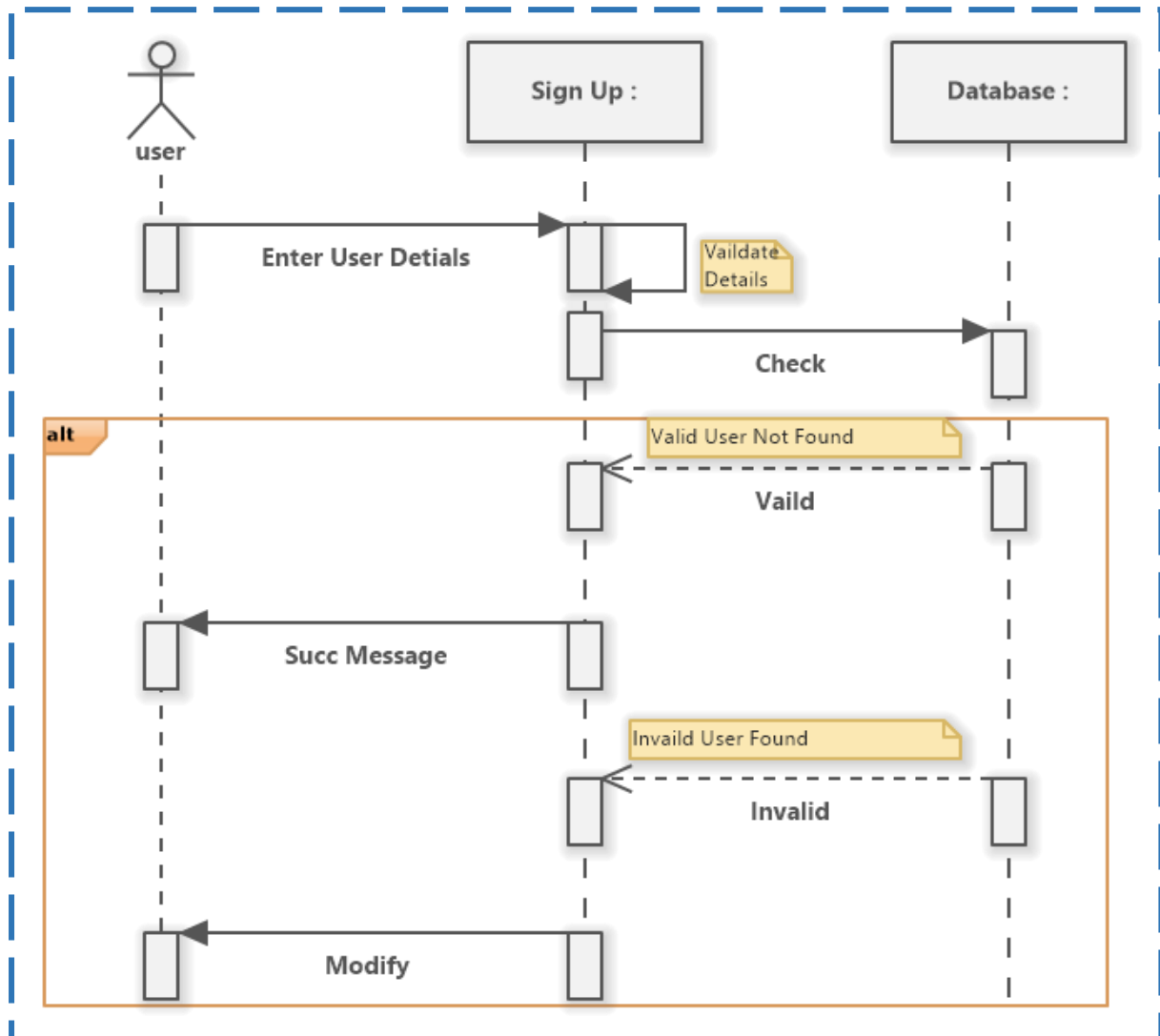


Figure 3.4: Sequence Diagram

Map sequence

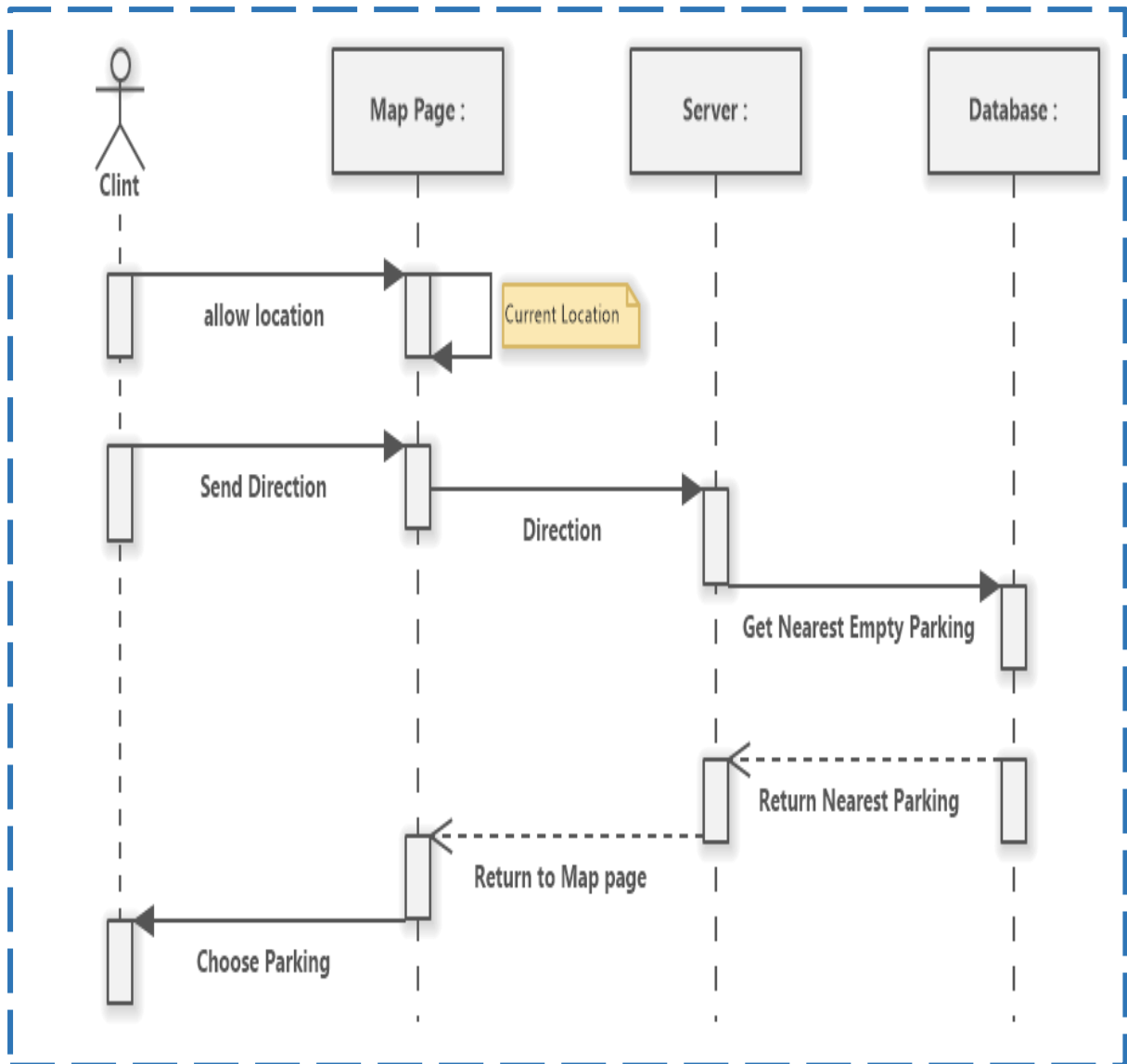


Figure 3.5: map Sequence Diagram

choose parking

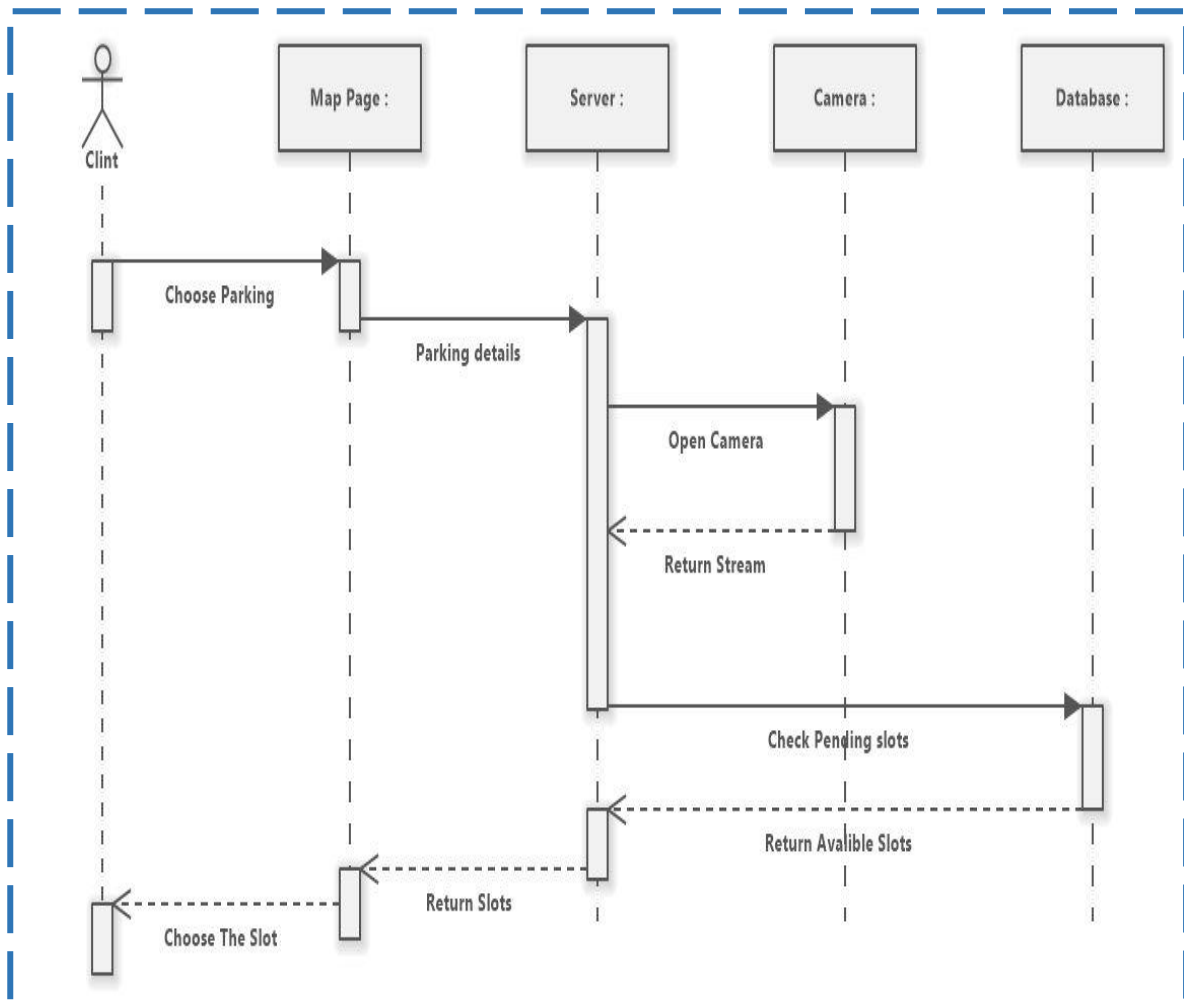


Figure 3.6: choose parking Sequence Diagram

choose slot

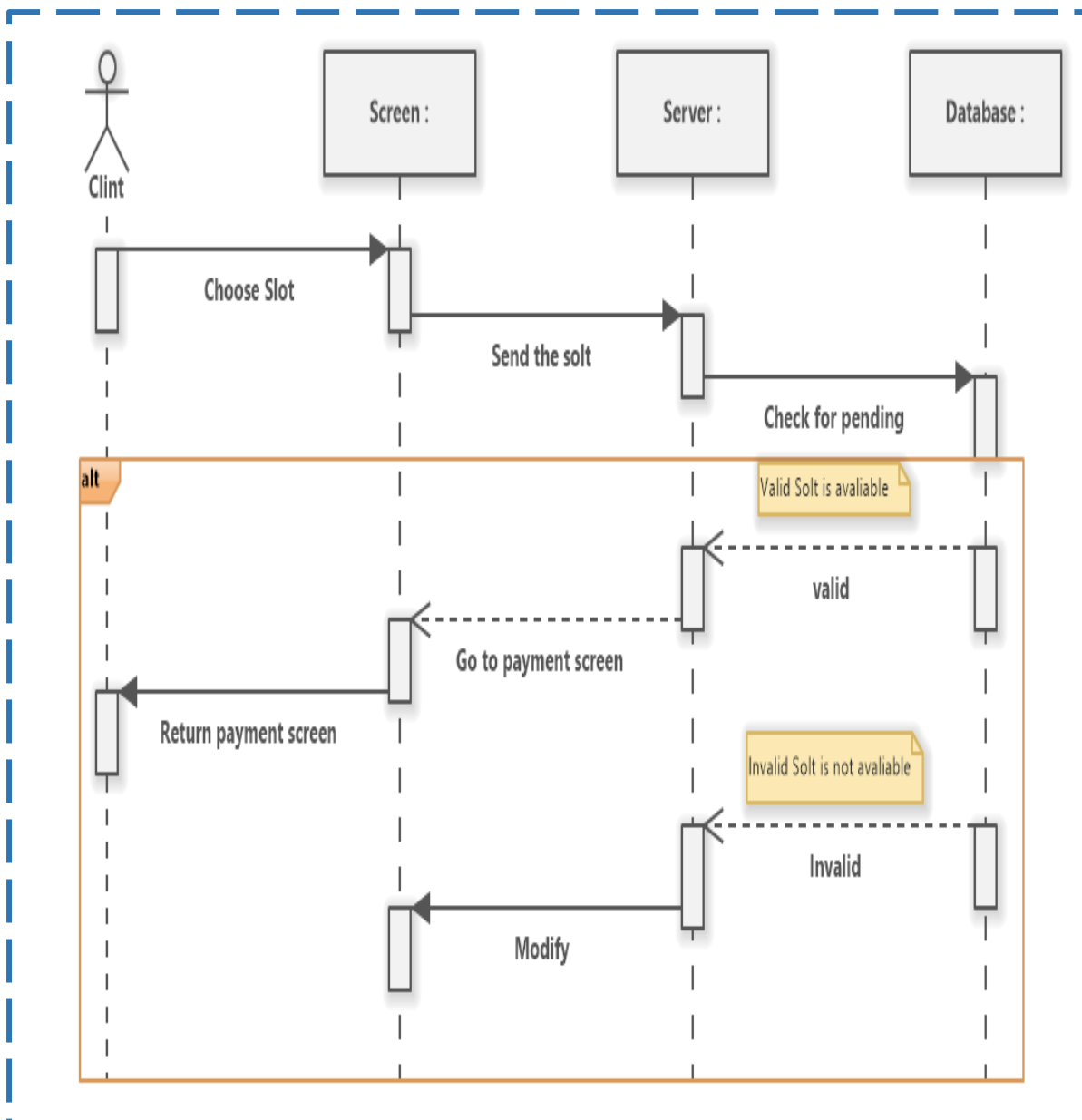


Figure 3.7: choose slot Sequence Diagram

payment

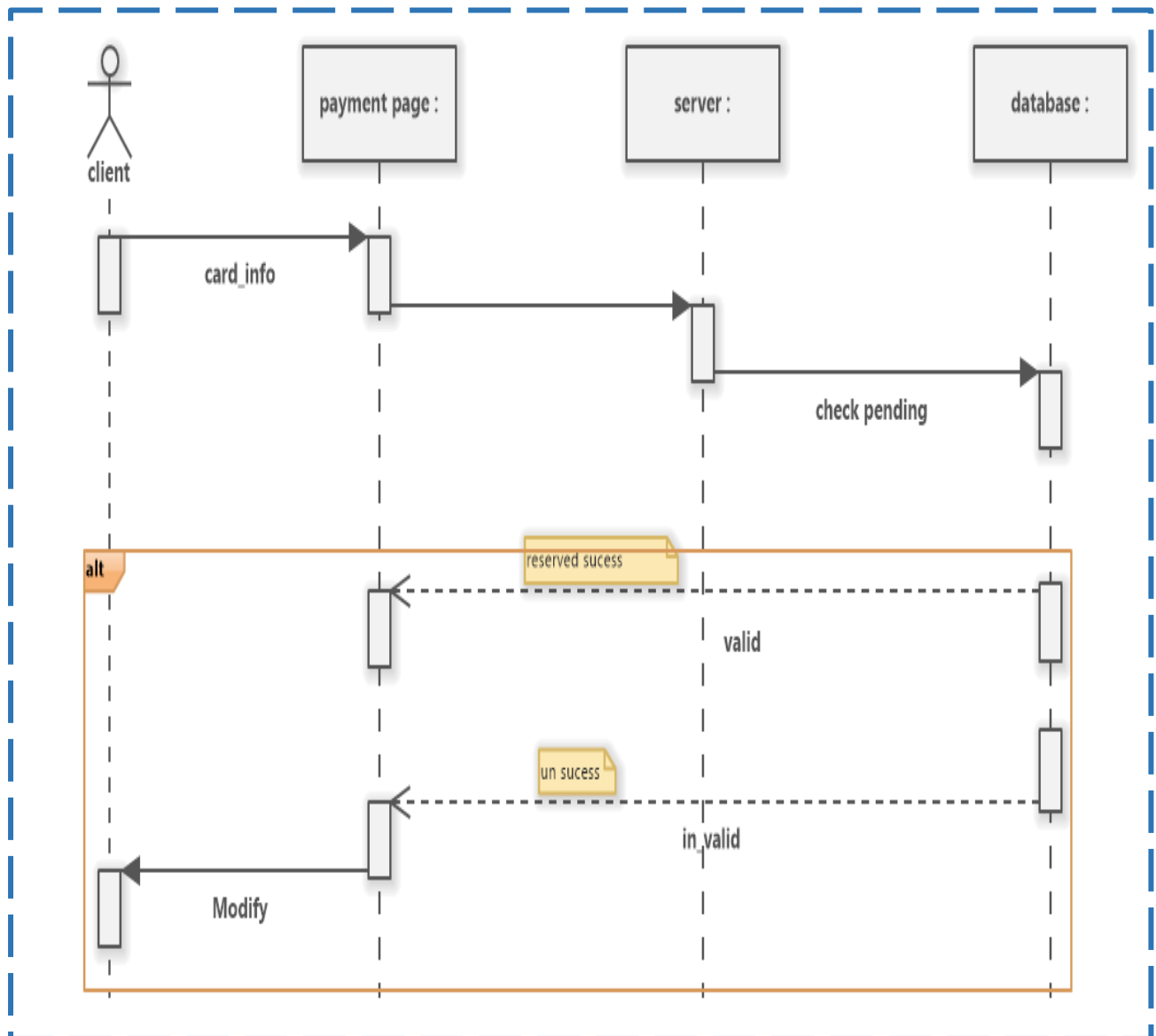


figure 3.8: payment sequence Diagram

3.3.4 Database vision

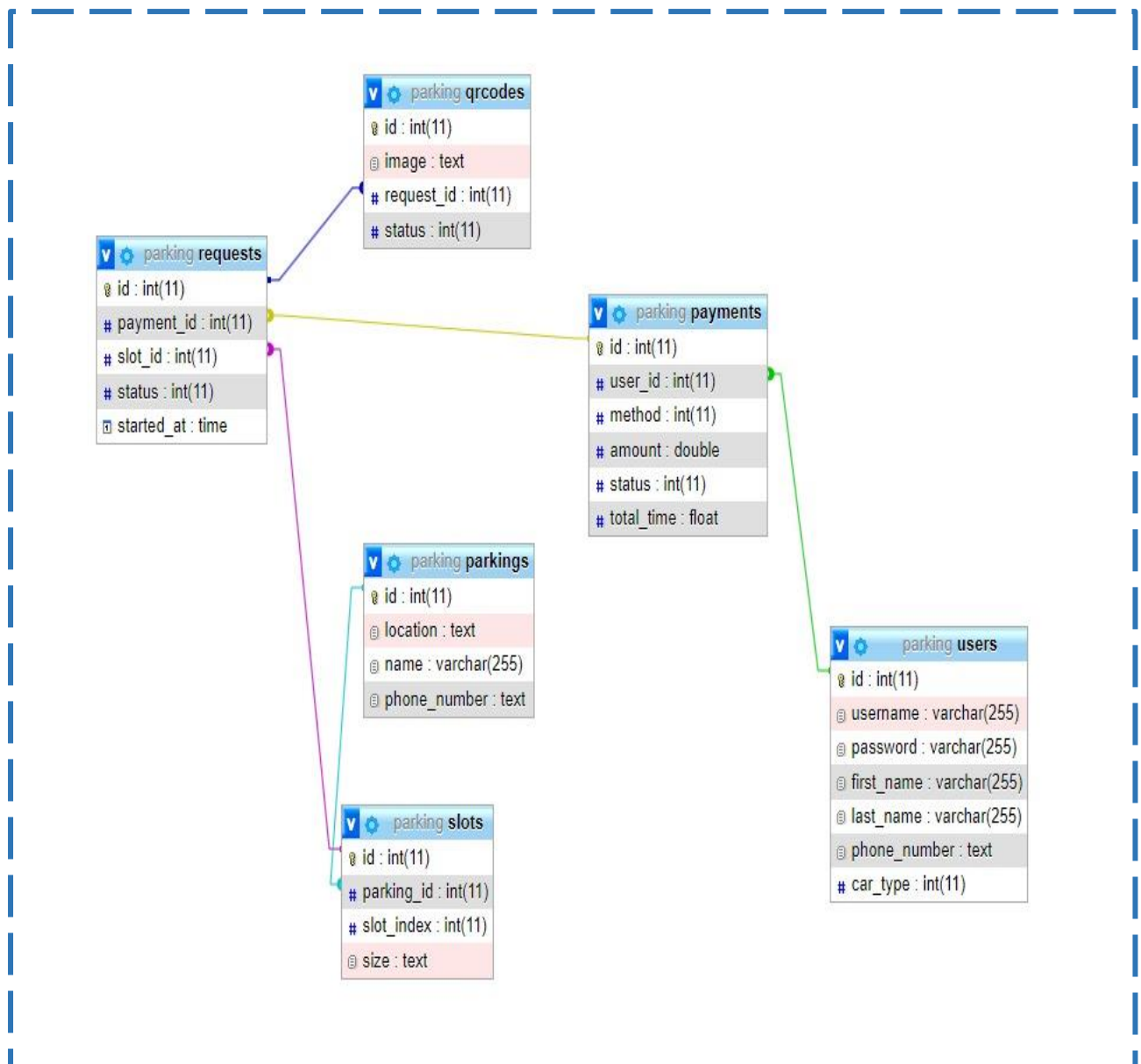


Figure 3.9: Database vision

3.3.5 Class Diagram

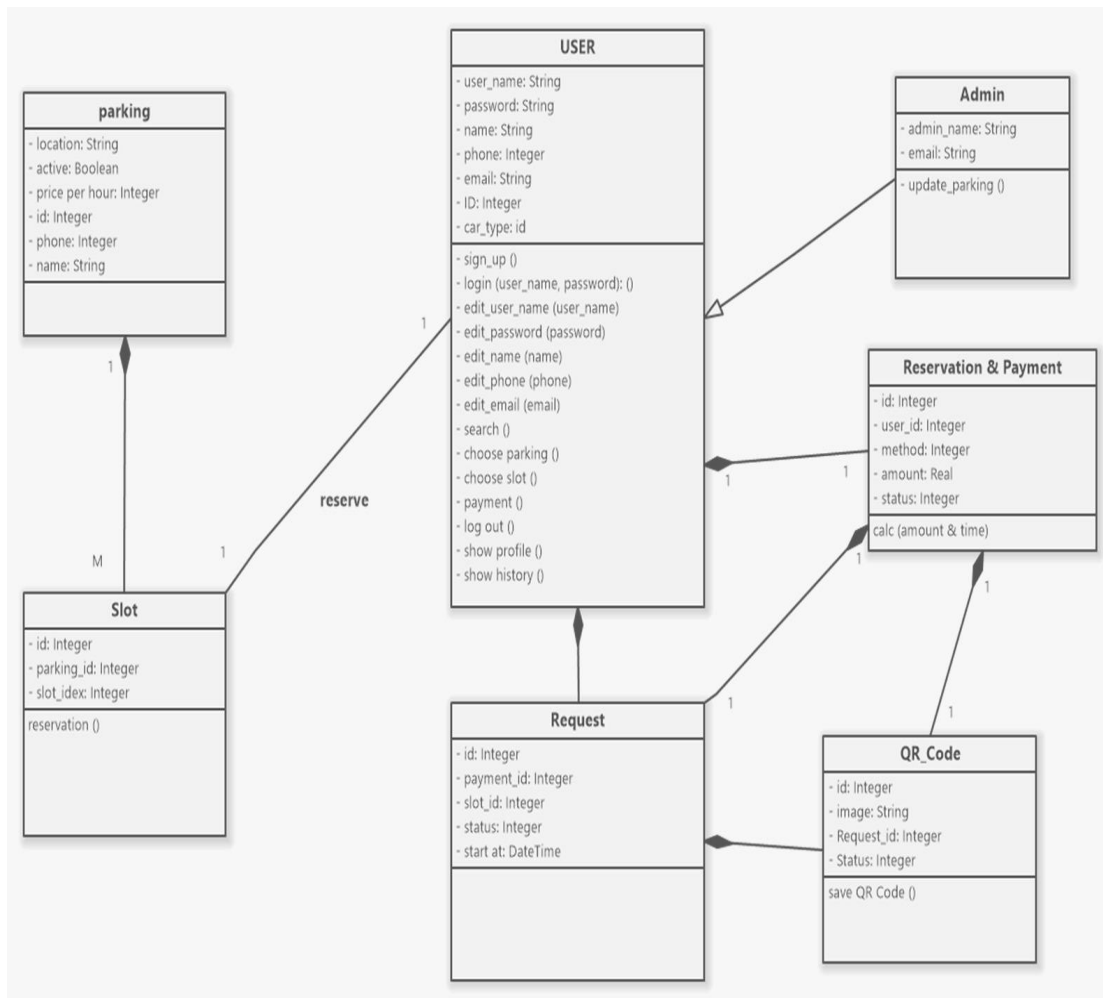


Figure 3.10: class diagram

3.3.6 Context Diagram

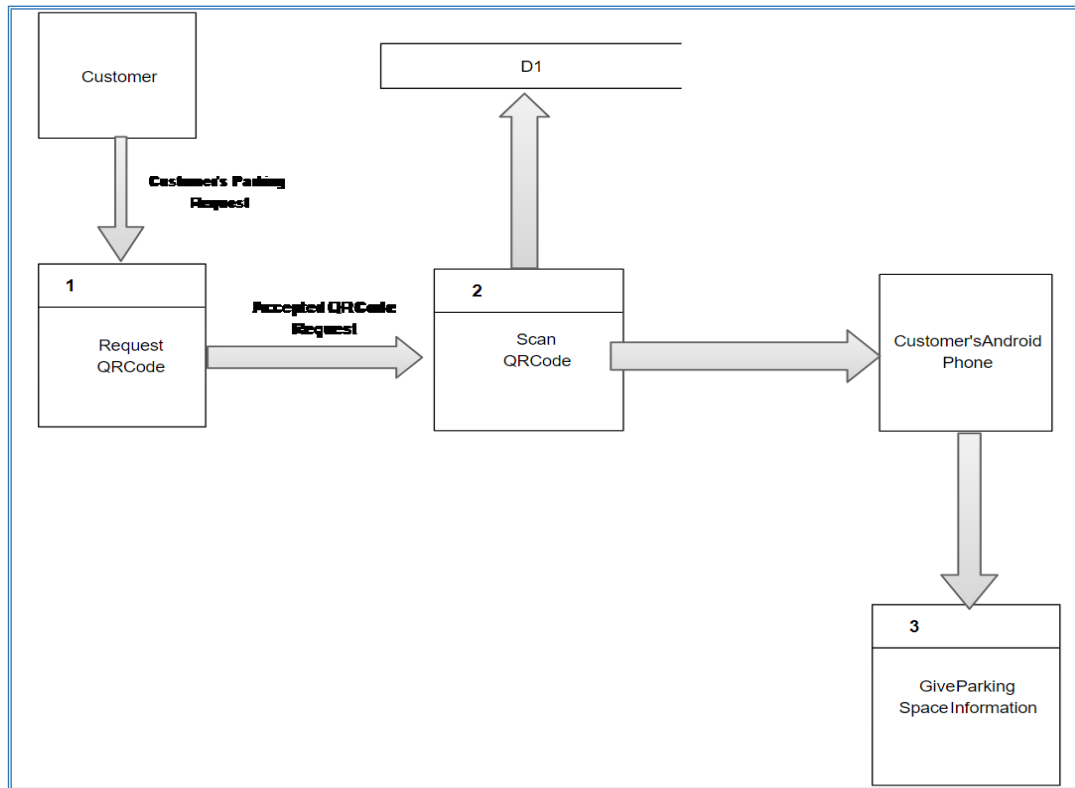


Figure 3.11: context diagram

3.3.7 Flowchart Diagram

A flowchart is a picture of the separate steps of a process in sequential order. It is a generic tool that can be adapted for a wide variety of purposes, and can be used to describe various processes, such as a manufacturing process, an administrative or service process, or a project plan. It's a common process analysis tool and one of the seven basic quality tools.

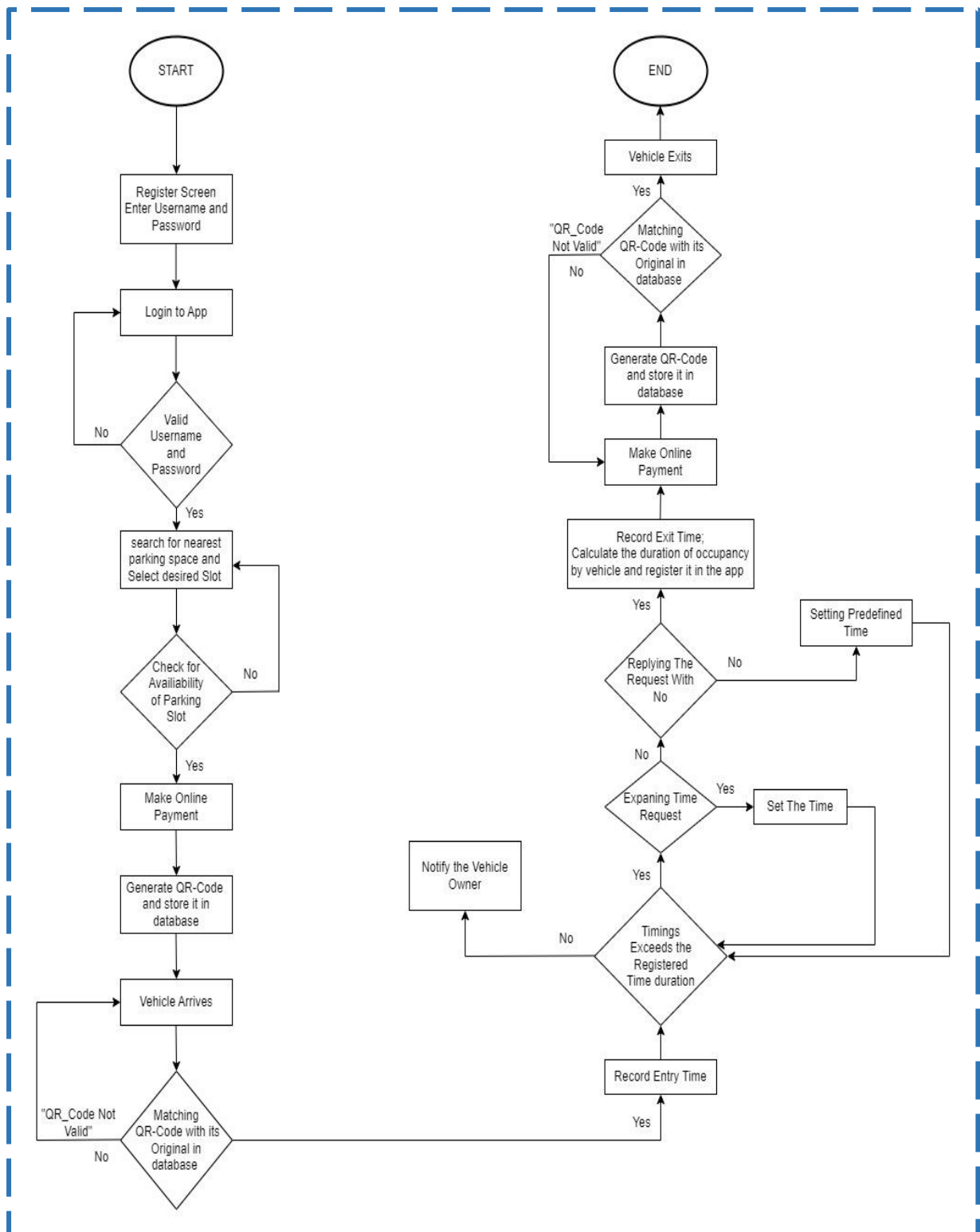


figure 3.12: Flowchart Diagram

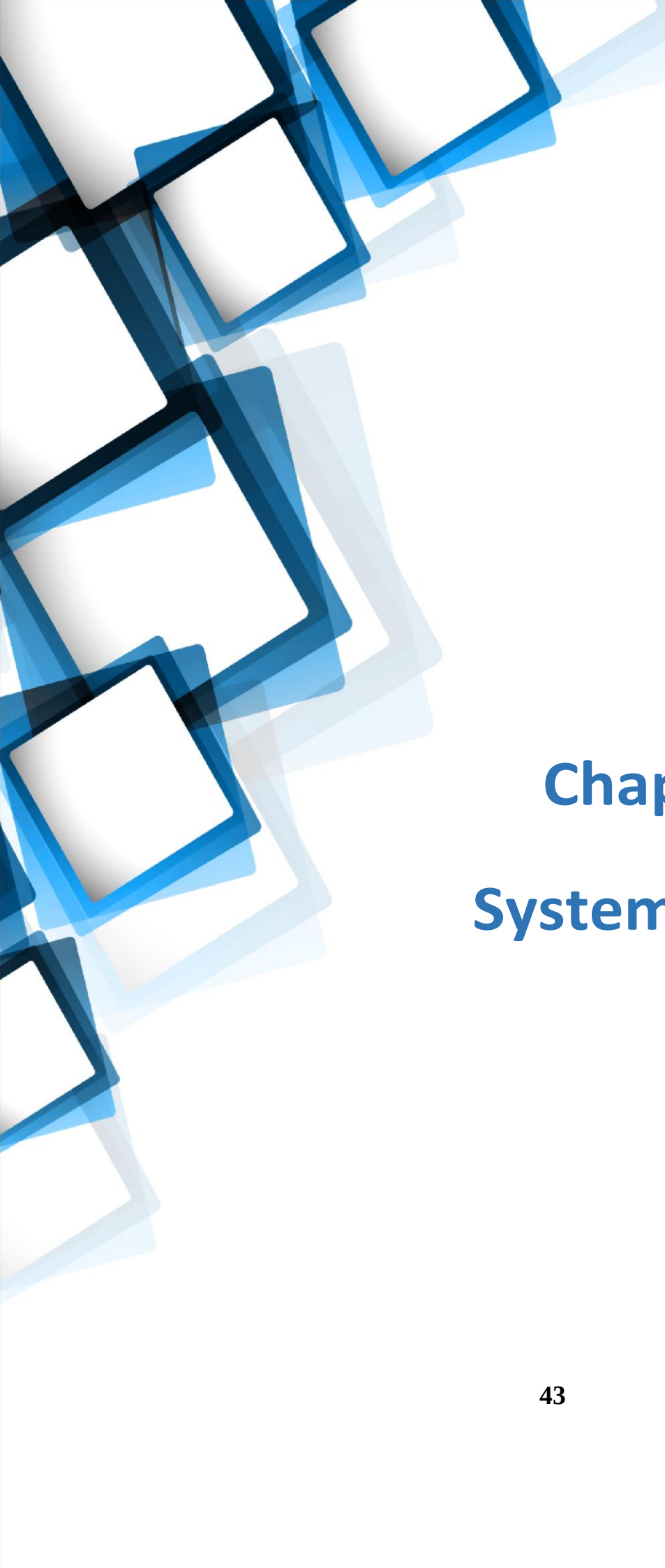
3.4 Tools and language

Developing our software application can be divided into main two parts, which are the design part, the implementation part. The design part involves designing diagrams and designing the user interface of the mobile application. The implementation part involves programming languages, IDEs, frameworks, and libraries. The following list shows the needed tools for the software development and a brief description about their usages:

1. Software Ideas Modeler – it is used to draw the UML diagrams.
2. Adobe XD – it is used to design user interface with user experience and prototypes
3. XAMPP – is a cross-platform web server solution stack package consisting mainly of the Apache HTTP Server.
4. visual studio code – cross-platform IDE for python.
5. Postman – is an API platform to design, build, test and iterate APIs.
6. TensorFlow - free and open source machine learning software library.
7. Android Studio provides app builders with an integrated development environment (IDE) optimized for Android apps.
8. Dart is a client-optimized language for fast apps on any platform.
9. Fastapi is a modern, fast (high-performance), web framework for building APIs with Python 3.7+ based on standard Python type hints.

3.5 Summary

In this chapter we provide the reader with detailed knowledge about our system. Section 3.1 provides system requirements Which is divided into functional and nonfunctional requirements, and user requirements which specify some different specifications for users, Section 3.2 includes system architecture which describe its major components, their relationships, and how they interact with each other, Section 3.3 provides development methodology which includes UML diagrams that shows the details of how the system will function. In the end of the chapter we listed the needed tools for us to build the system.



SPOTIT

Chapter 4

System Design

In this chapter we start to convert the analysis system to actual User interface screens according to User experience Design considering the determined requirements.

4.1 Software Application Design:

Software Application Design involves gathering and analyzing the customer business functions, then designing an application solution to meet the prioritized business requirements. The first step in this service is to establish a charter, which defines the scope and responsibilities for this service. The final step is a presentation of options for meeting customer business. The first step in this service is approval to proceed with website and mobile application so we try to focus on the look and feel of both. The design stage encompasses several different aspects, including user interface design (UI), usability (UX), content production, and graphic design. Within this post, we focused mainly on UI and UX design. The best application is chosen based on design, Usability, and Creativity.

4.2 UI in System design

UI stands for User Interface. UI is the part of the website and mobile app which a user interacts with. In simple terms, it's everything you see and touch, such as buttons, colors, fonts, navigation, etc.

4.3 UX in System design

UX stands for User Experience. UX focuses on how the users feel towards the application, and their experience using it. Was the web site or mobile application hard to use, was it slow, how users will interact with screens and view? These are the types of questions a UX designer will focus on when reviewing and working on a website and mobile application.

4.4 User Interface Screens (Mobile APP)

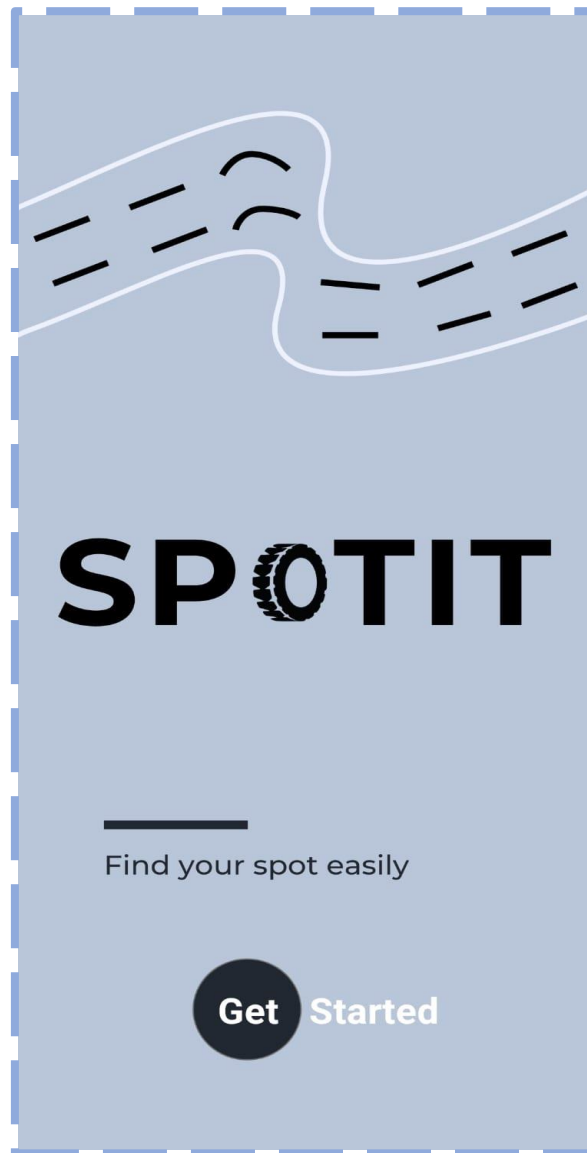


Figure 4.1: splash screen



Figure 4.2: Introduction page

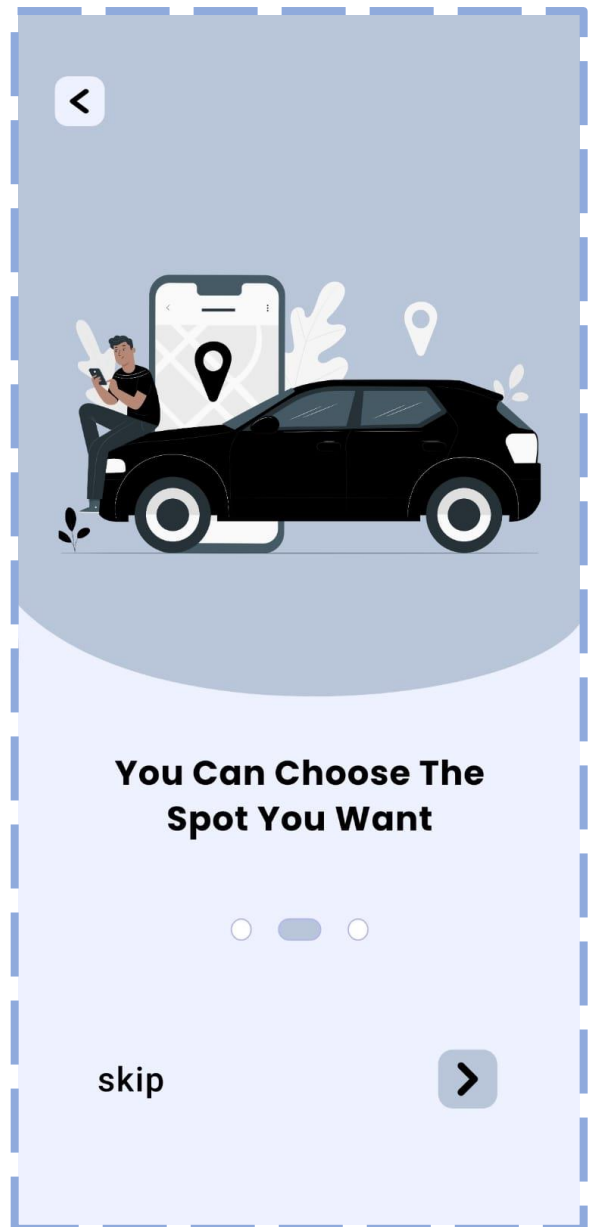
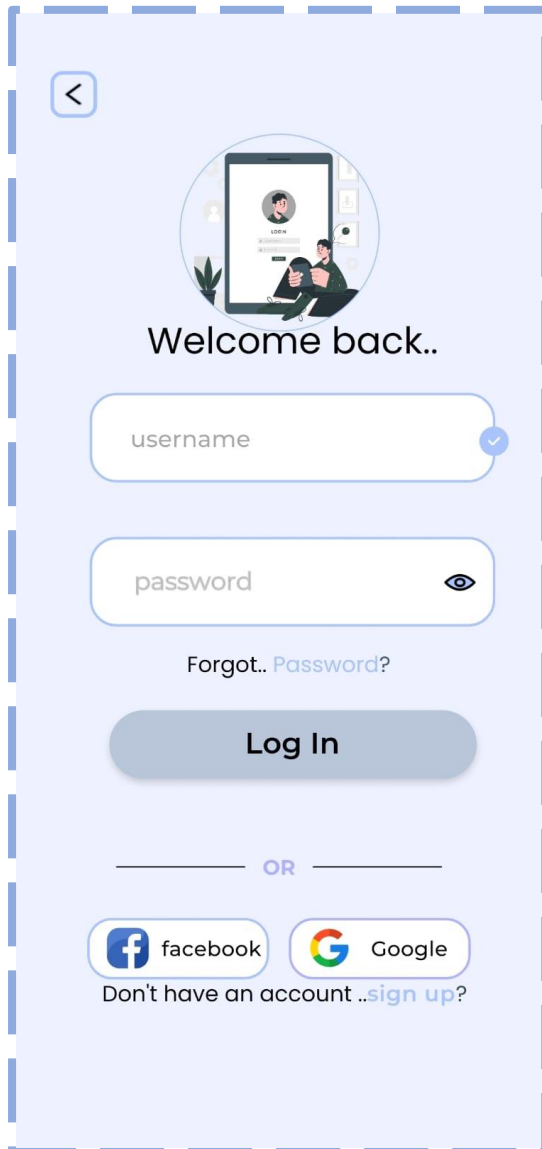


Figure 4.2: Introduction page



The login page features a light blue background with a dashed blue border. At the top left is a back arrow icon. Below it is a circular illustration of a person at a computer. The text "Welcome back.." is centered. Below this are two input fields: "username" with a checkmark icon on the right, and "password" with an eye icon on the right. A link "Forgot.. Password?" is positioned below the password field. A large "Log In" button is centered below the fields. Below the button is a horizontal line with the word "OR" in the center. At the bottom are two social login buttons for "facebook" and "Google". Below these is the text "Don't have an account ..sign up?".

<

Welcome back..

username ✓

password 👁

Forgot.. [Password?](#)

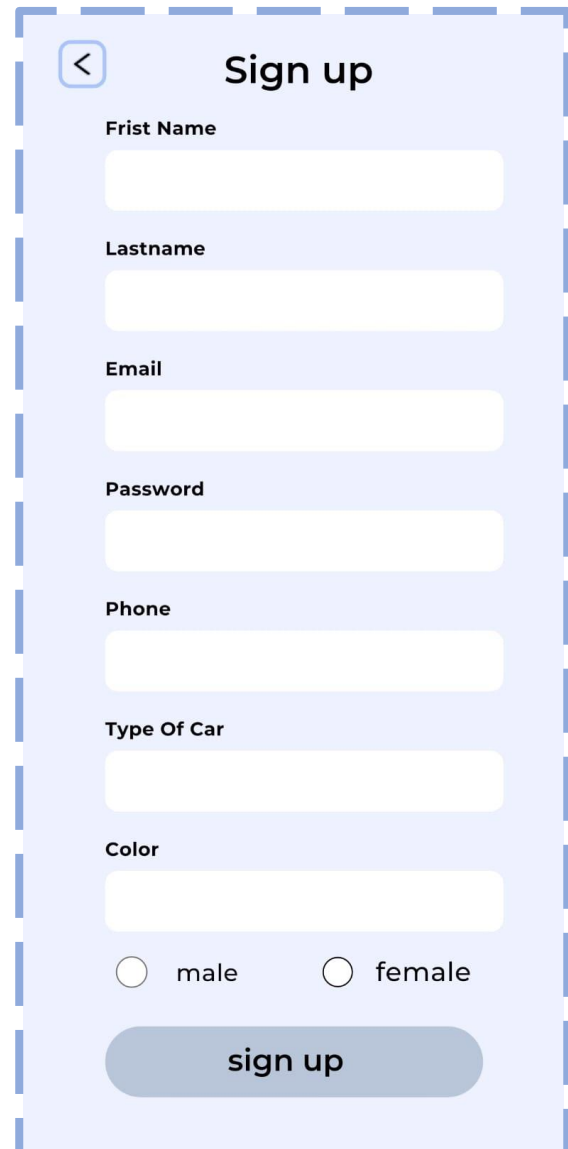
Log In

OR

facebook Google

Don't have an account ..[sign up?](#)

Figure 4.3: login page.



The sign up page has a light blue background with a dashed blue border. It starts with a back arrow icon and the title "Sign up". The form consists of several input fields: "Frist Name", "Lastname", "Email", "Password", "Phone", "Type Of Car", and "Color". Below the "Color" field are two radio button options: "male" and "female". A large "sign up" button is at the bottom.

< Sign up

Frist Name

Lastname

Email

Password

Phone

Type Of Car

Color

☐ male ☐ female

sign up

Figure 4.4: sign up page.

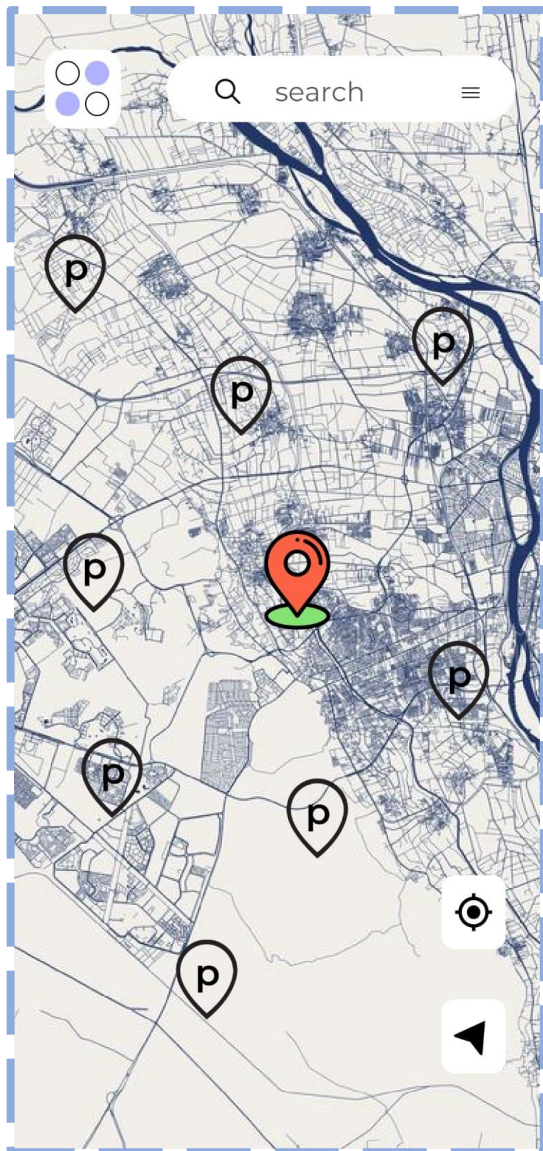


Figure 4.5: home page

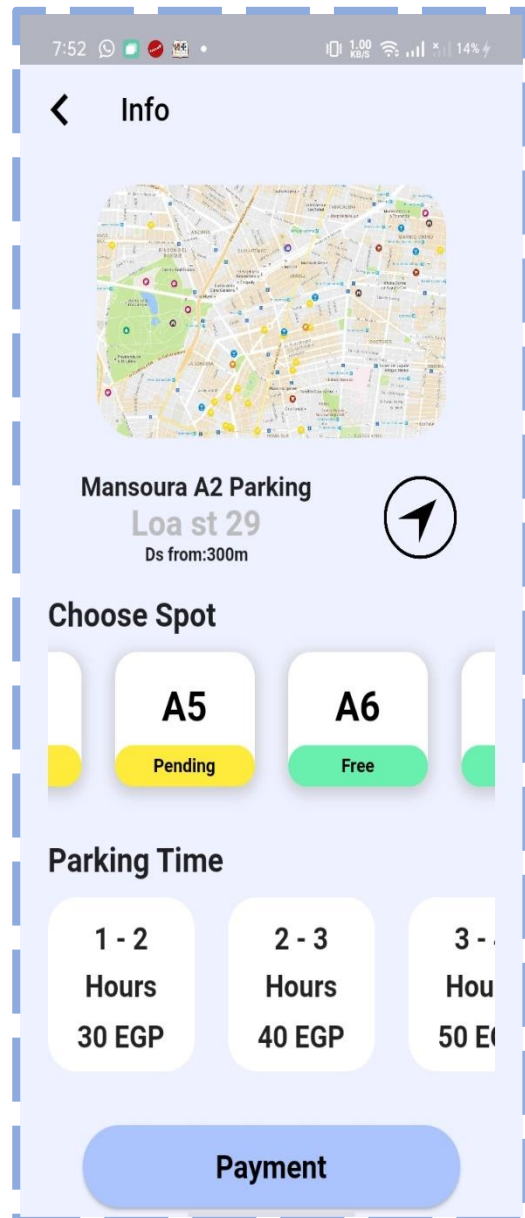


Figure 4.6: Parking reservation page

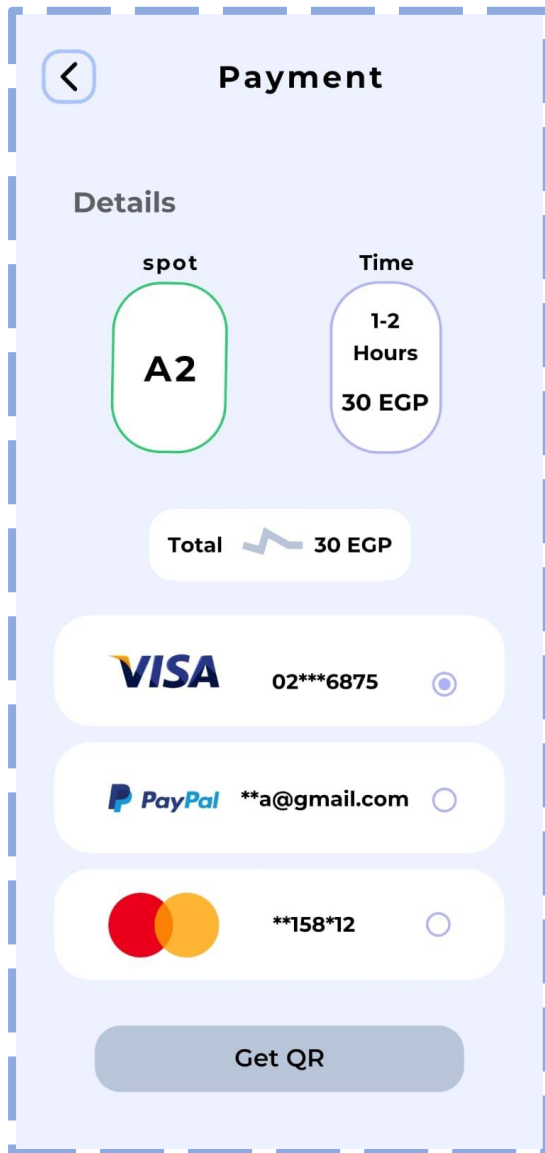


Figure 4.7: payment page

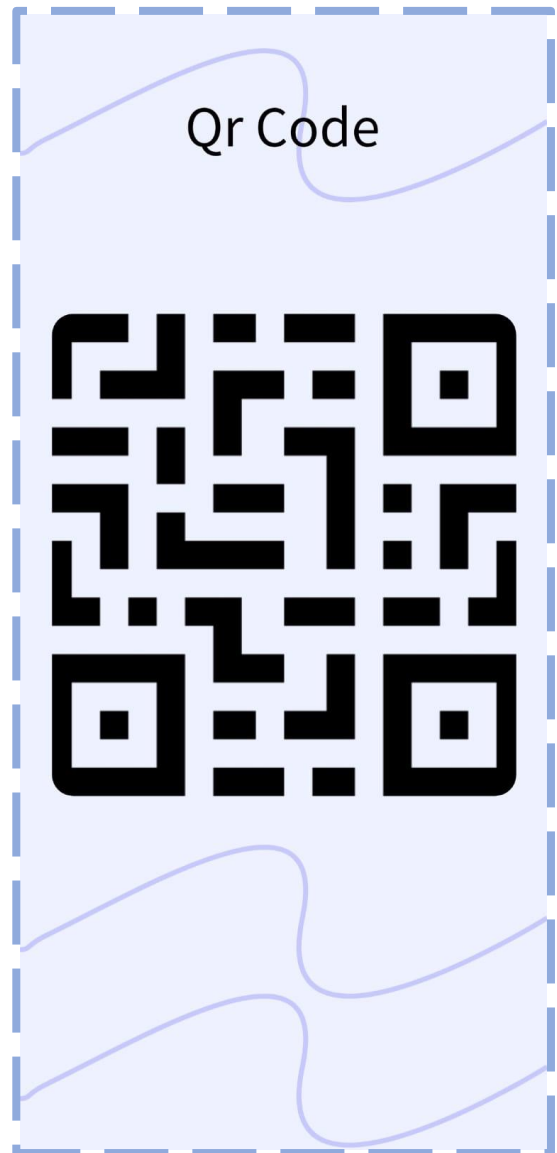


Figure 4.8: QR code page

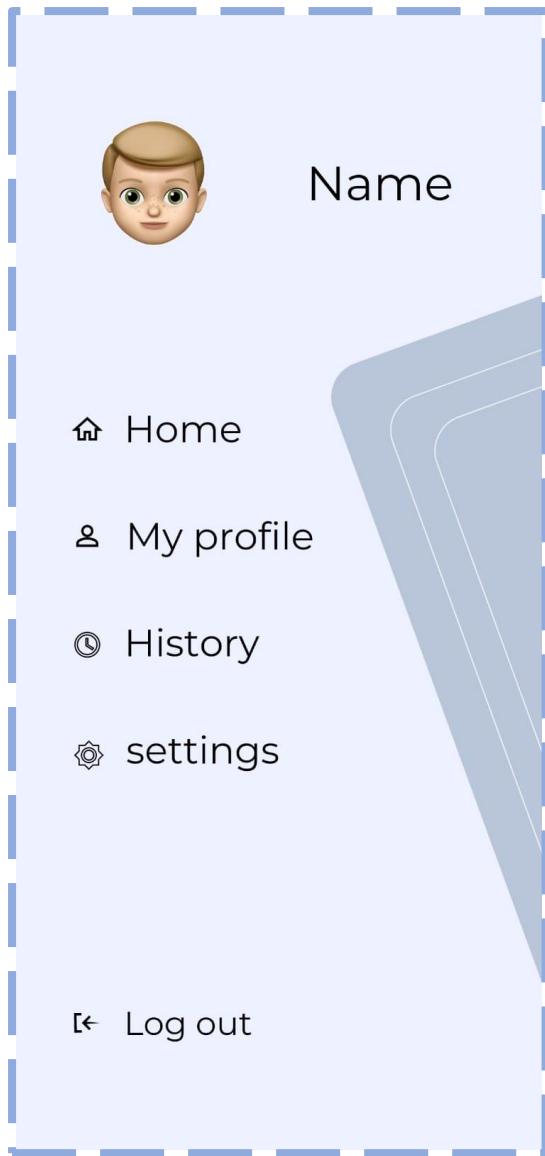


Figure 4.9: Sidebar page

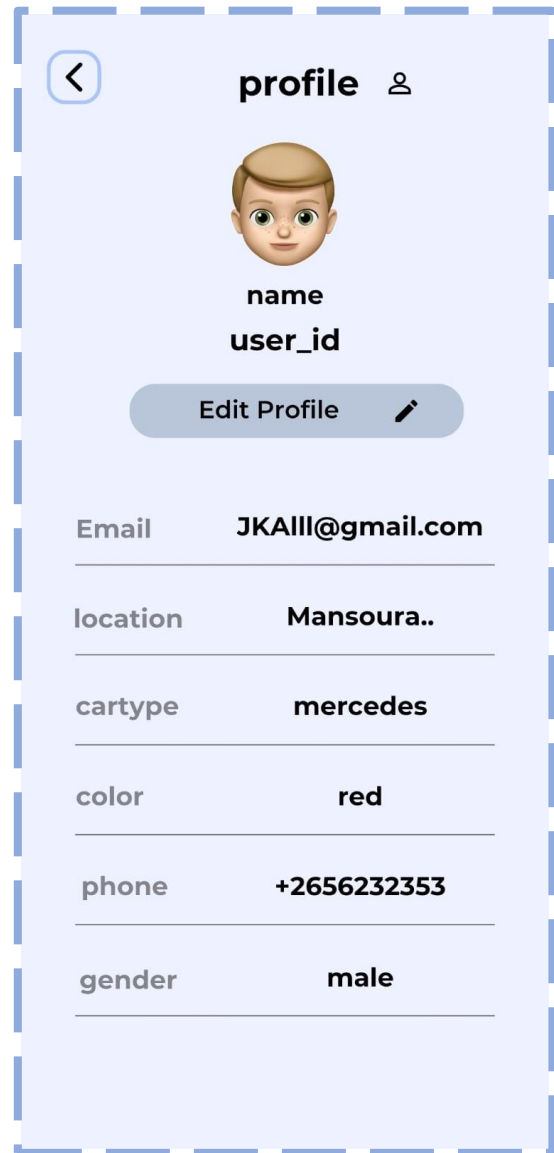


Figure 4.10: profile page

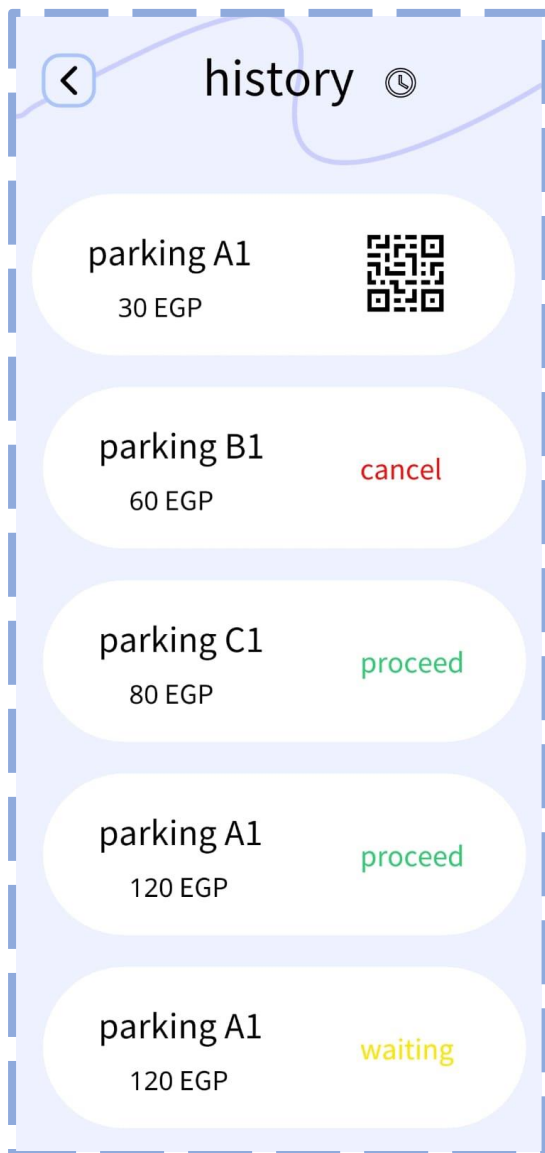


Figure 4.11:user parking history page

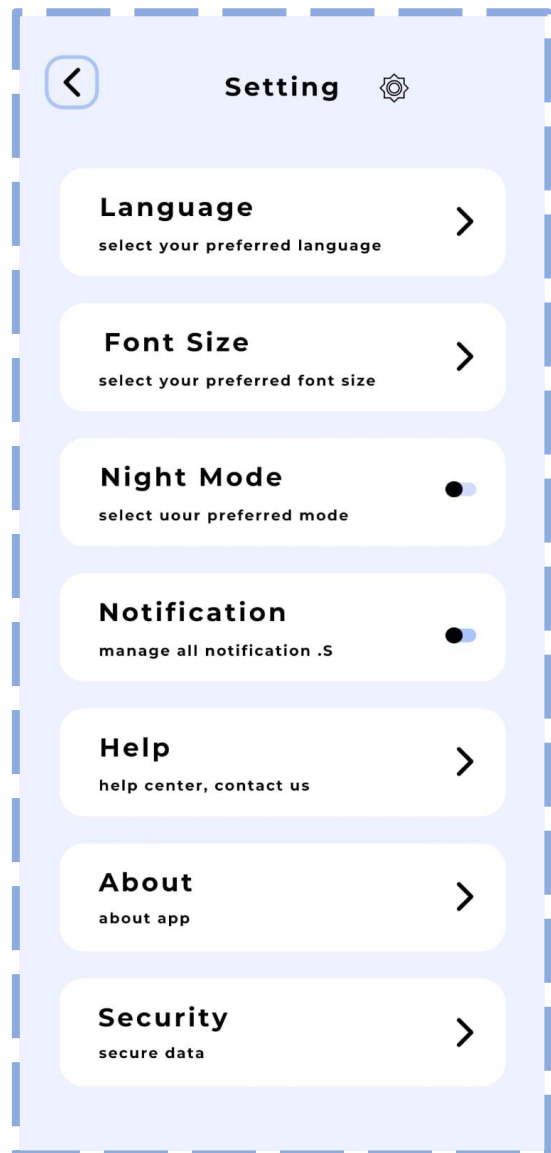


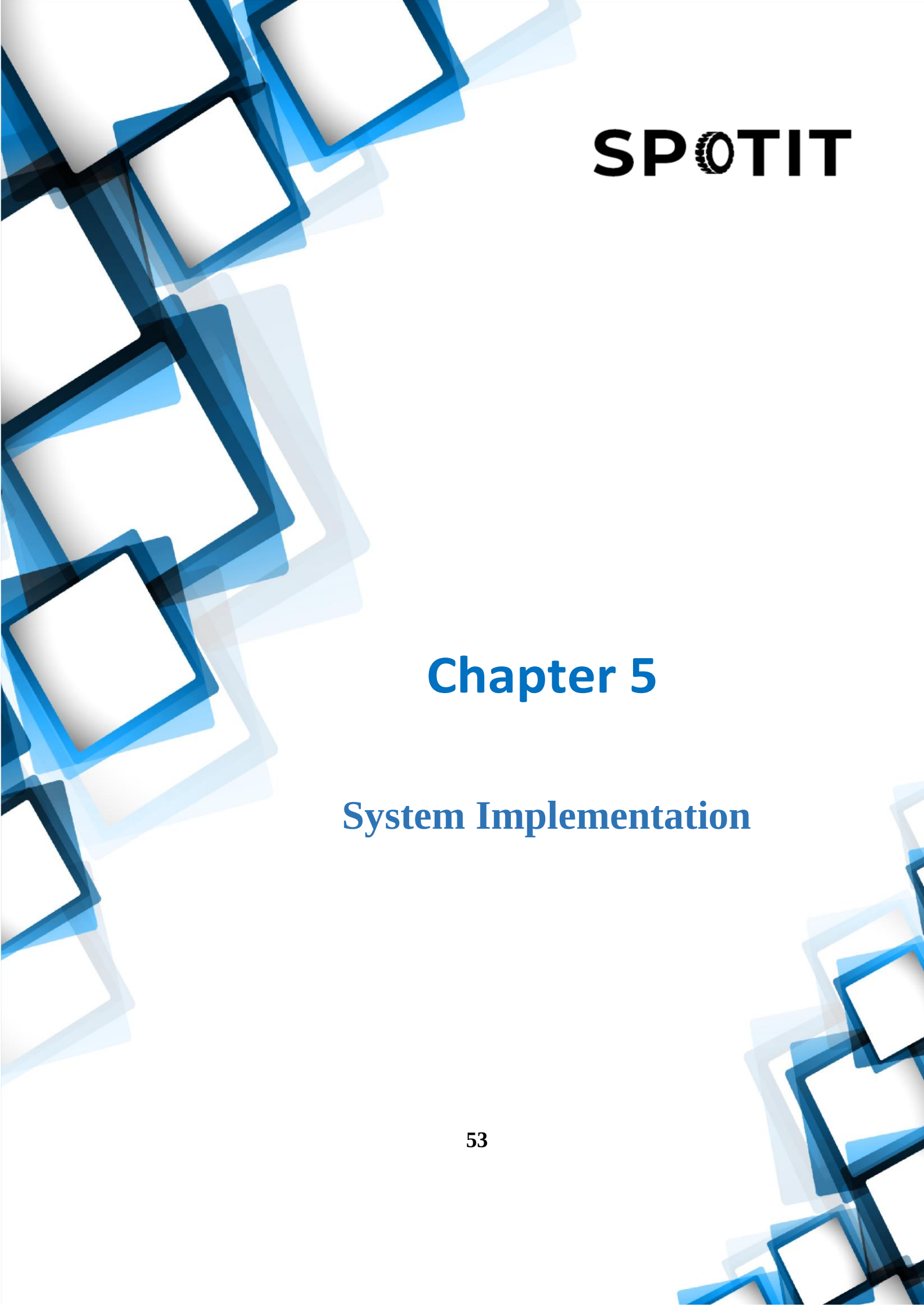
Figure 4.12: setting page.

A mobile app screen with a light blue background. At the top left is a back arrow icon. The main heading is "Verify your number with codes sent to you". Below it is a subtitle "Enter the code sent to (+0256858)". There are four input fields for a code; the first contains the number "5", the second contains a vertical bar "|", and the other two are empty. Below the input fields is a link "I didn't receive the code, Resend" where "Resend" is in blue. At the bottom is a rounded button labeled "Submit".

Figure 4.13: change password page

A mobile app screen with a light blue background. At the top left is a back arrow icon. The main heading is "Enter your phone number". Below it is a text input field with the placeholder text "phone". Underneath the input field is a rounded button labeled "find your account". At the bottom of the screen is the text "search by your email instead".

Figure 4.14: verify email page.



SPOTIT

Chapter 5

System Implementation

In this chapter we will represent the system implementation of our project, the system implementation is the carrying out, execution, or practice of a plan, a method, or any design, idea, model, specification, standard, or policy for doing something into a code. As such, implementation is the action that must follow any preliminary thinking for something to happen.

5.1 steps of implementation

In an information systems context, software or hardware implementation encompasses all the post-sale processes involved in something operating properly in its environment, including analyzing requirements, installation, configuration, customization, running, testing, systems integrations, user training, delivery, and making necessary changes. The word "deployment" is sometimes used to mean the same thing. For an implementation process to be successful, many tasks between different departments need to be accomplished in sequence. Companies strive to use proven methodologies and enlist professional help to guide them through the implementation of a system .

5.1.1 Mapping design to sample code:

The sample application code is considered to be the backbone of any system, because without it any part of the design will not be activated, meaning that the design will be without any importance. For example, using the development, we can activate all the buttons and give them orders to execute a specific event.

5.1.2 System testing:

After the implementation of the system through all its steps comes to the test step of the system to achieve all the requirements of the user and how to deal with the system in a safe and easy without complexity and problems and whether the components of the project were achieved clearly to benefit from it positively and achieve the most important step is the availability of Database at the highest level in terms of speed of storage and safety data for users.

5.2 Implementation

5.2.1 Convolutional Neural Networks

Computer vision problems were solved using traditional image processing and machine learning classification algorithms. CNN came later when more processing power and data became more available for research. CNN are nowadays used now for a variety of applications such as facial recognition, healthcare, self-driving cars, accessibility, and biology. Computer vision is founded on the ability to detect low dimension features from a high-dimensional input image. Previously, computer vision researchers had to master the domain knowledge depending on the task, define these features manually, and then detect these features using different classifiers. Humans' ability to identify these features is limited because of the variations that affect a 3D object in the world such as scale, viewpoints, Illumination, occlusion, deformations, background clutter, intra-class variations, and others. Figure 5.1 shows an example of such variations [21].



Figure 5.1: The different image variation that are considered as a challenge in front of manual feature extraction from images [22].

CNN uses neural networks to perform automatic feature extraction. Through this architecture, we can learn a hierarchy of features directly from the large datasets of images. An example of such a hierarchy of features is shown in Figure 5.2.

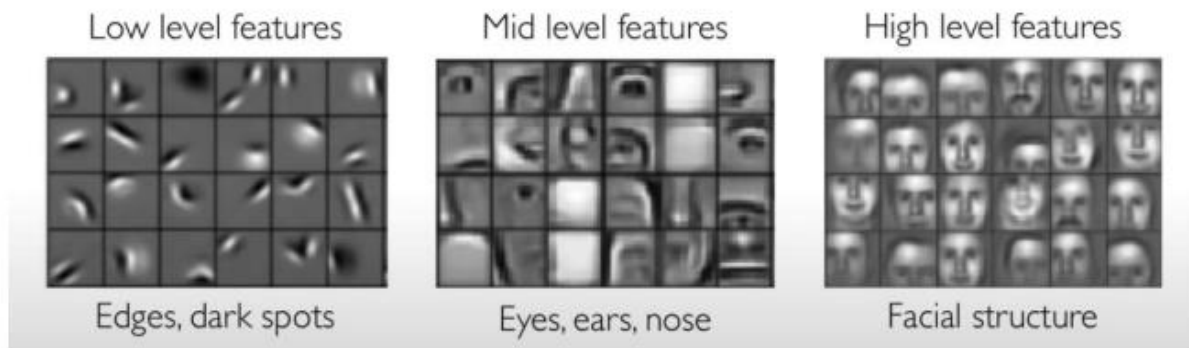


Figure 5.2: Example for a hierarchy of features that are detected by a CNN [22].

Given that an image representation is usually a 3D matrix of numbers, the use of regular ANNs through fully connected layers that will take pixel values of these matrices as an input will result in parameter explosions and the loss of the spatial information of the image. Therefore, CNN is based on the use of a 2D sliding patch over the input images and provides outputs to the neurons that will make the new extracted feature in 2D as well. This will maintain the spatial information as shown in Figure 5.3. This patching operation is based on the convolution mathematical operation. More specifically, it is sliding a weighted matrix, doing element-wise multiplication, and adding up the result. The weighted matrix is called a filter with weights that are learned in the training. The output of the convolution layer over the input image is called a feature [21].

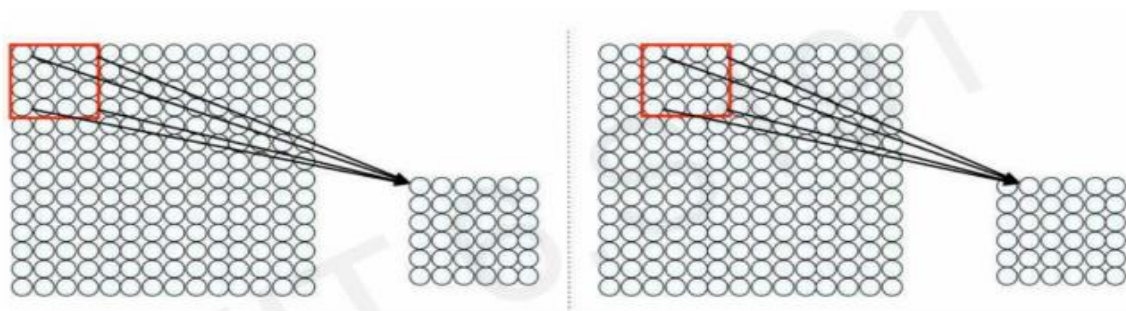


Figure 5.3: The sliding patch over the image and outputting another 2D weighted feature [21].

The CNN is thus designed based on three building blocks. First, are the convolution layers that generate feature volumes. The convolutional layer can have multiple filters with each filter producing a different feature map. The feature maps are stacked together to produce a feature volume that is next forwarded to the next layer. The second building block is applying non-linearity to reflect the non-linear nature of real life. ReLu function is the most common activation function used in CNN. Finally, the feature volumes are processed by pooling layers. The pooling layers are used to down sample the features and reduce dimensionality. This allows keeping the scalability of the input and spatial invariance and structure [21]. This process is shown in Figure 5.4.

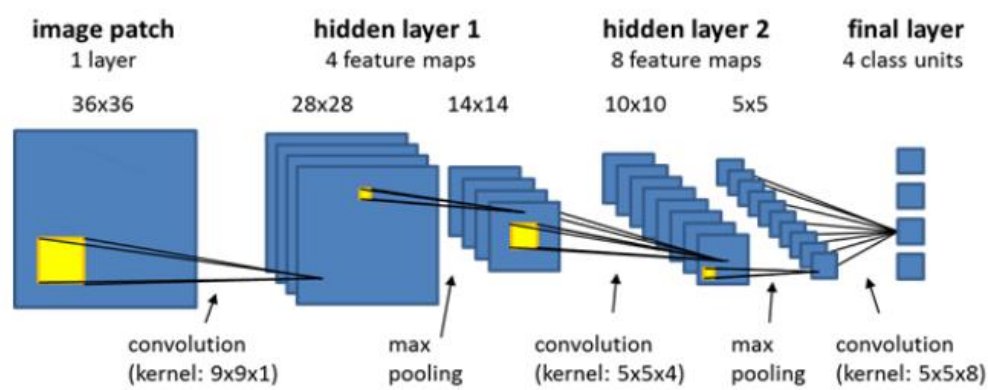


Figure 5.4: An example of convolution layers and maxpooling layers acting on an input image [23].

These three building blocks' key role is feature learning and extraction. Next, the output is forwarded to fully connected layers to perform the classification task [21] as shown in Figure 5.5.

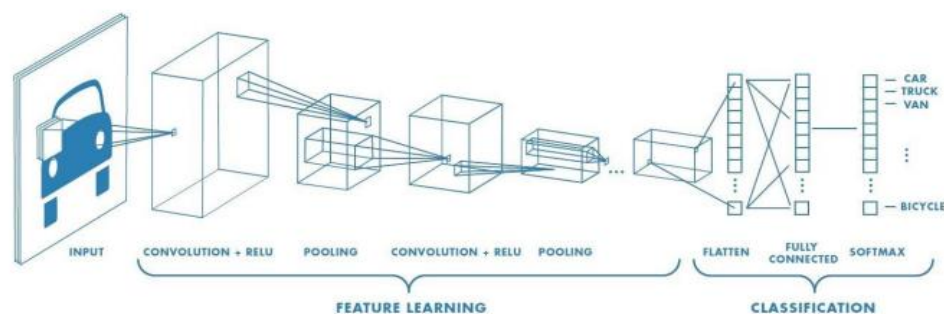


Figure 5.5: A complete CNN architecture with both the feature learning first and the classifier next [24].

5.2.2 Adam Optimizer and Learning Rate Decay

The CNN performs a forward pass through the neurons of the neural network. Then a backpropagation pass is done to compute the gradients of all the trainable neural network parameters over the loss function used. The next step is to update the model's weight in a direction that will minimize the loss. This updating step is decided by a learning rate that is set as a hyperparameter at the beginning of the training. The choice of this learning rate affects the training. A small learning rate can end up with the training stuck at a local minima, while a big learning rate can make the training chaotic and unable to find a minima [21]. The Adam optimizer has an algorithm that allows the learning rate to adapt to the specificities of the loss function. This helps also in increasing the training speed [25]. Also, combining the Adam optimizer with an inverse time learning decay allows the training to go faster as well. The inverse time decay we used "hyperbolically decreases the learning rate to 1/2 of the base rate at [26] epochs, 1/3 at [25] epochs, and so on" [27].

5.2.3 Dataset

PKLot: This dataset is one of the most appropriate candidate resources for facilitating this task, providing images captured in two parking lots from three separate camera views. This dataset contains images captured on sunny, rainy, and overcast days, and thus, we found it a very useful tool for our project. It contains 12,417 images of parking areas and 695,899 images of parking spaces segmented from them, which have been manually checked and labeled. All the images were captured in the parking areas of the Federal University of Parana (UFPR) and the Pontifical Catholic University of Parana (PUCPR), both located in Curitiba, Brazil. The cameras were located on the roof of the buildings to decrease the potential occlusion of adjacent cars [28].

Moreover, such settings allow series of pictures that vary widely in terms of changing illumination caused by weather variations to be obtained.

Unfortunately, this dataset doesn't have night shots since the illumination available in the parking lots was not sufficient to acquire good quality images. The resulting images were stored in JPEG color format with lossless compression (quality 100%) in a

resolution of 1280 X 720 pixels. They were organized into three subsets named UFPR04, UFPR05 and PUCPR. The first two contain images of different views of the same parking lot captured from the 4th and 5th floors of the UFPR building. The last dataset contains images captured from the 10th floor of the administration building of the PUCPR.

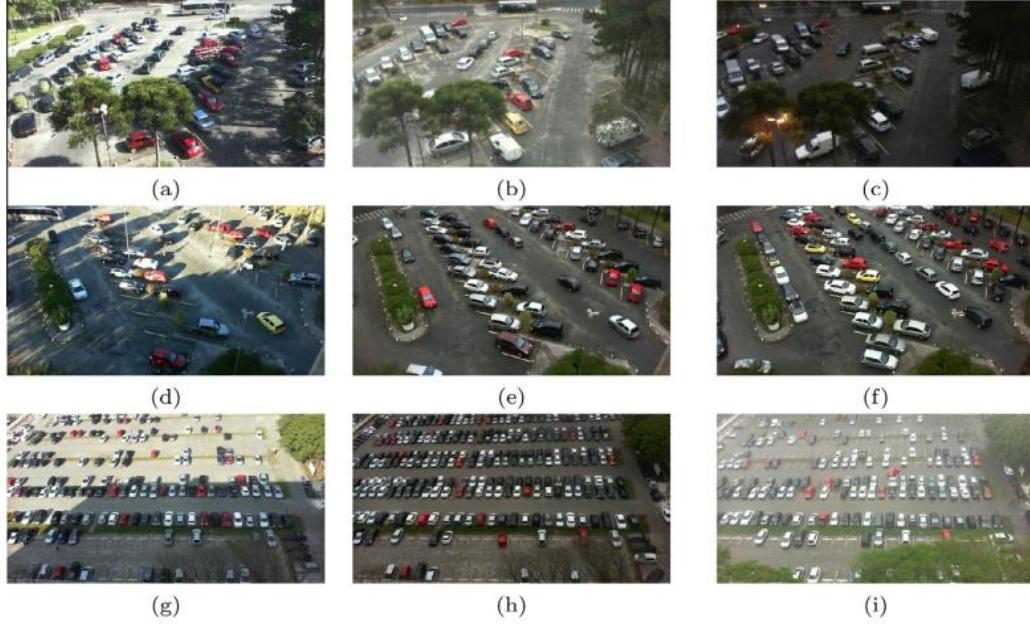


Figure 5.6. Images captured under weather conditions different: (a) sunny (b) overcast, and (c) rainy from UFPR04; (d) sunny (e) overcast, and (f) rainy from UFPR05; and (g) sunny (h) overcast, and (i) rainy from PUCPR.

CNRPark-EXT: A dataset of roughly 150.000 labeled images of vacant and occupied parking spaces, built on a parking lot of 164 parking spaces. CNRPark-EXT includes and significantly extends CNRPark(Amato et al., 2016 [29]), a smaller dataset of roughly 12.000 labeled images, which we also used. The smaller CNRPark dataset contains images of the parking lot collected in different days of July 2015, from 2 distinct cameras A and B, (see Figure 5.8, top row), which were placed in order to have different perspectives and angles of view. The full CNRPark-EXT extends CNRPark with images collected from November 2015 to February 2016 under various weather conditions by 9 cameras (see Figure 5.8, bottom row) with different perspectives and angles of view.



Figure 5.7: Training set patches segmented from the camera view. Images show four parking spaces in both status: busy (first row) and free (second row). They also present some occlusion and shadow situations that we faced.

It captures different situations of light conditions, and it includes partial occlusion patterns due to obstacles (trees, lampposts, other cars) and partial or global shadowed cars (see Figure 5.7). This allows training a classifier that is able to distinguish most of the difficult situations that can be found in a real scenario. Patches of the CNRPark-EXT dataset are grouped together into subsets corresponding to different weather conditions (Sunny, Overcast, Rainy), days of capture, and camera IDs. Patches are also grouped into training, validation, and test subsets, to provide a common and objective ground for training and testing classification algorithms. CNRPark-EXT is composed of 4287 screen-shots acquired in 23 different days, resulting in a dataset of 144.965 labeled parking space patches.



Figure 5.8: Parking space patches are segmented and numbered as shown in the images. Top images belong to the small dataset CNRPark, while bottom images belong to the full CNRPark-EXT dataset.

We used two datasets in our project: CNRPark-EXT, and PKLot(de Almeida et al., 2015 [28]). The two datasets are significantly different. Besides the fact that they contain pictures taken from different parking lots, it is worth highlighting the following differences:

- a) in CNRPark-EXT parking spaces masks are non-rotated squares; often images do not cover precisely or entirely the parking space volume, whereas in PKLot images are extracted using rotated rectangular masks, which are subsequently straightened, resulting in a more precise coverage of the parking space;
- b) CNRPark-EXT is composed also of heavily occluded spaces (almost entirely covered by trees and lampposts) which are not included in the set of segmented spaces of PKLot; moreover in CNRParkEXT images are taken from lower point of views with respect to PKLot, resulting in more occlusions due to adjacent vehicles. These aspects makes the classification of PKLot an easier challenge with respect to CNRPark-EXT, which shows higher variability between images, and include more noisy factors. The usage of two completely different datasets allowed us to extensively validate and compare the proposed approach.

5.2.4 Data Preparation

In order to preprocess the images for our binary classification problem, we implemented a preprocessing code using OpenCV in Python. This code allows us to extract the coordinates of each parking slot by drawing rectangles around them. after extracting the coordinates of each parking slot, these coordinates will be utilized for cropping the slots from the whole image of the parking area which captured by the parking camera. By applying these coordinates as reference points, we can accurately isolate and extract the individual slot images from the complete parking image. This allows us to focus solely on the relevant regions of interest and pass them to the model for classification. It is important to note that this preprocessing step only needs to be performed once, in case there are any changes to the parking area layout. Once the coordinates are obtained, we can simply forward the images to the model for classification without the need for additional preprocessing.

```

def draw_rectangles(image, rectangles):
    fixed_rectangles = []
    for rect in rectangles:
        # calculate fixed coordinates based on original image size
        fixed_rect = [(int(p[0] * image.shape[1] / window_size[0]),
                        int(p[1] * image.shape[0] / window_size[1])) for p in rect]
        fixed_rectangles.append(fixed_rect)

    # draw lines between all the points that have been clicked
    for i in range(len(refPt) - 1):
        fixed_line = [(int(refPt[i][0] * image.shape[1] / window_size[0]),
                        int(refPt[i][1] * image.shape[0] / window_size[1])),
                      (int(refPt[i+1][0] * image.shape[1] / window_size[0]),
                        int(refPt[i+1][1] * image.shape[0] / window_size[1]))]
        cv2.line(image, fixed_line[0], fixed_line[1], (0, 255, 0), 1)

    # draw the fixed rectangles on the image
    for rect in fixed_rectangles:
        cv2.line(image, rect[0], rect[1], (0, 255, 0), 1)
        cv2.line(image, rect[1], rect[2], (0, 255, 0), 1)
        cv2.line(image, rect[2], rect[3], (0, 255, 0), 1)
        cv2.line(image, rect[3], rect[0], (0, 255, 0), 1)

    return image

```

Figure 5.9: function which we used for drawing rectangle around each slot

```

def zoom_at(img, zoom, coord=None):
    # Translate to zoomed coordinates
    h, w, _ = [zoom * i for i in img.shape]

    if coord is None:
        cx, cy = w / 2, h / 2
    else:
        cx, cy = [zoom * c for c in coord]

    img = cv2.resize(img, (0, 0), fx=zoom, fy=zoom)
    img = img[int(round(cy - h / zoom * .5)): int(round(cy + h / zoom * .5)),
              int(round(cx - w / zoom * .5)): int(round(cx + w / zoom * .5)),
              :]

    return img

```

Figure 5.10: function which we used to enable zooming mode

5.2.4.1 PKLot

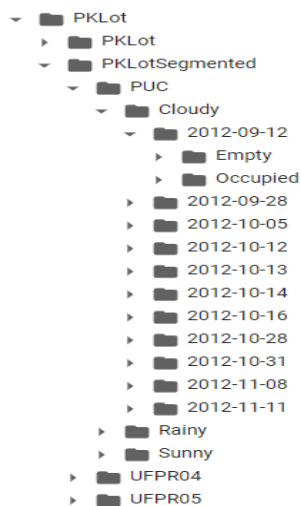


Figure 5.11: Original folder structure of the segmented images folder in the PKLot Dataset

The process involved extracting the file paths of all segmented images from each folder within the dataset. These file paths were then used to organize the images into two main folders: "train" and "test". Within each of these folders, two additional folders were created: "occupied" and "empty".

Before moving the images into their respective folders, a step of shuffling the data was performed to ensure randomness and prevent any bias in the training and testing sets. The data was divided in such a way that approximately 80% of the images were allocated for training, while the remaining 20% were set aside for testing.

```

#operate on Empty or occupied parking space (division)
def split_data(empty_directory_p,occupied_directory_p,path_test_vacant,path_test_occupied,path_train_vacant,path_train_occupied):
    import shutil , os
    from random import shuffle

    for empty in empty_directory_p:
        E=os.listdir(empty)
        shuffle(E)
        E1=round(len(E)*.80)
        empty_train=E[:E1]
        empty_test=E[E1:]

        for empty_train_1 in empty_train:
            shutil.move(empty+'/'+empty_train_1,path_train_vacant)

        for empty_test_1 in empty_test:
            shutil.move(empty+'/'+empty_test_1,path_test_vacant)

    #-----
    for occupied in occupied_directory_p:
        O=os.listdir(occupied)
        shuffle(O)
        O1=round(len(O)*.80)
        occupied_train=O[:O1]
        occupied_test=O[O1:]

        for occupied_train_1 in occupied_train :
            shutil.move(occupied+'/'+occupied_train_1,path_train_occupied)

        for occupied_test_1 in occupied_test:
            shutil.move(occupied+'/'+occupied_test_1,path_test_occupied)

    #-----

```

Figure 5.12: function which we used for splitting the whole data into train and test

5.2.4.2 CNRPark-EXT

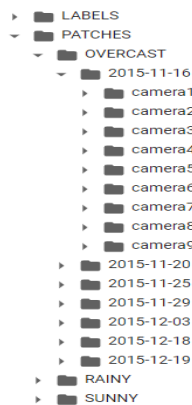


Figure 5.13: Original folder structure of the segmented images folder in the CNRPark-EXT Dataset

We extracted the file paths of all images from each weather folder then also moved them to the main train and test folder after applying the splitting function on them.

```
def move_data(path_label,number_of_camera):
    import shutil

    ref_to_file=open(path_label,'r')

    for line in ref_to_file:
        l=line.split(" ")

        if not l[0].find('SUNNY'):
            if int(l[1])==1:
                shutil.move(line.split(" ")[0], '/content/CNR-EXT/Sunny/C-{}/occupied'.format(number_of_camera))
            else:
                shutil.move(line.split(" ")[0], '/content/CNR-EXT/Sunny/C-{}/vacent'.format(number_of_camera))
        elif not l[0].find('RAINY'):
            if int(l[1])==1:
                shutil.move(line.split(" ")[0], '/content/CNR-EXT/Rainy/C-{}/occupied'.format(number_of_camera))
            else:
                shutil.move(line.split(" ")[0], '/content/CNR-EXT/Rainy/C-{}/vacent'.format(number_of_camera))
        else:
            if int(l[1])==1:
                shutil.move(line.split(" ")[0], '/content/CNR-EXT/OverCast/C-{}/occupied'.format(number_of_camera))
            else:
                shutil.move(line.split(" ")[0], '/content/CNR-EXT/OverCast/C-{}/vacent'.format(number_of_camera))
```

Figure 5.14: function which we used for extracting images from the main folders

```
from keras.preprocessing.image import ImageDataGenerator
def generate_data(path_to_data_dir):
    my_gen=ImageDataGenerator(1./255)

    data_gen=my_gen.flow_from_directory(
        path_to_data_dir,
        target_size=(150,150),
        batch_size=128,
        class_mode='categorical',
        shuffle=True
    )
    return data_gen
```

Figure 5.15: Data Generation Function

5.2.5 Our proposed architecture of the CNN model

Since our problem is simple binary classification, where the goal is to classify whether a slot is empty or not, a smaller model should be sufficient to capture the necessary features and achieve satisfactory accuracy. It eliminates the need for excessive parameters and complex architectures, making it easier to deploy and run the model in real-time on devices with limited computational capabilities. using a pretrained model in this case could lead to the model have a large number of parameters, making it relatively larger in size. The pretrained model is typically trained on diverse and extensive datasets such as ImageNet, which consists of millions of images across numerous categories. By training on such a large dataset, the pretrained model learns a wide range of features and patterns that may not be directly relevant to our specific problem of binary slot classification. This can lead to unnecessary computation and processing of irrelevant features.

On the other hand, building a model from scratch allows us to design a smaller model that is more lightweight and can be easily deployed on resource-constrained devices like a Raspberry Pi camera. This is particularly important when considering the limited computational resources and memory capacity of such devices, so we preferred to build our model from scratch.

Our model is built using the Sequential API from Keras. It starts with a Conv2D layer with 32 filters, a kernel size of 3x3, and a ReLU activation function. The input shape is set to (150, 150, 3) to accommodate images with width and height of 150 pixels and 3 color channels. After the first Conv2D layer, a MaxPooling2D layer with a pool size of 2x2 is added to perform down-sampling and extract the most important features. The model continues with another Conv2D layer with 32 filters and a kernel size of 3x3, followed by another MaxPooling2D layer. To prepare the features for classification, the output of the previous layers is flattened into a 1D vector using the Flatten layer.

Next, a Dense layer with 64 neurons and ReLU activation function is added to introduce non-linearity and learn more complex patterns in the data. The final layer of the model is a Dense layer with 2 neurons and a softmax activation function, which produces the probabilities for the two classes in the classification task.

In our experiments, we explored different configurations for the last layer of the model in the context of binary classification. One approach we tried was using a single neuron in the last layer to directly output the probability of the slot being occupied. However, we observed that this configuration resulted in lower accuracy compared to using two neurons in the last layer.

The reason behind this observation lies in the nature of the binary classification problem. While it might seem intuitive to use a single neuron to represent the binary outcome (occupied or empty), it is important to consider the complexity of the underlying data and the model's capacity to learn from it. By using two neurons in the last layer, we provide the model with the flexibility to learn and represent more nuanced patterns and decision boundaries.

With softmax activation, each neuron in the output layer represents the probability of belonging to a specific class. In this case, one neuron represents the probability of belonging to the first class, and the other neuron represents the probability of belonging to the second class.

During training, the model learns to assign higher probabilities to the appropriate class for each input instance. By having two neurons, the model can capture and distribute the relevant features and patterns associated with each class to the respective neuron. This allows for more flexibility in capturing complex relationships between input features and class labels.

In summary, using two neurons with softmax activation enables the model to distribute and assign the features and characteristics of each class to the appropriate neuron, facilitating the classification process and enhancing the model's ability to capture distinct patterns for each class.

Model: "sequential"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 148, 148, 32)	896
max_pooling2d (MaxPooling2D)	(None, 74, 74, 32)	0
conv2d_1 (Conv2D)	(None, 72, 72, 32)	9248
max_pooling2d_1 (MaxPooling2D)	(None, 36, 36, 32)	0
flatten (Flatten)	(None, 41472)	0
dense (Dense)	(None, 64)	2654272
dense_1 (Dense)	(None, 2)	130

=====
Total params: 2,664,546
Trainable params: 2,664,546
Non-trainable params: 0

Figure 5.16: Summary of our CNN Model Architecture

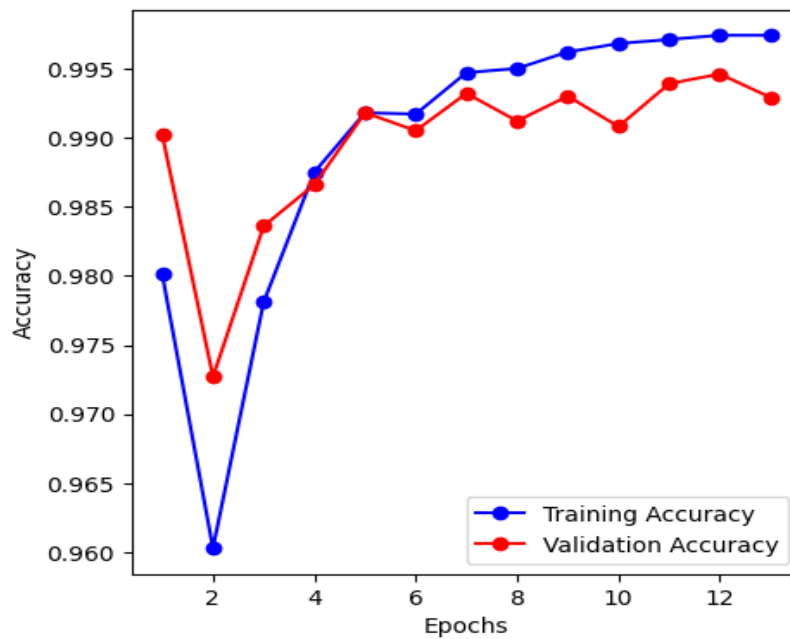


Figure 5.17: Training accuracy vs validation accuracy when training after the first 13 epochs

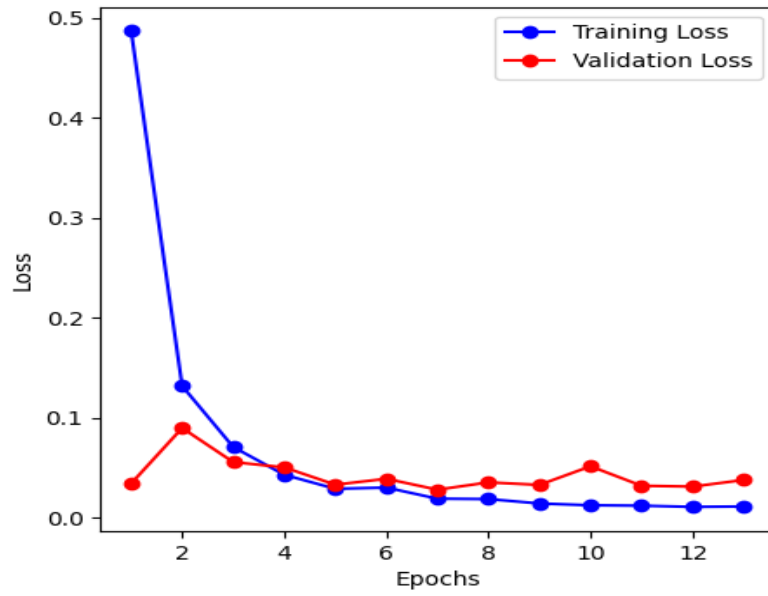


Figure 5.18: Training loss vs validation loss when training after the first 13 epochs

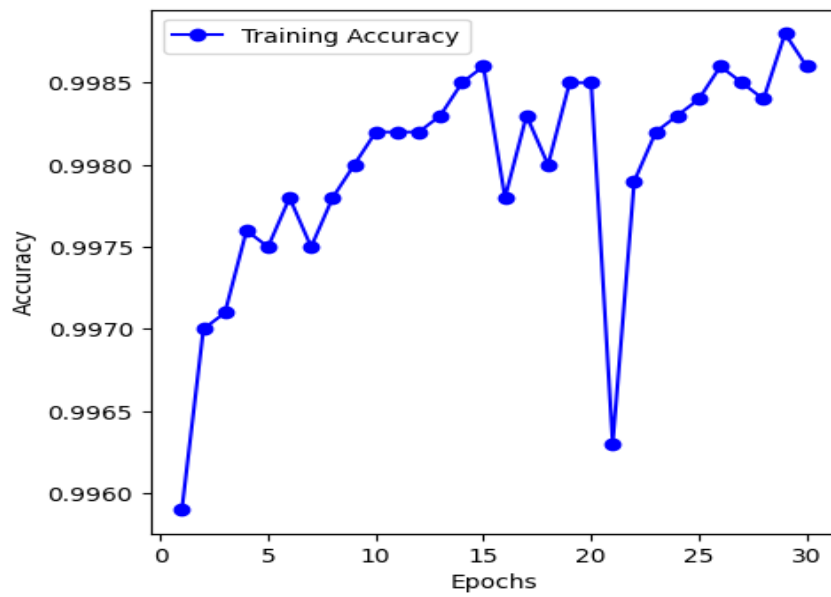


Figure 5.19: Training accuracy when training after the second 30 epochs.

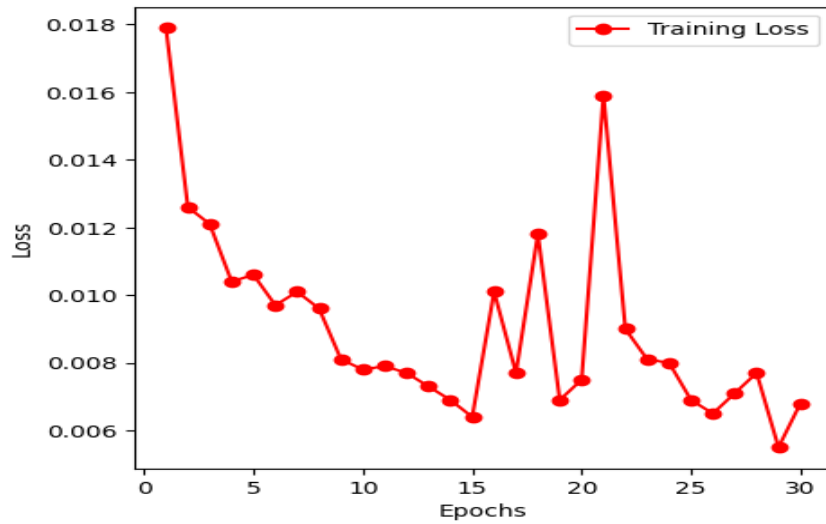


Figure 5.20: Training loss when training after the second 30 epochs.

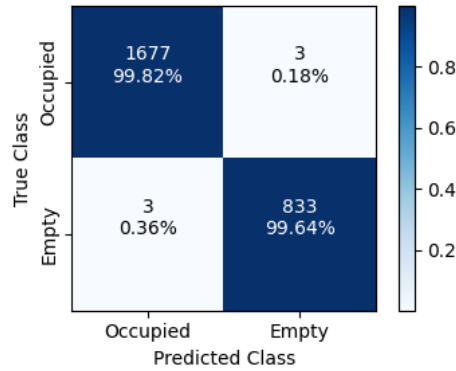


Figure 5.21: Confusion matrix of the testing experiment of the model trained on CNR dataset.

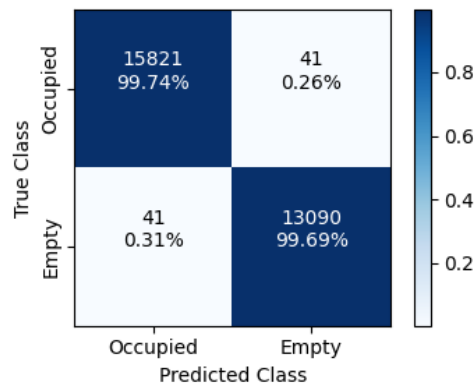


Figure 5.22: Confusion matrix of the testing experiment of the model trained on CNR-EXT dataset.

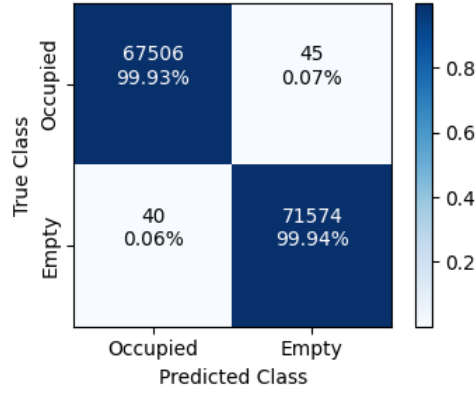


Figure 5.23: Confusion matrix of the testing experiment of the model trained on PKLot dataset.

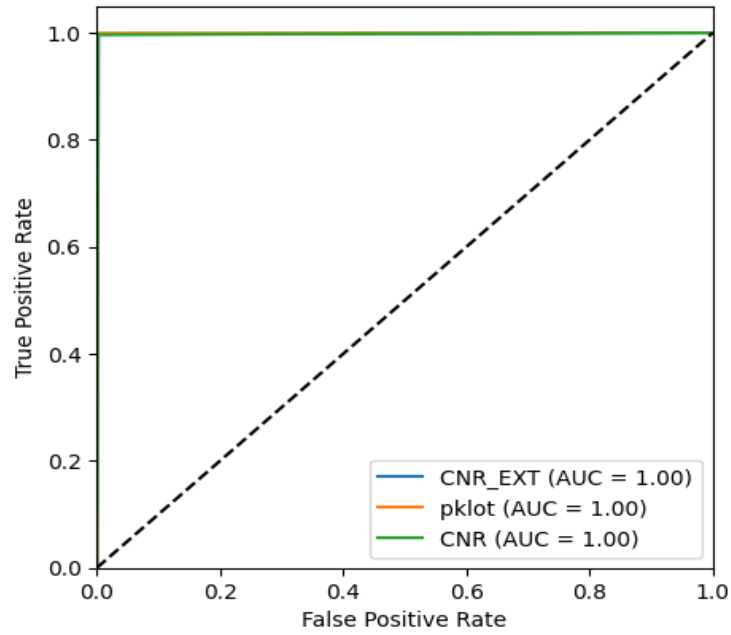


Figure 5.24: The ROC of the testing experiment of the model trained on multiple parking

This confirms the ability of our model to absorb the variability in camera angles, weather conditions, and surface texture.

We used another dataset called “Parking Lot Occupancy Database” [30]. This dataset contains images of a complex parking scenario (day and night, perspective occlusions, background occlusions, etc.) (see Figure 5.25). However, it is important to note that this dataset presented several challenges.

Firstly, the height of the camera used in capturing the images was not ideal, as it was not positioned at an optimal height above the parking slots. The camera should be positioned at the top of a building to minimize the possible occlusion between adjacent vehicles

Furthermore, the camera was placed inside a room, which introduced additional complexities such as reflections, shadows, and occlusions caused by objects within the room. These factors added to the difficulty of accurately detecting and classifying the occupancy of the parking slots.

Moreover, the dataset included cases where a parking slot could accommodate one or two cars, but this variability did not impact our model's evaluation. However, it is important to note that it is crucial to ensure that such variability does not exist in our smart parking system's implementation.

Despite the aforementioned challenges posed by the "Parking Lot Occupancy Database" dataset, we decided to test our model on this dataset to evaluate its performance under these conditions. However, before conducting the full evaluation, we performed a retraining process using a subset of images from this dataset. This retraining step was crucial as it allowed our model to adapt to the specific characteristics of the dataset, particularly the poor image quality and night mode conditions. This approach ensured that our model learned to extract meaningful features from the images despite their lower quality and varying conditions. By retraining the model with this subset of images, we aimed to improve its performance and increase its robustness in handling real-world parking scenarios with similar characteristics.

While the "Parking Lot Occupancy Database" dataset introduced challenges like the height of the camera, it is important to note that our solution was not specifically designed to address this. Our focus was on implementing a smart parking system according to predefined guidelines, including considerations such as camera positioning and the presence of multiple cars in a single slot. As previously mentioned, a retraining process using a subset of images from this dataset to allow our model to better handle the complexities associated with poor image quality and low-light environments.

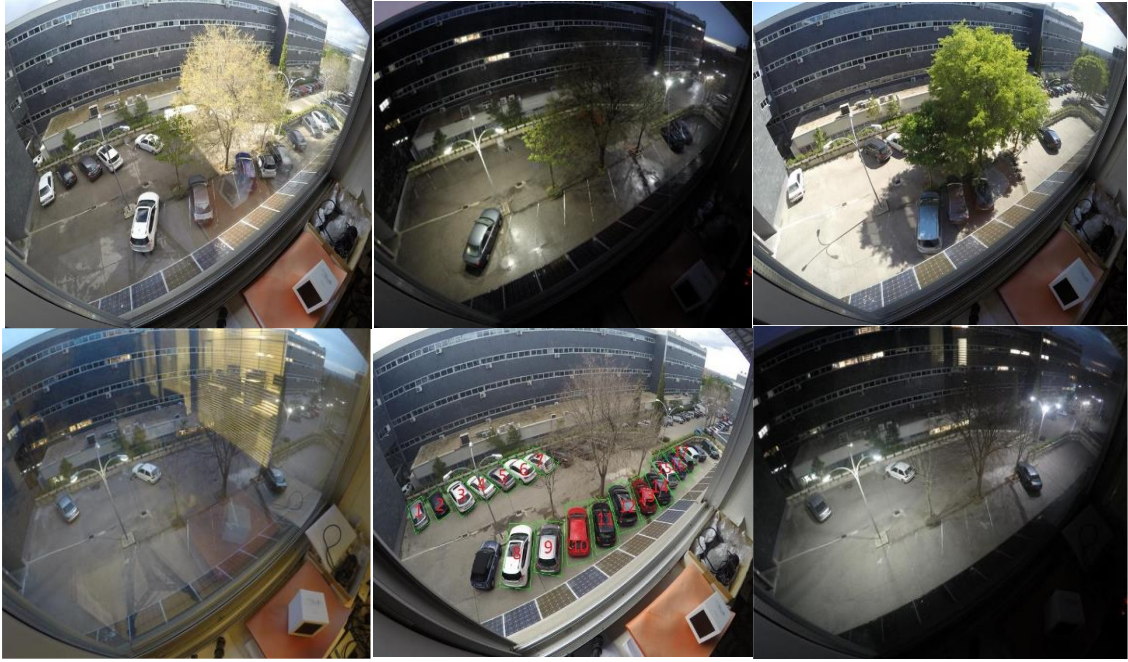


Figure 5.25: Images captured under different weather conditions from Parking Lot Occupancy Database.

During the evaluation of our model, we selected the first 17 slots for analysis, excluding the more challenging slots from the assessment. Despite these challenges, our model achieved a very good accuracy of **95.7%**. This performance is particularly noteworthy considering the dataset's complexities, such as the height of camera which is not expected to arise when deploying our smart parking system and most of the error that comes from the difference between coordinates for each one that may be make the patch[slot] difference in their appearance from other , one may be contain partial of vehicle and the other maybe not, In order to address this issue, we have implemented gates for each parking slot in our smart parking system. These gates serve to prevent inconsistencies and variations in appearance that may occur due to differences in the ground truth and the model's predictions. By incorporating these measures, we aim to build a robust and accurate smart parking system that mitigates such challenges.

5.2.6 Deployment of the proposed solution in a real scenario within the same parking space

As we already said, smart cameras were built around Raspberry Pi 2 model B, equipped with standard Raspberry Pi camera modules. Each smart camera has been mounted in an outdoor camera box and has been installed on the roof of a building in front of the parking lot (see Figure 5.26)

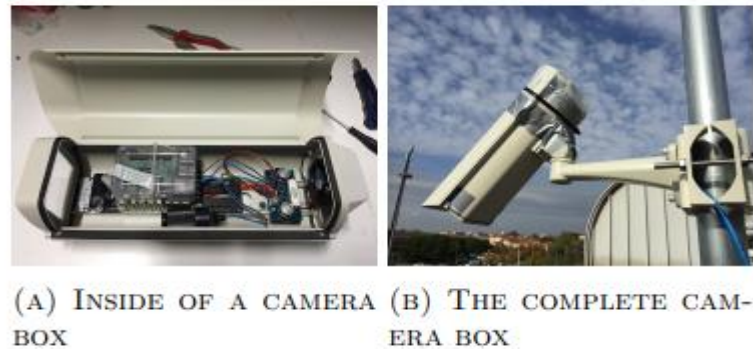


Figure 5.26: Each Raspberry Pi is mounted inside a outdoor camera box (Figure A on the left) and it is mounted on top of the roof of the building, attached to a steel pole (Figure B on the right).

The entire framework was deployed in the parking lot of the research campus of the CNR in Pisa as a Smart City application. The monitored parking lot consists of 164 parking spaces, organized in five rows, four of which are composed of about 35 parking spaces each, and one row is composed of 18 parking spaces. Although a single Raspberry Pi equipped with the standard camera module is able to monitor more than 50 parking spaces (i.e. with the given height and distance of the cameras from the parking lot), due to the conformation of the parking lot monitored, CNR-EXT had to deploy 9 smart cameras in order to monitor all the parking spaces. Some cameras monitor about 20 parking spaces, while some others more than 50.

Due to the angle of view and the perspective of the camera module used, and the position of the smart cameras, most of the parking spaces closest to the building are monitored by just one camera, while the parking spaces farthest from the building are monitored by more cameras. they used the redundancy of the overlapping parking spaces to reduce occlusion problems (for examples trees).

In particular, they assigned a weight value to each pair (parking-space, camera), representing how good is the view of that parking space seen by that camera. A high value of this parameter means that the parking space is in the center of the image and that there is no obstacle between the camera and the parking space. The confidence returned by the CNN for a given parking space is weighted with this value and, in case of a parking space monitored by more cameras, the highest weighted confidence value is selected at server side. In this way, they are able to correct classification errors on parking spaces occluded for some cameras, by choosing the confidence value of another camera that has a clearer view of that parking space.

Using smart cameras, rather than ground sensors, has two relevant advantages: lower cost per parking space and versatility. The cost of a Raspberry Pi equipped with a camera module is about 34.95\$, and the outdoor camera box with pole support has about the same price. These are very limited costs, as shown in (Amato et al., 2016 [29]) and in our model, the accuracy of this approach is very good, and it is comparable to the accuracy of a ground sensor. Therefore, it is possible to monitor a large parking lot with a cost per parking space which is an order of magnitude lower than ground sensors, and still achieving comparable accuracy results.

Moreover, with smart cameras we are not limited solely to parking lots monitoring applications. We could exploit the smart camera to perform additional activities like, for example, video surveillance activities, such as face/people recognition, or tracking and logging of people and moving vehicles.

5.2.7 Real Implementation on a camera



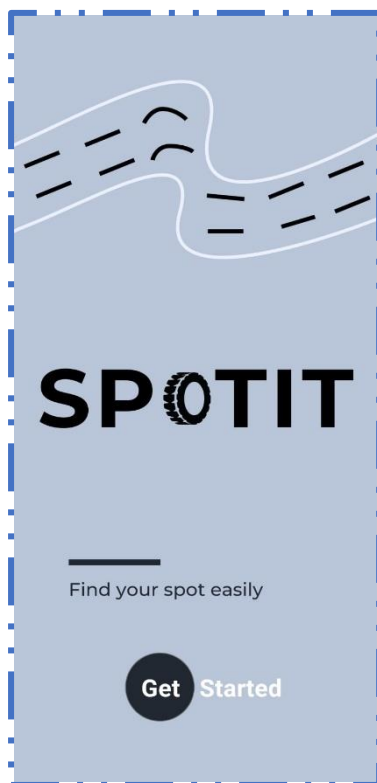
Figure 5.27: Example of classification of a portion of the parking lot.

The software running on the smart cameras periodically captures the image of a portion of the parking lot and, for each parking space, determines the occupancy status by using a CNN trained offline. Pictures captured by cameras are filtered by a mask that identifies the various parking spaces. The mask was built manually once and for all. Examples of masks built for different cameras are shown in Figure 5.8. At run time, each frame is automatically segmented in patches (we recall that a patch is the portion of the original image containing a single parking space) corresponding to the parking spaces monitored by that camera, by using the generated masks. Every patch is then classified using the trained CNN, to decide whether the corresponding parking space is empty or busy, if the status over all patches already changed from previous status stored in camera the camera will store new status and send it for sever to update her status on the server and wait to the acknowledgement that must be come from the server to ensure his receiving for new status if not the camera will resend the status again to the server . On the Raspberry Pi 2, the classification of 50 parking spaces and the transmission of the results to a web server takes about 15 seconds. Figure 5.27 shows an example of the classification of a portion of the parking lot monitored by a smart camera. As can be seen in figure, our approach deals very well with common challenging visual classification problems such as shadows, obstacles (trees or lamps) or even people occupying the parking spaces. A key aspect of the proposed solution is its decentralized strategy and the delegation of the parking decision to the smart cameras themselves. This solution has the clear advantage of being scalable, as it requires no additional elaboration on the server side. In a centralized solution, images of the parking lot acquired at high resolution (of about 3MB each) are sent to the server which would thus become a bottleneck and a single point of failure.

5.3 Mapping Design to Implementation:

We used flutter in our Mobile APP Because Flutter is as good as everyone says and even better. This framework grabs the attention of many companies and development teams and some have already started making commercial apps with Flutter. We also worked on web native programming as back end.

5.3.1 Implementation to Flutter:



```
import 'dart:ui';

import 'package:flutter/cupertino.dart';
import 'package:flutter/material.dart';
import 'package:flutter/painting.dart';
import 'package:flutter/rendering.dart';
import 'package:flutter_bloc/flutter_bloc.dart';
import 'package:spotit/bloc/gps_cubit.dart';
import 'package:spotit/cach.dart';
import 'package:spotit/cubit/app_cubit.dart';
import 'package:spotit/locatin.dart';
import 'package:spotit/secondpage.dart';
import 'package:provider/provider.dart';
import 'Login_screen.dart';
import 'cubit/bloc_observer.dart';
import 'layout_screen.dart';

void main() async {
  WidgetsFlutterBinding.ensureInitialized();
  Bloc.observer = MyBlocObserver();
  await CacheHelper.init();
  runApp(const MyApp());
}

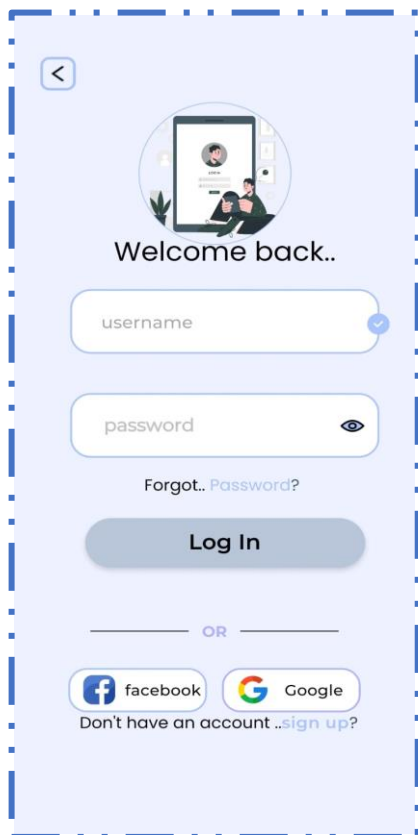
class MyApp extends StatelessWidget {
  const MyApp({Key? key}) : super(key: key);

  @override
  Widget build(BuildContext context) {
    return ChangeNotifierProvider(
      create: (context) => HomeModel(),
      child: BlocProvider(
        create: (context) => AppCubit()..getHistory(),
        child: MaterialApp(
          debugShowCheckedModeBanner: false,
          home: LoginScreen(),
        ),
      ),
    );
  }
}

class firstpage extends StatefulWidget {
  const firstpage({Key? key}) : super(key: key);

  @override
  State<firstpage> createState() => _firstpageState();
}

class _firstpageState extends State<firstpage> {
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      body: Stack(
        children: [
          Container(
            decoration: BoxDecoration(color: Color(0xffb9c6d9)),
          ),
          Positioned.fill(
            bottom: 450,
            child: Container(
              decoration: BoxDecoration(
                image: DecorationImage(
                  image: AssetImage("assets/images/firstscreen1.png")),
              ),
            ),
          ),
          Positioned.fill(
            top: 10,
            child: Padding(
              padding: const EdgeInsets.all(30),
              child: Container(
                decoration: BoxDecoration(
                  image: DecorationImage(
                    image: AssetImage("assets/images/firstscreen2.png")),
                  ),
                ),
              ),
            ),
          Positioned.fill(
            top: 395,
            right: 60,
            child: Container(
              decoration: BoxDecoration(
                image: DecorationImage(
                  fit: BoxFit.cover,
                  image: AssetImage("assets/images/blackline5.png")),
                ),
              ),
            ),
          ),
        ],
      ),
    );
  }
}
```



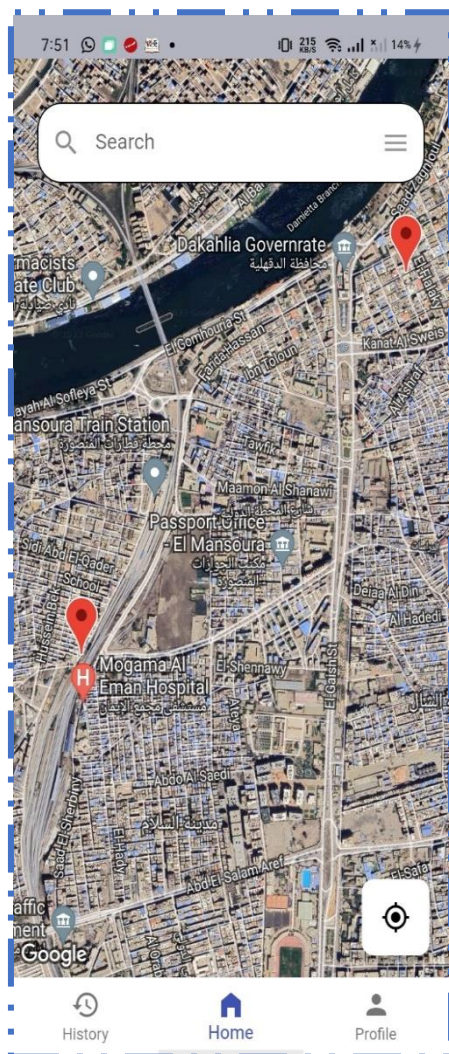
```
import 'dart:convert';
import 'dart:developer';
import 'dart:ui';
import 'package:http/http.dart' as http;
import 'package:flutter/cupertino.dart';
import 'package:flutter/material.dart';
import 'package:flutter/painting.dart';
import 'package:flutter/rendering.dart';
import 'package:flutter/widgets.dart';
import 'package:shared_preferences/shared_preferences.dart';
import 'package:spotit/constant.dart';
import 'package:spotit/forgetemailone.dart';
import 'package:spotit/layout_screen.dart';
import 'package:spotit/locatin.dart';
import 'package:spotit/secondpage.dart';
import 'package:provider/provider.dart';
import 'package:spotit/register_screen.dart';
import 'package:spotit/widget/core_components.dart';

class LoginScreen extends StatefulWidget {
  const LoginScreen({Key? key}) : super(key: key);

  @override
  State<LoginScreen> createState() => _LoginScreenState();
}

class _LoginScreenState extends State<LoginScreen> {
  final String apiUrl = '${baseUrl}login'; //replace with your API URL
  final TextEditingController email = TextEditingController();
  final TextEditingController password = TextEditingController();

  @override
  Widget build(BuildContext context) {
    final model = Provider.of<HomeModel>(context);
    return Scaffold(
      backgroundColor: Color(0xffe0f0ff),
      body: SingleChildScrollView(
        child: Stack(
          children: [
            Padding(
              padding: const EdgeInsets.only(left: 40, top: 70),
              child: Container(
                width: 35,
                height: 35,
                decoration: BoxDecoration(
                  borderRadius: BorderRadius.all(Radius.circular(10)),
                  border: Border.all(
                    color: Color(0xffabc4fd),
                    width: 2,
                  ),
                ),
              ),
            ),
            Padding(
              padding: const EdgeInsets.only(top: 59, left: 29),
              child: Container(
                child: IconButton(
                  onPressed: () {
                    Navigator.push(
                      context,
                      MaterialPageRoute(
                        builder: (context) => onsecondpage(),
                      ),
                    );
                  },
                  icon: Icon(
                    Icons.chevron_left,
                    size: 41,
                  ),
                ),
              ),
            ),
            Padding(
              padding: const EdgeInsets.only(top: 110, left: 110),
              child: Container(
                child: CircleAvatar(
                  radius: 25,
                  backgroundImage: AssetImage(
                    "assets/images/fifthscreen.png",
                  ),
                ),
                backgroundColor: Color(0xffe0f0ff),
                child: Container(
                  decoration: BoxDecoration(
                    border: Border.all(
                      color: Color(0xffabc4fd),
                    ),
                    borderRadius: BorderRadius.circular(80),
                  ),
                ),
              ),
            ),
            Padding(
              padding: const EdgeInsets.only(top: 268, left: 89),
              child: Container(
                child: Column(
                  children: [
                    Text(
                      "Welcome back..",
                      style: TextStyle(fontSize: 25, fontWeight: FontWeight.bold),
                    ),
                    Text(
                      "username",
                      style: TextStyle(fontSize: 16, color: Color(0xff000000)),
                    ),
                    Text(
                      "password",
                      style: TextStyle(fontSize: 16, color: Color(0xff000000)),
                    ),
                    Text(
                      "Forgot.. Password?",
                      style: TextStyle(fontSize: 14, color: Color(0xff000000)),
                    ),
                    Text(
                      "Log In",
                      style: TextStyle(fontSize: 16, color: Color(0xff000000)),
                    ),
                    Text(
                      "OR",
                      style: TextStyle(fontSize: 16, color: Color(0xff000000)),
                    ),
                    Text(
                      "facebook",
                      style: TextStyle(fontSize: 16, color: Color(0xff000000)),
                    ),
                    Text(
                      "Google",
                      style: TextStyle(fontSize: 16, color: Color(0xff000000)),
                    ),
                    Text(
                      "Don't have an account ..sign up?",
                      style: TextStyle(fontSize: 14, color: Color(0xff000000)),
                    ),
                  ],
                ),
              ),
            ),
            Padding(
              padding: const EdgeInsets.only(top: 330, left: 30, right: 30),
              child: TextFieldWidget(
                controller: email,
                prefixIconData: Icons.mail,
                hintText: "E-mail",
                obscureText: false,
                suffixIconData:
                  model.isValid ? Icons.check_circle : Icons.check_circle,
                onChanged: (value) {
                  model.inputemail(value);
                },
              ),
            ),
            Padding(
              padding: const EdgeInsets.only(top: 420, left: 30, right: 30),
              child: TextFieldWidget(
                controller: password,
                prefixIconData: Icons.lock,
                hintText: "Password",
                obscureText: model.isVisible ? false : true,
              ),
            ),
          ],
        ),
      ),
    );
  }
}
```



```
import 'dart:async';

import 'package:flutter/material.dart';
import 'package:flutter_bloc/flutter_bloc.dart';
import 'package:google_maps_flutter/google_maps_flutter.dart';
import 'package:provider/provider.dart';
import 'package:spotit/bloc/gps_cubit.dart';
import 'package:spotit/bloc/gps_state.dart';
import 'package:spotit/cubit/app_cubit.dart';
import 'package:spotit/cubit/app_states.dart';
import 'package:spotit/home.dart';
import 'package:spotit/layout_screen.dart';

import 'Login_screen.dart';
import 'widget/custom_text_field.dart';

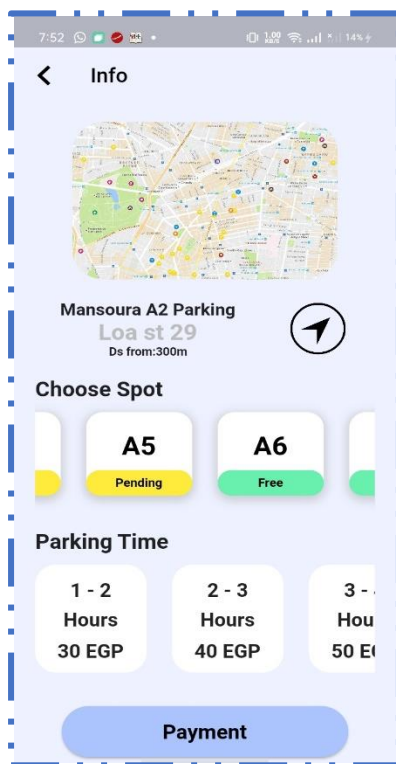
class LocationViewBody extends StatelessWidget {
  TextEditingController searchController = TextEditingController();
  final Completer<GoogleMapController> _controller =
    Completer<GoogleMapController>();

  LocationViewBody({Key? key}) : super(key: key);

  initState() {}

  @override
  Widget build(BuildContext context) {
    return Stack(
      children: [
        BlocConsumer<MapCubit, MapState>(
          listener: (context, state) {},
          builder: (context, state) {
            if (state.currentLocation == null) {
              return const Center(
                child: CircularProgressIndicator(),
              );
            } else {
              var cubit = MapCubit.get(context);

              return GoogleMap(
                mapType: MapType.hybrid,
                mapToolbarEnabled: false,
                zoomControlsEnabled: false,
                initialCameraPosition: CameraPosition(
                  target: state.currentLocation!,
                  zoom: 15,
                ),
                markers: state.currentLocation != null
                  ? <Marker>{
                      Marker(
                        markerId: const MarkerId('Current_location'),
                        position: state.currentLocation!,
                        infoWindow: const InfoWindow(
                          title: 'Current Location',
                        ),
                      ),
                      Marker(
                        markerId: const MarkerId("Mansoura 1 "),
                        position: LatLng(31.041230, 31.385680),
                        infoWindow: const InfoWindow(
                          title: "Mansoura 1 ",
                        ),
                        onTap: () {
                          AppCubit.get(context).chooseParking('1');
                          Navigator.push(
                            context,
                            MaterialPageRoute(
                              builder: (context) => HomeScreen(),
                            ),
                          );
                        },
                      ),
                      Marker(
                        markerId: const MarkerId("Mansoura 2"),
                        position: LatLng(31.0514989, 31.397176),
                        infoWindow: const InfoWindow(
                          title: "Mansoura 2",
                        ),
                        onTap: () {
                          AppCubit.get(context).chooseParking('1');
                          Navigator.push(
                            context,
                            MaterialPageRoute(
                              builder: (context) => HomeScreen(),
                            ),
                          );
                        },
                      ),
                    ] : null,
              );
            }
          },
        ),
      ],
    );
  }
}
```



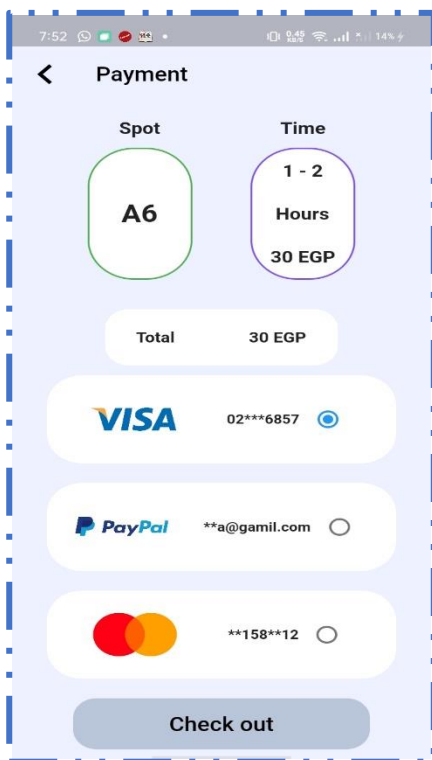
```
import 'dart:convert';
import 'dart:ui';
import 'package:conditional_builder_null_safety/conditional_builder_null_safety.dart';
import 'package:flutter_bloc/flutter_bloc.dart';
import 'package:http/http.dart' as http;

import 'package:flutter/cupertino.dart';
import 'package:flutter/material.dart';
import 'package:flutter/painting.dart';
import 'package:flutter/rendering.dart';
import 'package:flutter/widgets.dart';
import 'package:spotit/constant.dart';
import 'package:spotit/cubit/app_cubit.dart';
import 'package:spotit/locatin.dart';
import 'package:spotit/payment.dart';
import 'package:spotit/widget/core_components.dart';
import 'package:spotit/widget/custom_button.dart';

import 'cubit/app_states.dart';

class HomeScreen extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return BlocConsumer<AppCubit, AppStates>(
      listener: (context, state) {},
      builder: (context, state) {
        AppCubit cubit = AppCubit.get(context);
        return Scaffold(
          backgroundColor: Color(0xffeeffff),
          appBar: AppBar(
            title: Text(
              'Info',
              style: TextStyle(color: Colors.black),
            ),
            backgroundColor: Colors.transparent,
            elevation: 0.0,
            leading: IconButton(
              onPressed: () {
                Navigator.pop(context);
              },
              icon: Icon(
                Icons.chevron_left,
                color: Colors.black,
                size: 41,
              ),
            ),
            body: SingleChildScrollView(
              child: Padding(
                padding: const EdgeInsets.all(20.0),
                child: Column(
```

```
ConditionalBuilder(
  condition: state is AppChooseParkingLoadingState,
  builder: (context) => const Center(
    child: CircularProgressIndicator(),
  ),
  fallback: (context) => SizedBox(
    height: 110,
    child: ListView.separated(
      padding: EdgeInsets.symmetric(vertical: 10),
      scrollDirection: Axis.horizontal,
      itemBuilder: (context, index) => SpotItem(
        item: cubit.parkingSlots[index],
        index: index,
      ),
      separatorBuilder: (context, index) => const SizedBox(
        width: 24,
      ),
      itemCount: cubit.parkingSlots.length,
    ),
  ),
  const SizedBox(
    height: 20,
  ),
  Row(
    children: [
      Text(
        "Parking Time",
        style: TextStyle(
          fontSize: 21,
          fontWeight: FontWeight.bold,
        ),
      ),
    ],
  ),
  SizedBox(
    height: 130,
    child: ListView.separated(
      padding: EdgeInsets.symmetric(vertical: 10),
      scrollDirection: Axis.horizontal,
      itemBuilder: (context, index) => MakeParkingtime(
        model: cubit.times[index],
        index: index,
        context: context,
      ),
      separatorBuilder: (context, index) => const SizedBox(
        width: 24,
      ),
      itemCount: cubit.times.length,
    ),
  ),
),
),
```



```
import 'dart:ui';

import 'package:flutter/cupertino.dart';
import 'package:flutter/material.dart';
import 'package:flutter/painting.dart';
import 'package:flutter/rendering.dart';
import 'package:flutter/widgets.dart';
import 'package:spotit/cubit/app_cubit.dart';

import 'Qrcode.dart';

class payment extends StatefulWidget {
  TimeModel model;
  Map<String, dynamic> spot;
  payment({Key? key, required this.model, required this.spot})
    : super(key: key);

  @override
  State<payment> createState() => _paymentState(model: model, spot: spot);
}

class _paymentState extends State<payment> {
  int _value = 1;
  TimeModel model;
  Map<String, dynamic> spot;
  _paymentState({Key? key, required this.model, required this.spot});
  //_paymentState(());
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: Text(
          'Payment',
          style: TextStyle(color: Colors.black),
        ),
        backgroundColor: Colors.transparent,
        elevation: 0.0,
        leading: IconButton(
          onPressed: () {
            Navigator.pop(context);
          },
          icon: Icon(
            Icons.chevron_left,
            color: Colors.black,
            size: 41,
          ),
        ),
        backgroundColor: Color(0xffeef1ff),
      ),
      body: SafeArea(
        child: SingleChildScrollView(
          child: Stack(
            children: [
              Column(crossAxisAlignment: CrossAxisAlignment.start, children:
                Padding(
                  padding: const EdgeInsets.symmetric(horizontal: 30),
                  child: Container(
                    value: 1,
                    groupValue: _value,
                    onChanged: (int? val) {
                      setState(() {
                        _value = val!;
                      });
                    },
                  ),
                ),
              ),
            ),
            padding: const EdgeInsets.symmetric(vertical: 10),
            child: Center(
              child: Container(
                decoration: BoxDecoration(
                  color: Colors.white,
                  borderRadius: BorderRadius.circular(20)),
                child: Container(
                  height: 52,
                  width: 255,
                  decoration: BoxDecoration(
                    color: Color(0xffb9c6d9),
                    borderRadius:
                      BorderRadius.all(Radius.circular(20))),
                child: Row(
                  children: [
                    Expanded(
                      child: TextButton(
                        onPressed: () {
                          AppCubit.get(context).lastPayment(
                            amount: model.price,
                            totalTime:
                              (model.to - model.from) * 60,
                            slotId: spot['id'],
                            context: context);
                        },
                      child: Text(
                        'Check out',
                        style: TextStyle(
                          color: Colors.black,
                          fontSize: 20,
                          fontWeight: FontWeight.w500),
                      ),
                    ),
                  ],
                ),
              ),
            ),
          ),
        ),
      ),
    );
  }
}
```

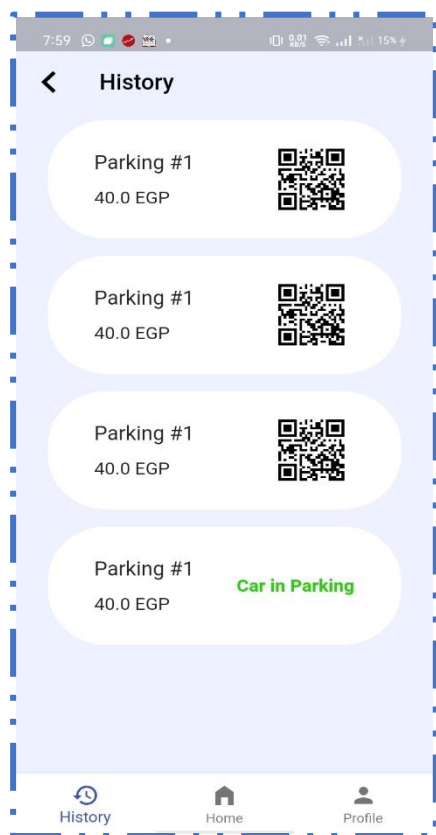


```
import 'dart:ui';

import 'package:conditional_builder_null_safety/conditional_builder_null_safety.dart';
import 'package:flutter/cupertino.dart';
import 'package:flutter/material.dart';
import 'package:flutter/painting.dart';
import 'package:flutter_bloc/flutter_bloc.dart';
import 'package:spotit/cubit/app_states.dart';

import 'cubit/app_cubit.dart';

class Qrcode extends StatelessWidget {
  String image;
  Qrcode(this.image);
  @override
  Widget build(BuildContext context) {
    return BlocConsumer<AppCubit, AppStates>(
      listener: (context, state) {},
      builder: (context, state) {
        AppCubit cubit = AppCubit.get(context);
        return Scaffold(
          backgroundColor: Color(0xffeef1ff),
          appBar: AppBar(
            title: Text(
              'QR Code',
              style: TextStyle(color: Colors.black),
            ),
            backgroundColor: Colors.transparent,
            elevation: 0.0,
            leading: IconButton(
              onPressed: () {
                Navigator.pop(context);
              },
              icon: Icon(
                Icons.chevron_left,
                color: Colors.black,
                size: 41,
              ),
            ),
          ),
          body: Stack(
            children: [
              Center(
                child: Padding(
                  padding:
                    const EdgeInsets.symmetric(vertical: 40, horizontal: 0),
                  child: Column(children: [
                    ConditionalBuilder(
                      condition: state is AppPaymentLoadingState,
                      builder: (context) => const Center(
                        child: CircularProgressIndicator(),
                      ),
                      fallback: (context) => SizedBox(
                        child: Container(
                          width: 320,
                          height: 320,
                          decoration: BoxDecoration(
                            image: DecorationImage(
                              fit: BoxFit.cover,
                              image: NetworkImage(
                                image))) //NetworkImage(),
                        ),
                      ),
                    ),
                  ]),
                ),
              ),
            ],
          ),
        );
      },
    );
  }
}
```



```
import 'dart:ui';

import 'package:flutter/material.dart';
import 'package:flutter/cupertino.dart';
import 'package:flutter/painting.dart';
import 'package:flutter/rendering.dart';
import 'package:flutter/widgets.dart';
import 'package:flutter_bloc/flutter_bloc.dart';
import 'package:spotit/Qrcode.dart';
import 'package:spotit/cubit/app_cubit.dart';
import 'package:spotit/cubit/app_states.dart';
import 'package:spotit/endpayment.dart';

import 'layout_screen.dart';

class history extends StatefulWidget {
  const history({Key? key}) : super(key: key);

  @override
  State<history> createState() => _historyState();
}

class _historyState extends State<history> {
  @override
  Widget build(BuildContext context) {
    return BlocConsumer<AppCubit, AppStates>(
      listener: (context, state) {},
      builder: (context, state) {
        AppCubit cubit = AppCubit.get(context);
        return Scaffold(
          backgroundColor: Color(0xffeef1ff),
          appBar: AppBar(
            title: Text(
              'History',
              style: TextStyle(color: Colors.black),
            ),
            backgroundColor: Colors.transparent,
            elevation: 0.0,
            leading: IconButton(
              onPressed: () {
                Navigator.pop(context);
              },
              icon: Icon(
                Icons.chevron_left,
                color: Colors.black,
                size: 41,
              ),
            ),
          ),
          body: ListView.builder(
            itemCount: cubit.historyList.length,
            itemBuilder: (context, index) {
              print(cubit.historyList[index]['Payment']['status']);
              return Padding(
                padding: const EdgeInsets.symmetric(vertical: 8),
                child: Center(
                  child: historywidget(
                    h1: "Parking #${cubit.historyList[index]['Slot']['parking_id']}",
                    h2: "${cubit.historyList[index]['Payment']['amount']} EGP",
                    status: cubit.historyList[index]['Request']['status'],
                    payment_id: cubit.historyList[index]['Payment']['id'],
                    qrcode: cubit.historyList[index]['Qrcode']['image'],
                  ),
                ),
              ),
            ),
          ),
        );
      },
    );
  }
}
```

We used cubit as a state management:

```
import 'dart:convert';
import 'dart:io';
import 'package:google_maps_flutter/google_maps_flutter.dart';
import 'package:http/http.dart' as http;
import 'package:bloc/bloc.dart';
import 'package:flutter/material.dart';
import 'package:flutter_bloc/flutter_bloc.dart';
import 'package:location/location.dart';
import 'package:spotit/QrCode.dart';
import 'package:spotit/bloc/gps_state.dart';
import 'package:spotit/cach.dart';
import 'package:spotit/locatin.dart';
import 'package:spotit/payment.dart';
import 'package:spotit/widget/core_components.dart';

import '../constant.dart';
import '../history.dart';
import '../home.dart';
import '../profile.dart';
import '../app_states.dart';
// import '../locatin.dart';
import 'package:location/location.dart';
import 'package:flutter_bloc/flutter_bloc.dart';
import 'package:equatable/equatable.dart';
import 'package:shared_preferences/shared_preferences.dart';

class AppCubit extends Cubit<AppState> {
  AppCubit() : super(AppInitialState());

  static AppCubit get(context) => BlocProvider.of(context);

  List<Widget> layoutScreens = [
    history(),
    locationView(),
    profile(),
  ];

  final Location location = Location();
  static Set<Marker> markers = {
    Marker(
      markerId: const MarkerId('Current_location'),
      position: Latlng(0.0, 0.0),
      infoWindow: const InfoWindow(
        title: 'Current Location',
      ),
    ),
  ];
};

int currentIndex = 1;
void changeBottomNavBarIndex(int index) {
  currentIndex = index;
  emit(AppChangeBottomNavBarState());
}

final String chooseParkingApiUrl =
  '${baseUrl}choose_parking'; //replace with your API URL
List<dynamic> parkingSlots = [];
Future<void> chooseParking(String id) async {
  parkingSlots = [];
  emit(AppChooseParkingLoadingState());
  final response = await http.post(
    Uri.parse(chooseParkingApiUrl),
    headers: {'Content-Type': 'application/json'},
    body: jsonEncode({
      "parking_id": id,
    }),
  );
  if (response.statusCode == 200) {
    final responseData = jsonDecode(response.body);
    parkingSlots = responseData['slots'];
    print(parkingSlots.toString());
    emit(AppChooseParkingSuccessState());
  } else {
    emit(AppChooseParkingErrorState());
    throw Exception('Failed to login: ${response.statusCode}');
    print('Failed to login: ${response.statusCode}');
  }
}
```

```
final String chckSlotApiUrl =
  '${baseUrl}choose_slot'; //replace with your API URL
Future<void> chckSpot({
  context,
}) async {
  emit(AppChckSpotLoadingState());
  final response = await http.post(
    Uri.parse(chckSlotApiUrl),
    headers: {'Content-Type': 'application/json'},
    body: jsonEncode({
      "slot_id": parkingSlots[selectedSpotIndex]['id'],
    }),
  );
  if (response.statusCode == 200) {
    final responseData = jsonDecode(response.body);
    String msg = responseData['msg'];
    if (msg == "Successful reservation") {
      navigateTo(
        context,
        payment(
          spot: parkingSlots[selectedSpotIndex],
          model: times[selectedTimeIndex],
        ),
      );
    } else {
      // todo pending
      print('pending');
    }
    emit(AppChckSpotSuccessState());
  } else {
    print(jsonDecode(response.body));
    emit(AppChckSpotErrorState());
    throw Exception('Failed to login: ${response.statusCode}');
    print('Failed to login: ${response.statusCode}');
  }
}

List<TimeModel> times = [
  TimeModel(price: 30, from: 1, to: 2),
  TimeModel(price: 40, from: 2, to: 3),
  TimeModel(price: 50, from: 3, to: 4),
  TimeModel(price: 60, from: 4, to: 5),
  TimeModel(price: 70, from: 5, to: 6),
  TimeModel(price: 80, from: 6, to: 7),
  TimeModel(price: 90, from: 7, to: 8),
  TimeModel(price: 100, from: 8, to: 9),
];

int? selectedSpotIndex;
int? selectedTimeIndex;

void changeSpot(int index) {
  selectedSpotIndex = index;
  emit(AppChangeSpotState());
}

void changeTime(int index) {
  selectedTimeIndex = index;
  emit(AppChangeTimeState());
}

final String paymentApiUrl = '${baseUrl}payment'; //replace with your API URL
Future<void> lastPayment(
  {required int amount,
  required int totalTime,
  required int slotId,
  required context}) async {
  emit(AppPaymentLoadingState());
  SharedPreferences prefs = await SharedPreferences.getInstance();
```

```
final response = await http.post(
  Uri.parse(paymentApiUrl),
  headers: <String, String>{
    'Content-Type': 'application/json; charset=UTF-8',
  },
  body: jsonEncode(requestBody),
);
final responseData = jsonDecode(response.body);
if (response.statusCode == 200) {
  final responseData = jsonDecode(response.body);
  final paymentId = responseData['payment_id'] as int?;
  if (paymentId != null) {
    final qrCodeUrl = await getQrCode(paymentId);
    print(qrCodeUrl);
    navigateTo(context, QrCode(qrCodeUrl));
    await getHistory();
    emit(AppPaymentSuccessState());
  } else {
    print('Response data did not contain payment_id');
    emit(AppPaymentErrorState());
  }
} else {
  print('POST request failed with status: ${response.statusCode}');
  emit(AppPaymentErrorState());
}

Future<dynamic> getQrCode(int id) async {
  final String apiUrl =
    '${baseUrl}get_qrcode/$id'; // replace with your API URL
  print(apiUrl);
  final response = await http.get(Uri.parse(apiUrl));
  if (response.statusCode == 200) {
    return jsonDecode(response.body);
  } else {
    throw Exception('Failed to Fetch data');
  }
}

List<dynamic> historyList = [];
Future<dynamic> getHistory() async {
  final String apiUrl =
    '${baseUrl}myresrvations/2'; // replace with your API URL
  print(apiUrl);
  final response = await http.get(Uri.parse(apiUrl));
  if (response.statusCode == 200) {
    historyList.clear();
    historyList = jsonDecode(response.body);
    print(historyList);
    print("=====");
    return jsonDecode(response.body);
  } else {
    throw Exception('Failed to Fetch data');
  }
}

class TimeModel {
  int price;
  int from;
  int to;

  TimeModel({required this.price, required this.from, required this.to});
}
```

Backend Implementation:

- Db connection

```
from sqlalchemy import create_engine, MetaData
from sqlalchemy.orm import sessionmaker
from sqlalchemy.ext.declarative import declarative_base
engine = create_engine("mysql+pymysql://root@localhost:3306/parking"
)
SessionLocal = sessionmaker(autocommit=False, bind=engine)
Base = declarative_base()

def get_db():
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()

Base.metadata.create_all(bind=engine)
```

users' function and auth handler with JWT

```
from typing import List
from fastapi import APIRouter, Depends, HTTPException
from config.db import *
from models.index import *
from schemas.index import *
from controllers.auth import AuthHandler
from sqlalchemy.orm import Session

user = APIRouter()
Base.metadata.create_all(bind=engine)
auth_handler = AuthHandler()

@user.post('/register', status_code=200)
def register(auth_details: UserSchema, db: Session = Depends(get_db)):
    select_user = db.query(User).filter(User.username == auth_details.username).first()
    if select_user:
        raise HTTPException(status_code=400, detail='Username is taken')
    hashed_password = auth_handler.get_password_hash(auth_details.password)
    query = User(
        username = auth_details.username,
        email = auth_details.email,
        first_name = auth_details.first_name,
        last_name = auth_details.last_name,
        phone_number = auth_details.phone_number,
        car_type = auth_details.car_type,
        password = hashed_password
    )
    db.add(query)
    db.commit()

    token = auth_handler.encode_token(query.email)
    return { 'token': token, 'user': {
        'id': query.id,
        'user_name': query.username,
        'first_name': query.first_name,
        'last_name': query.last_name,
        'email': query.email,
        'phone_number': query.phone_number,
    }}

@user.post('/login')
def login(auth_details: UserLoginSchema, db: Session = Depends(get_db)):
    select_user = db.query(User).filter(User.email == auth_details.email).first()
    if (select_user is None) or (not auth_handler.verify_password(auth_details.password, select_user
    .password)):
        raise HTTPException(status_code=401, detail='Invalid email and/or password')
    token = auth_handler.encode_token(select_user.email)
    return { 'token': token, 'user': {
        'id': select_user.id,
        'user_name': select_user.username,
        'first_name': select_user.first_name,
        'last_name': select_user.last_name,
        'email': select_user.email,
        'phone_number': select_user.phone_number,
    }}
}
```

```

import jwt
from fastapi import HTTPException, Security
from fastapi.security import HTTPAuthorizationCredentials, HTTPBearer
from passlib.context import CryptContext
from datetime import datetime, timedelta

class AuthHandler():
    security = HTTPBearer()
    pwd_context = CryptContext(schemes=["bcrypt"], deprecated="auto")
    secret = 'SECRET'

    def get_password_hash(self, password):
        return self.pwd_context.hash(password)

    def verify_password(self, plain_password, hashed_password):
        return self.pwd_context.verify(plain_password, hashed_password)

    def encode_token(self, user_id):
        payload = {
            'exp': datetime.utcnow() + timedelta(days=1, minutes=5),
            'iat': datetime.utcnow(),
            'sub': user_id
        }
        return jwt.encode(
            payload,
            self.secret,
            algorithm='HS256'
        )

    def decode_token(self, token):
        try:
            payload = jwt.decode(token, self.secret, algorithms=['HS256'])
            return payload['sub']
        except jwt.ExpiredSignatureError:
            raise HTTPException(status_code=401, detail='Signature has expired')
        except jwt.InvalidTokenError as e:
            raise HTTPException(status_code=401, detail='Invalid token')

    def auth_wrapper(self, auth: HTTPAuthorizationCredentials = Security(self.security)):
        return self.decode_token(auth.credentials)

```

5.3.2 Parkings Api Routes

```
from select import select
import qrcode
from typing import List
import os
import collections
from urllib import request
from uuid import UUID
from fastapi import APIRouter, HTTPException, Depends, Response
from config.db import *
from models.index import *
from schemas.index import *
from random import randint
import datetime
import json
import boto3
import requests
from PIL import Image, ImageTransform

from tensorflow.keras.models import load_model
from tensorflow.keras.preprocessing import image
import cv2
import numpy as np
import warnings
warnings.filterwarnings("ignore", category=UserWarning)

from fastapi.responses import FileResponse

from sqlalchemy.orm import Session

def get_cropped_images(image_path, coords):
    cropped_images = []
    for coord in coords:
        im = Image.open(image_path)
        result2 = im.transform((96, 96), ImageTransform.QuadTransform(coord), resample=Image.BILINEAR)
        hh = np.array(result2)
        cropped_images.append(hh)
    return cropped_images

# Get the cropped images and forward them through the model

def preprocess_image(img_path, target_size=(224, 224)):
    img = image.load_img(img_path, target_size=target_size)
    x = image.img_to_array(img)
    x = np.expand_dims(x, axis=0)
    x = x / 255.0 # Scale pixel values to [0, 1]
    return x

parking = APIRouter()
Base.metadata.create_all(bind=engine)

def capture_and_save_frame(output_path):
    # Create a video capture object for the webcam
    video = cv2.VideoCapture(1, cv2.CAP_DSHOW)

    video.set(cv2.CAP_PROP_FRAME_WIDTH, 800)
    video.set(cv2.CAP_PROP_FRAME_HEIGHT, 600)

    # Check if the webcam was successfully opened
    if not video.isOpened():
        print("Error opening webcam")
        return None

    # Read a frame from the webcam
    ret, frame = video.read()

    # Check if the frame was successfully read
    if not ret:
        print("Error reading frame")
        return None

    # Save the frame as an image with the specified quality
    cv2.imwrite(output_path, frame)
    print("Frame captured and saved as", output_path)

    # Release the video capture object
    video.release()

    return output_path
```

```

@parking.post('/get_nearest_parking', status_code=200)
async def get_nearest_parking(parking: NearestParkingSchema, db: Session = Depends(get_db)):
    select_all_parkings = db.query(Parking).all()
    total_lat = 0.0
    total_lng = 0.0
    avg_lat = 0
    avg_lng = 0
    for x in select_all_parkings:
        total_lat += float(x.lat)
        total_lng += float(x.long)

    avg_lat = total_lat / len(select_all_parkings)
    avg_lng = total_lng / len(select_all_parkings)

    if(len(select_all_parkings)>0):
        return {
            'avg_location':{
                'lat':avg_lat,
                'lng':avg_lng
            },
            'data':select_all_parkings
        }

    else:
        raise HTTPException(status_code=404, detail="Parking not found")
    #ml model process

@parking.post('/choose_parking', status_code=200)
async def choose_parking(camera: CameraSchema, db: Session = Depends(get_db)):
    parking = db.query(Parking).filter(Parking.id == camera.parking_id).first()
    output_path = f'{parking.name}.jpg'
    image = capture_and_save_frame(output_path)

    print(image)
    model = load_model('model.h5')
    print('Model loaded!')
    coords = []
    with open("./coords.txt", 'r') as f:
        for line in f:
            line=line.split(":")[1]
            coords.append([int(x) for x in line.split(",")])

    cropped_images = get_cropped_images(image, coords)

    cropped_images_array = np.array(cropped_images)
    labels = model.predict(cropped_images_array)

    labels = np.round(labels).astype(int).flatten()

    predicted_labels_for_image = []
    for j in range(len(coords)):
        predicted_labels_for_image.append(labels[j])

    parking_slots = []
    allslots = db.query(Slot).filter(Slot.parking_id == camera.parking_id).all()
    for v in allslots:
        parking_slots.append(v.slot_index)

    json_data = {}

    for i in range(len(predicted_labels_for_image)):
        parking_slot = parking_slots[i]
        json_data[parking_slot] = predicted_labels_for_image[i]

    free_slots = {}

    for x ,v in json_data.items():
        select_slot = db.query(Slot, Request).join(Slot).filter(Slot.parking_id == camera.parking_id , Slot.slot_index == x ).first()
        if(v==1):
            if(select_slot is not None):
                if(select_slot.Request.status == 0):
                    free_slots.update({
                        x : {
                            'id':select_slot.Slot.id,
                            'status': 'Pending'
                        }
                    })
                elif(select_slot.Request.status == 1):
                    free_slots.update({
                        x : {
                            'id':select_slot.Slot.id,
                            'status': 'Unavalabile'
                        }
                    })
            else:
                select_slot_free = db.query(Slot).filter(Slot.slot_index == x, Slot.parking_id == camera.parking_id).first()
                free_slots.update({
                    x : {
                        'id':select_slot_free.id,
                        'status': 'Free'
                    }
                })
        else:
            select_slot_unav = db.query(Slot).filter(Slot.slot_index == x, Slot.parking_id == camera.parking_id).first()
            free_slots.update({
                x : {
                    'id':select_slot_unav.id,
                    'status': 'Unavalabile'
                }
            })

    od = collections.OrderedDict(sorted(free_slots.items()))
    slot_list = [value for key, value in od.items()]

    return {
        'slots' : slot_list
    }

```

```

@parking.post('/choose_slot',status_code=200)
async def choose_slot(slot: ChooseSlotSchema,db:Session=Depends(get_db)):
    getSlot = db.query(Slot).filter(Slot.id == slot.slot_id).first()
    if(getSlot is None):
        raise HTTPException(status_code=404, detail='Slot is not found')

    select_request = db.query(Request).filter(Request.slot_id == slot.slot_id,Request.status >= 0 ).first()

    if(select_request is None):
        return {"msg":'Successful reservation'}

    return {'msg':'This slot is not available now, Choose another one'}

s3 = boto3.client('s3',
aws_access_key_id='AKIA2EELWDBYRWSMZYYD',
aws_secret_access_key='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',
region_name='eu-central-1'

)
bucket_name = 'elba5t-stored-images'

def qrGenrator(status,user_id,slot_id,payment_id,request_id):
    data = {
        "user_id": user_id,
        "slot_id":slot_id,
        "payment_id":payment_id,
        "request_id":request_id,
        "status":status
    }
    print(json.dumps(data))
    now = datetime.datetime.now()
    qr = qrcode.QRCode(
        version=1,
        box_size=15,
        border=5
    )
    qr.add_data(json.dumps(data))
    qr.make(fit=True)
    img = qr.make_image(fill='black',back_color='white')
    range_start = 10**(5-1)
    range_end = (10**5)-1
    num = randint(range_start, range_end)

    name = str(user_id)+'_'+str(slot_id)+'_'+str(num)+'.png'
    img.save('qrcodes/'+name)

    s3.upload_file('qrcodes/' + name, bucket_name, 'qrcodes/' + name,ExtraArgs={'ACL': 'public-read'})

    return name

@parking.post('/payment',status_code=200)
async def payment(payment: PaymentSchema,slot: ChooseSlotSchema,db:Session=Depends(get_db)):
    payment = Payment(
        user_id = payment.user_id,
        method = payment.method,
        amount = payment.amount,
        status = payment.status,
        total_time = payment.total_time,
    )

    db.add(payment)
    db.commit()

    request = Request (
        payment_id = payment.id,
        slot_id = slot.slot_id,
        status = 0,
        started_at = 0
    )
    db.add(request)
    db.commit()

    name = qrGenrator(0,payment.user_id,slot.slot_id,payment.id,request.id)
    addQRCode = Qrcode (
        image = name,
        request_id = request.id,
        status = 0,
    )

    db.add(addQRCode)
    db.commit()

    return {'payment_id':payment.id}

```

```

@parking.get('/myresrvations/{id}', status_code=200)
async def get_my_reservations(id: int, db:Session=Depends(get_db)):
    myallrequests = db.query(Payment,Request,Qrcode,Slot).join(Payment).join(Qrcode).join(Slot).filter(Payment.user_id == id).order_by(
    Payment.id.desc()).filter(Request.status <= 3).all()
    return myallrequests

@parking.get('/get_complete_payment/{id}', status_code=200)
async def get_complete_payment(id: int,db:Session=Depends(get_db)):
    request = db.query(Payment,Request,Qrcode,Slot).join(Payment).join(Qrcode).join(Slot).filter(Payment.id == id).first()
    if(request is None):
        raise HTTPException(status_code=404, detail='Request is not found')
    now = datetime.datetime.now().time()
    specified_time = datetime.datetime.combine(datetime.date.today(), request.Request.started_at)
    formatted_time = datetime.datetime.combine(datetime.date.today(), now)

    if specified_time > formatted_time:
        formatted_time += datetime.timedelta(days=1)

    time_difference = formatted_time - specified_time
    # Specify a time to compare with the current time

    # Calculate the difference between the two times
    minutes_difference = int(time_difference.total_seconds() / 60)

    # print(time_difference)
    total_amount = (30*minutes_difference)/60
    # # Print the time difference in seconds

    return {
        'payment_id':request.Payment.id,
        'slot':request.Slot.slot_index,
        'price_hour':round(total_amount/(minutes_difference/60)),
        'total_time':round(minutes_difference/60) ,
        'total_amount':total_amount
    }

@parking.post('/complete_payment', status_code=200)
async def complete_payment(payment: LastPaymentSchema,db:Session=Depends(get_db)):
    request = db.query(Payment,Request,Qrcode).join(Payment).join(Qrcode).filter(Payment.id == payment.payment_id).first()
    if(request is None):
        raise HTTPException(status_code=404, detail='Request is not found')
    # # Print the time difference in seconds
    request.Payment.total_time = payment.total_time
    request.Payment.amount = payment.total_amount
    db.commit()
    name = qrGenerator(1,request.Payment.user_id,request.Request.slot_id,payment.payment_id,request.Request.id)
    request.Qrcode.image = name
    request.Qrcode.status = 1
    db.commit()
    request.Request.status = 2
    db.commit()

    return {'msg':'Successful Payment',
    'status' : 200}

@parking.get('/get_qrcode/{id}', status_code=200)
async def get_qrcode(id: int,db:Session=Depends(get_db)):
    request = db.query(Payment, Request, Qrcode).join(Payment).join(Qrcode).filter(Payment.id == id).filter(Payment.request_payment.any(
    )).first()
    if not request:
        raise HTTPException(status_code=404, detail='Request not found')
    key = f'qrcores/{request.Qrcode.image}'
    url = s3.generate_presigned_url('get_object', Params={'Bucket': bucket_name, 'Key': key})
    response = requests.get(url)
    print(response)
    return url

```

5.3.3 Search algorithm

```
def get_distance(lat1, lon1, lat2, lon2):
    R = 6371000.0

    lat1_rad = radians(lat1)
    lon1_rad = radians(lon1)
    lat2_rad = radians(lat2)
    lon2_rad = radians(lon2)

    dlon = lon2_rad - lon1_rad
    dlat = lat2_rad - lat1_rad

    a = sin(dlat / 2)**2 + cos(lat1_rad) * cos(lat2_rad) * sin(dlon / 2)**2
    c = 2 * atan2(sqrt(a), sqrt(1 - a))

    distance = R * c
    return distance
```

```
locations = select_all_parkings
close_locations = [
    location for location in locations
    if get_distance(
        parking.lat,
        parking.lng,
        float(location.lat),
        float(location.long)
    ) <= 500 # Filter locations within 2000 meters (adjust as needed)
```

5.3.4 Gate Scanner

```
import cv2
from pyzbar.pyzbar import decode
from sqlalchemy.orm import sessionmaker
from models.index import *
from schemas.index import *
from config.db import *
import json
import datetime

cap = cv2.VideoCapture(0) # 0 represents the default webcam, change it if necessary

# Read the image
parking_id = 1
# slot_id
slot_id = 11
Session = sessionmaker(bind=engine)
session = Session()
global status
status = 0

# Decode the QR code

def inQrCodeScanner(json):
    global status
    request = session.query(Request).filter(Request.payment_id == json['payment_id'], Request.status == 0).first()
    if(request is not None):
        if(request.slot_id == slot_id):
            print('Correct Slot')
            usercheck = session.query(Payment).filter(Payment.id == json['payment_id']).first()
            if(usercheck is not None):
                if(usercheck.user_id == json['user_id']):
                    request.status = 1
                    request.started_at = datetime.datetime.now().time()
                    session.commit()
                    status = 1
                    print('Valid !')
                else:
                    print('Not same user')
            else:
                print('Payment not found')
        else:
            print('Not correct Slot')
    else:
        print('invalid Qrcode !')

def outQrCodeScanner(json):
    global status
    request = session.query(Request).filter(Request.payment_id == json['payment_id'], Request.status == 2).first()
    if(request is not None):
        if(request.slot_id == slot_id):
            print('Correct Slot')
            usercheck = session.query(Payment).filter(Payment.id == json['payment_id']).first()
            if(usercheck is not None):
                if(usercheck.user_id == json['user_id']):
                    # request.status = 3
                    session.query(Request).filter(Request.payment_id == json['payment_id'], Request.status == 2).delete()
                    session.commit()
                    print('Deleted !')
                    status = 1
                else:
                    print('Not same user')
            else:
                print('Payment not found')
        else:
            print('Not correct Slot')
    else:
        print('invalid Qrcode !')

while True:
    # Read a frame from the webcam
    ret, frame = cap.read()

    # Decode the QR code from the captured frame
    decoded = decode(frame)

    # Iterate over the decoded QR codes
    for barcode in decoded:
        # Extract the bounding box coordinates of the QR code
        x, y, w, h = barcode.rect

        # Draw a green border around the QR code
        cv2.rectangle(frame, (x, y), (x+w, y+h), (0, 255, 0), 2)

        # Extract the QR code data
        res = barcode.data.decode('utf-8')
        json_data = json.loads(res)
        print(json_data)
        qrCode = session.query(Qrcode).filter(Qrcode.request_id == json_data['request_id']).first()
        if json_data['status'] == 0:
            inQrCodeScanner(json_data)
        elif json_data['status'] == 1:
            outQrCodeScanner(json_data)

    # Display the frame in a window
    cv2.imshow('QR Code Scanner', frame)
    if status == 1:
        break
    # # Wait for 'q' key to exit
    if cv2.waitKey(1) & 0xFF == ord('q'):
        break

# Release the VideoCapture and close the window
cap.release()
cv2.destroyAllWindows()
```

5.4 Testing

Name	Description	Test Result
Register new user	"Register & Login" user with validations and filters	Success
Get nearest parking	Get nearest parking based on direction	Success
Select parking	Choose a parking from the map and see it based on the parking ID	Success
Show slots of parking	Get parking slots based on parking ID with all statuses (pending - available - unavailable)	Success
Select slot	Select the slot from the list of slots and check if it is reserved or not if the reserved slot is not available	Success
Make Payment	We have simulated the payment process by specifying the reservation period and slot number	Success
Show QR code	Show QR code based on Payment	Success
QR code Scanner	QR Code Scanner works well and handles both status QR codes (inside and outside)	Success
Get history of user processes	Get all users processes such as (pending - in parking - end payment -cancel payment)	Success
Get user profile	Show user data such as name, phone number and email	Success
User Logout	Remove the token from the application and log the user out	Success



SPOTIT

Chapter 6

Conclusion and Future Work

6.1 Conclusion

A decentralized and efficient solution for visual detection of the parking status was presented, which exploits deep Convolutional Neural Networks (CNNs) to classify the parking space occupancy. The solution employs smart cameras built using Raspberry Pi platform equipped with a camera module. Each smart camera can simultaneously monitor up to fifty parking slots.

A deep CNN architecture designed to run on embedded systems such as smart cameras, is used to classify images of parking spaces as occupied or vacant directly on board of the smart camera. In this way, the only information that is sent to a central server for visualization is the binary output of the classification.

As a further contribution, we used CNRPark-EXT, a dataset containing images of a real parking lot taken by nine smart cameras, in different days, with different weather and light conditions. CNRPark-EXT contains images with high variability related to occlusions, point of views, illumination and weather conditions. This makes the dataset more compatible with a real scenario of an outdoor parking lot, and represents a good complement to other publicly available datasets, for more reliable assessments. Using Parking Lot Occupancy Database, CNRPark, CNRPark-EXT and PKLot make our approach very high accuracy, even in presence of noise due to light conditions variation, shadows, and partial occlusions.

6.2 Future Works

For future studies, we plan to develop our service with the capability to guide drivers to a specific parking slot. This feature will enable users to easily locate and navigate to their desired parking space. By providing step-by-step directions and real-time guidance, we aim to simplify the parking process and improve the overall user experience. This feature will save time, reduce frustration, and enhance the convenience of parking for our users.

Also, We plan to create 3-D parking map visuals from parking spot occupancy. This planar view would show information about the parking lot with exact available parking spots in a parking lot.

References

- [1] Almeida, P., Oliveira, L. S., Silva Jr., E., Britto Jr., A. & Koerich, A. (2013). Parking Space Detection using Textual Descriptors. In 2013 IEEE International Conference on Systems, Man, and Cybernetics, (pp. 3603- 3608). Manchester, UK: IEEE.
- [2] B. Y. Cai, R. Alvarez, M. Sit, F. Duarte and C. Ratti, "Deep Learning-Based Video System for Accurate and Real-Time Parking Measurement," in IEEE Internet of Things Journal, vol. 6, no. 5, pp. 7693-7701, Oct. 2019, doi: 10.1109/JIOT.2019.2902887.
- [3] Basavaraju S R. (2015) .Automatic Smart parking System using Internet of Things (IOT).International Journal of Scientific and Research Publications, Volume 5, Issue 12, December 2015
- [4] Gandhi,B.M.K.,&Rao,M.K.(2016).APrototypeforIoTbasedCar Parking Management system for Smart cities. Indian Journal of Science and Technology, 9(17).
- [5] Patil, M., & Bhonge, V. N. (2013). Wireless sensor network and RFID for smart parking system. International Journal of Emerging Technology and Advanced Engineering, 3(4), 188-192.
- [6] Singh, H., Anand, C., Kumar, V., & Sharma, A. Automated Parking System With Bluetooth Access. International Journal Of Engineering And Computer Science ISSN, 2319-7242.
- [7] Orrie, O., Silva, B., & Hancke, G. P. (2015, November). A wireless smart parking system. In Industrial Electronics Society, IECON 2015- 41st Annual Conference of the IEEE (pp. 004110-004114). IEEE.
- [8] D. Acharya, W. Yan, and K. Khoshelham, "Real-time image-based parking occupancy detection using deep learning," in Proceedings of the 5th Annual Conference of Research@Locate, Adelaide, Australia, 2018, vol. 2087, pp. 33–40 [Online]. Available: <http://ceur-ws.org/Vol-2087/paper5.pdf>
- [9] S. Funck, N. Mohler, and W. Oertel, "Determining car-park occupancy from single images," in IEEE Intelligent Vehicles Symposium, 2004, Parma, Italy, 2004, pp. 325–328, doi: 10.1109/IVS.2004.1336403 [Online]. Available: <https://ieeexplore.ieee.org/document/1336403/>. [Accessed: Apr. 17, 2022]
- [10] S. Valipour, M. Siam, E. Stroulia, and M. Jagersand, "Parking-stall vacancy indicator system, based on deep convolutional neural networks," in 2016 IEEE 3rd World Forum on Internet of Things (WF-IoT), Reston, VA, USA, Dec. 2016, pp. 655–660, doi: 10.1109/WFIoT.2016.7845408 [Online]. Available: <http://ieeexplore.ieee.org/document/7845408/>. [Accessed: Apr. 17, 2022]

- [11] Bin, Z., Dalin, J., Fang, W., & Tingting, W. (2009). A Design of Parking Space Detector Based on Video Image. In 2009 9th International Conference on Electronic Measurement & Instruments, (pp. 253-256). Beijing, China: IEEE.
- [12] Yusnita, R., Norbaya, F. & Basharuddin, N. (2012). Intelligent Parking Space Detection System Based on Image Processing. *International Journal of Innovation, Management and Technology*, 3(3), 232-235.
- [13] Enríquez, F., Soria, L. M., & Álvarez-García, J. A. (2017). Existing Approaches to Smart Parking: An Overview. In *International Conference on Smart Cities (Smart-CT 2017)*, LNCS 10268, (pp. 63–74). Malaga, Spain: Springer.
- [14] Amato, G., Carrara, F., Falchi, F., Gennaro, C., & Vairo, C. (2016). Car Parking Occupancy Detection Using Smart Camera Networks and Deep Learning. In 2016 IEEE Symposium on Computers and Communication (ISCC), (pp. 1212-1217). Mersina, Italy: IEEE.
- [15] Mora, J. E. G., Lopera, J. C. L., & Cortes, D. A. P. (2018). Automatic visual classification of parking lot spaces: A comparison between bof and cnn approaches. In *Workshop on Engineering Applications* (pp. 160–170). Springer.
- [16] Dhuri, V., Khan, A., Kamtekar, Y., Patel, D., & Jaiswal, I. (2021). Real-time parking lot occupancy detection system with vgg16 deep neural network using decentralized processing for public, private parking facilities. In 2021 International Conference on Advances in Electrical, Computing, Communication and Sustainable Technologies (ICAECT) (pp. 1–8). doi:10.1109/ICAECT49130. 2021.9392506.
- [17] J. Nyambal and R. Klein, “Automated parking space detection using convolutional neural networks,” in 2017 Pattern Recognition Association of South Africa and Robotics and Mechatronics (PRASA-RobMech), Bloemfontein, South Africa, Nov. 2017, pp. 1–6, doi: 10.1109/RoboMech.2017.8261114 [Online]. Available: <http://ieeexplore.ieee.org/document/8261114/>. [Accessed: Apr. 17, 2022]
- [18] Nurullayev, S., & Lee, S. W. (2019). Generalized parking occupancy analysis based on dilated convolutional neural network. *Sensors*, 19(2), 277.
- [19] J. Cazamias and M. Marek, “Parking Space Classification using Convolutional Neural Networks,” Stanford University, Course Project Report [Online]. Available: http://cs231n.stanford.edu/reports/2016/pdfs/280_Report.pdf
- [20] P. R. L. de Almeida, L. S. Oliveira, A. S. Britto, E. J. Silva, and A. L. Koerich, “PKLot – A robust dataset for parking lot classification,” *Expert Systems with Applications*, vol. 42, no. 11, pp. 4937–4949, Jul. 2015, doi: 10.1016/j.eswa.2015.02.009. [Online]. Available: <https://linkinghub.elsevier.com/retrieve/pii/S0957417415001086>. [Accessed: Apr. 17, 2022]

- [21] M. D. Learning, “MIT Deep Learning 6.S191,” MIT Deep Learning 6.S191. [Online]. Available: <http://introtodeeplearning.com>. [Accessed: Apr. 17, 2022]
- [22] K. Melcher, “Introduction to Convolutional Neural Networks and Computer Vision,” Low Code for Advanced Data Science, Aug. 20, 2021. [Online]. Available: <https://medium.com/low-code-foradvanced-data-science/introduction-to-convolutional-neural-networks-and-computer-vision72b2d85dd1c0>. [Accessed: Apr. 17, 2022]
- [23] “Using Deep Learning Models / Convolutional Neural Networks.” [Online]. Available: https://docs.ecognition.com/eCognition_documentation/User%20Guide%20Developer/8%20Classification%20-%20Deep%20Learning.htm. [Accessed: Apr. 17, 2022]
- [24] S. Saha, “A Comprehensive Guide to Convolutional Neural Networks — the ELI5 way,” Medium, Dec. 17, 2018. [Online]. Available: <https://towardsdatascience.com/a-comprehensive-guideto-convolutional-neural-networks-the-eli5-way-3bd2b1164a53>. [Accessed: Apr. 17, 2022]
- [25] J. Cazamias and M. Marek, “Parking Space Classification using Convolutional Neural Networks,” Stanford University, Course Project Report [Online]. Available: http://cs231n.stanford.edu/reports/2016/pdfs/280_Report.pdf
- [26] P. R. L. de Almeida, L. S. Oliveira, A. S. Britto, E. J. Silva, and A. L. Koerich, “PKLot – A robust dataset for parking lot classification,” Expert Systems with Applications, vol. 42, no. 11, pp. 4937–4949, Jul. 2015, doi: 10.1016/j.eswa.2015.02.009. [Online]. Available: <https://linkinghub.elsevier.com/retrieve/pii/S0957417415001086>. [Accessed: Apr. 17, 2022]
- [27] “Overfit and underfit | TensorFlow Core,” TensorFlow. [Online]. Available: https://tensorflow.google.cn/tutorials/keras/overfit_and_underfit. [Accessed: Apr. 17, 2022]
- [28] de Almeida, P. R., Oliveira, L. S., Britto, A. S., Silva, E. J., & Koerich, A. L. (2015). Pklot—a robust dataset for parking lot classification. Expert Systems with Applications, 42, 4937–4949.
- [29] Amato, G., Carrara, F., Falchi, F., Gennaro, C., & Vairo, C. (2016). Car parking occupancy detection using smartcamera networks and deep learning. In 21th IEEE Symposium on Computers and Communications (ISCC) (pp. 1212–1217). IEEE.
- [30] <https://sites.google.com/view/visionlearning/databases/parking-lot-database> (last visited June 2023)